

Solution Design - DA	3
TODO Checklist	4
Introduction	5
Scope and Assumptions	6
High level Architecture	7
Runtime Architecture	8
EDS Architecture	9
Author (Adobe Document Authoring)	10
DA 4 - Non-functional Requirements	11
DA SEO	12
DA Site Performance	13
DA Key Performance Indicators (KPIs)	14
DA Accessibility	20
DA Backup & Recovery	21
DA EDS Best practices	22
Development Design	28
Sites	29
Document Authoring (DA) — Authoring Model	30
Block Library	44
Accordion	45
Tabs	51
Hero	59
Text and Image block(Cards)	68
CTA	82
Breadcrumb	89
Anchor list	93
Column	96
Custom Table	101
Sub Navigation	110
Basic Table	113
Testimonial	121
Embed	125
Immersive	126
PDP Document Display	135
Carousel	141
Product Selection Guide	143
FeaturedCollection	148
Media Formulation	151
Product List	167
Video and Video playlist	177
Dynamic Merchandising Offer	186
Header and Footer -Design	197
Forms Solution	198
Form Authoring in DA/EDS	202
Forms — Rule Editor, Validations	240

Form - Request a Quote	248
Dynamic Dropdown, Cascading Options, Prefil	252
Forms - Recaptcha	263
Form – Render, Submission, and Document Upload	271
User Groups and Permissions	276
Workflows	277
Data Layer & Analytics	278
Integrations	279
Content Syndication - Integration with PDP	280
Brightcove	292
EDS Migration Strategy	299
Migration Strategy -- Forms	300
Non-functional Requirements	302
DA-SEO	303
EDS Site Performance	306
Key Performance Indicators	307
EDS Accessibility	314
Backup and Recovery	315
Best practices EDS	316
Testing Details	322

Solution Design - DA

TODO Checklist

- ☐ Block library to update DA
- ☐ Templates , Page Properties
- ☐ Permissions
- ☐ Configuration and Config Mapping in DA
- ☐ Governance Preflight
- ☐ Analytics and Target Integration
- ☒ SEO/parity
- ☐ Solution/Logical Architecture with DA
- ☒ Introduction text where out of sope, risks all are highlighted (summary)
- ☐ Migration strategy , reference components
- ☐ MSM, traslations,XF solution
- ☐ Workflows
- ☐ Assets and Asset migration
- ☐ Cutover, coexistence and rollback strategy (phased parallel run, go-live sequencing, rollback plan)
- ☐ Test strategy and CI quality gates

Introduction

Scope and Assumptions

High level Architecture

Runtime Architecture

EDS Architecture

Author (Adobe Document Authoring)

DA 4 - Non-functional Requirements

- [DA SEO](#)
- [DA Site Performance](#)
- [DA Accessibility](#)
- [DA Backup & Recovery](#)
- [DA EDS Best practices](#)

This section captures the platform capabilities and project-specific expectations for **SEO, site performance**, and **accessibility** when using **AEM Sites as a Cloud Service with Edge Delivery Services (EDS) and Universal Editor (UE)**.

Supported Browsers

- Internet Explorer is not supported.
- Latest version of modern edge browser Google Chrome, Microsoft Edge, Apple Safari, Mozilla Firefox are supported. [\[SD1\]](#)

Devices

Supported devices: Mobile, Tablet and Desktop

View Ports

```
/* Mobile-first (phones) */

@media (width < 768px) {

  /* Mobile styles */

}

@media (width >= 768px) and (width < 1200px) {

  /* Tablet & small laptop styles */

}

@media (width >= 1200px) {

  /* Desktop styles */

}
```

DA SEO

- The project will rely on **Edge Delivery Services' performance-first architecture** to support Web Vitals that influence search ranking:
 - Content is served from a global edge CDN with aggressive caching of HTML, assets, and APIs.
 - **Phased rendering** and lightweight client-side includes for shared elements (e.g., header/footer) ensure above-the-fold content is delivered quickly.

EDS best-practice guidance (“keeping it 100”) is followed for block implementations, images, and fonts.

DA Site Performance

- HTML, media assets, and static resources will be delivered via **Adobe-managed Edge Delivery Services CDN** (or an approved BYO CDN pattern), providing:
 - Global edge caching with automatic invalidation / revalidation on publish.
 - Optimized delivery of images and static assets (compression, modern formats where supported).
 - Support for customer CDN integration patterns where required.
- The implementation will follow **EDS performance best practices** for blocks, client-side code, fonts, and third-party scripts.
- Cloud Manager **Experience Audit** (Lighthouse) and/or EDS pre-flight checks will be used as a gate in the CI/CD pipeline.

We would also Suggest having current KPIs baselined, so that after Implementation we can compare these with parameters.

DA Key Performance Indicators (KPIs)

What are KPIs?

Key Performance Indicators (KPIs) are **quantifiable measures** that show how effectively we are achieving our business and digital experience objectives.

In our context, KPIs help us measure the impact of improvements to Thermo Fisher's **digital experience**.

Why do we need a KPI baseline?

A KPI baseline is the “**before**” **picture** – the current performance level *before* we introduce new capabilities, content, or optimizations. It will allow us to compare **before vs. after**.

Our KPI Framework Focus

1. Business KPI

Drives measurable growth through conversions, leads, and customer acquisition.

2. Digital Visibility KPI

Strengthens search presence and content discoverability.

3. Customer Experience KPI

Enhances user engagement and journey quality across all touchpoints

4. Operational Excellence KPI

Ensures a fast, stable, and scalable digital platform

Business KPIs				
Pillar	KPI	Description	Ownership Level	Values
Conversion	Overall Conversion Rate	% of sessions resulting in a conversion	Tertiary	
Conversion	Total Conversions	Total conversions tracked monthly/quarterly	Tertiary	

Lead Generation	Form Submissions	Completed forms submitted by users	Secondary	
Lead Generation	Form Starts	Initiated form interactions	Secondary	
Friction	Form Error Rate	% of form submissions with errors	Secondary	
Friction	Drop-off Rate	% of users abandoning funnel steps	Secondary	
Efficiency	Time to Publish	Time from creation to live	Primary	
Efficiency	Pages per Author	Avg pages published per author	Primary	
Efficiency	Content Volume	Total content produced-Pages and Assets	Primary	
Reusability	Component Reusability	% of reused components	Primary	
Workflow	Approval Cycle Time	Time taken for content approval	Primary	
Content velocity	Average Time to Create a Page	Time to create a standard page	Primary	
Content velocity	Average Time to Update a Page	Time to update an existing page	Primary	
Content velocity	Pages Updated per Year	Average no of pages updated per year	Secondary	

Customer Experience KPIs				
Pillar	KPI	Description	Ownership Level	Values
Behavior	Bounce Rate	% of users leaving after one page	Secondary	
Behavior	Engagement Rate	GA4 metric for meaningful interactions	Secondary	
Behavior	Pages per Session	Average number of pages viewed per visit	Secondary	
Behavior	Avg Session Duration	Time spent per session	Secondary	
Behavior	Avg Time per User	Time spent per unique user	Secondary	
Interaction	CTA Click Rate	% of users clicking CTAs	Secondary	
Device	Mobile vs Desktop Split	Traffic breakdown by device type	Secondary	
Device	Engagement by Device	Engagement metrics segmented by device	Secondary	
Consumption	Page Views per User	Avg pages viewed per user	Secondary	

Digital Visibility KPIs				
Pillar	KPI	Description	Ownership Level	Values

Traffic	Organic Traffic	Visits from search engines	Secondary	
Traffic	Organic Traffic per Page	Avg organic traffic per page	Tertiary	
Visibility	Keyword Rankings	Position of target keywords	Tertiary	
Visibility	Indexed Pages	Pages indexed by search engines	Primary	
Migration Health	Redirect Success Rate	% of successful redirects post-migration	Primary	
Migration Health	Broken Links / Crawl Errors	404 errors and crawl issues	Primary	
Performance	Core Web Vitals	LCP, CLS, INP scores for UX quality	Primary	
Distribution	Top Pages Traffic Share	% of traffic to top pages	Secondary	
Quality	High Bounce Pages	% of pages with high bounce rates	Secondary	
Conversion	Content Conversion Rate	Conversion rate from content pages	Secondary	
Journey	Homepage Path Depth	Avg depth of paths from homepage	Secondary	
Journey	Content → CTA Click Rate	% of CTA clicks from content pages	Secondary	

Contribution	Content → Form Contribution	% of forms initiated from content	Secondary	
--------------	-----------------------------	-----------------------------------	-----------	--

Operational Excellence KPIs				
Pillar	KPI	Description	Ownership Level	Values
Speed	Page Load Time	Avg time to load a page	Primary	
Speed	Time to First Byte	Server response time	Primary	
Stability	Platform Uptime	% of time site is operational	Primary	
Stability	System Stability	Frequency of crashes/issues	Primary	
Errors	404 / 403 / 503 Errors	Error types encountered by users	Primary	

We ran a report on the home page using Page Insight and obtained the following results. These data are indicative, averaged from three runs. Results may vary under different conditions. The tool's actual data is the most reliable baseline. Hence, it would be great to get that data from tool.

KPI	Desktop	Mobile	Unit
Largest Contentful Paint (LCP)	2.7	2.7	s
Interaction to Next Paint (INP)	62	95	ms
Cumulative Layout Shift (CLS)	0	0.04	

First Contentful Paint (FCP)	1	1	s
Time to First Byte (TTFB)	0.7	0.6	s
Performance score	84	68	0-100
Accessibility score	79	88	0-100
Best Practices score	96	96	0-100
SEO score	85	85	0-100
First Contentful Paint (FCP)	0.4	2.1	s
Largest Contentful Paint (LCP)	2.5	9.4	s
Total Blocking Time (TBT)	110	80	ms
Cumulative Layout Shift (CLS)	0	0	
Speed Index	1.3	5.7	s

DA Accessibility

Edge Delivery Services provides Semantic and Accessible markup by OOTB. We will to follow the OOTB approach as well as ensure Accessibility is maintained at the same level as the current .

Details on how the semantic markup is created in Edge Delivery Services by default is documented here: [Markup, Sections, Blocks, and Auto Blocking](#)

DA Backup & Recovery

Self-Service Restore: Cloud Manager provides a self-service restore process that copies data from Adobe system backups and restores it to its original environment

Types of Backups:

Point-In-Time (PIT): Restores from continuous system backups from the last 24 hours.

Last Week: Restores from system backups in the last seven days, excluding the previous 24 hours

Selective Content Restoration: Options include reinstalling packages, using the Restore Tree function, re-uploading original files, and restoring previous versions of assets.

References:

1.  [Restore Content in AEM as a Cloud Service | Adobe Experience Manager](#)

DA EDS Best practices

During implementation Adobe will make sure that below best practices are followed, these have to be adhered during authoring as well.

1. **Lean, Semantic Markup**

- Use Standard, Meaningful HTML
- Prefer semantic elements such as header, main, section, article, nav, and footer.
- Avoid unnecessary wrapper div elements.
- Use ARIA only when native HTML semantics are insufficient.
- Keep DOM structure shallow to improve performance and maintainability.

2. **Avoid Deep Fragment Nesting**

- Do not embed full fragments inside other fragments unless strictly required.
- Keep fragments modular, focused, and purpose-driven.

3. **Accessibility by Default**

- Ensure proper heading hierarchy (h1 → h2 → h3).
- Always provide alt text for images.
- Ensure interactive elements are keyboard accessible with visible focus states.

4. **Minimal and Focused JavaScript**

- Prefer Vanilla JavaScript

- Use native browser APIs wherever possible.
- Avoid introducing heavy frameworks for simple interactions.
- Reduce client-side payload to improve performance and stability.

5. Load Strategy

- Use defer or async for non-critical scripts.
- Avoid blocking rendering with synchronous scripts.
- Serve critical assets from the same host when possible.

6. Code Organization

- Keep scripts modular and component scoped.
- Avoid global variables.
- Ensure progressive enhancement where core functionality works without JavaScript.

7. Optimized Asset Delivery

- Images
 - o Use lazy loading for non-critical images.
 - o Use responsive image techniques.
 - o Compress and optimize all assets before deployment.
- Icons
 - o Prefer lightweight SVG usage.
 - o Avoid large icon libraries when only a few icons are needed.
- Performance Monitoring
 - o Maintain performance budgets.

- o Validate performance in pull requests.
- o Monitor Core Web Vitals metrics.

8. **Consistency and Modularity**

- Block-Based Authoring
- Each block must be self-contained.
- Avoid hidden cross-dependencies between blocks.
- Keep styling scoped to the block.

9. **Naming Conventions**

- Use consistent naming methodology such as BEM.

10. **Reusability**

- Prefer composable components over duplicated code.
- Maintain shared utilities carefully to avoid tight coupling.

11. **Code Quality Enforcement**

- ESLint (JavaScript / TypeScript)
- Detect bugs early and maintain consistent coding standards.
- Use a shared base ESLint configuration.
- Integrate ESLint into local development, pre-commit hooks, and CI pipelines.
- Fail CI builds on lint errors.
- Restrict console usage in production builds.

12. **Stylelint (CSS / SCSS)**

- Enforce consistent styling rules.

- Prevent specificity conflicts.
- Use a shared Stylelint configuration.
- Integrate Stylelint into pre-commit hooks, CI pipelines, and editor extensions.
- Limit nesting depth.
- Disallow use of !important.
- Enforce consistent naming patterns.
- Prevent duplicate properties.

13. **Design Tokens for CSS and Cross-Platform Consistency**

- What Are Design Tokens?

Centralized, reusable variables representing design decisions such as colors, typography, spacing, radius, shadows, opacity, and motion.

- Token Principles
 - o Use semantic, intent-based naming.
 - o Separate base tokens from semantic tokens.
 - o Version and document tokens with changelog and migration guidance.
- Implementation Guidelines
 - o Generate CSS variables from token definitions.
 - o Avoid hard-coded values in components.
 - o Maintain tokens in a central repository.
 - o Require design and engineering review for token changes.

14. **CI/CD Quality Gates**

- All pull requests must pass ESLint.
- All pull requests must pass Stylelint.
- Performance benchmarks must be validated.

- Accessibility checks must pass.
- Avoid unnecessary bundle size increases.

15. **Accessibility and Performance Standards**

- Accessibility
 - o Ensure keyboard navigation.
 - o Provide visible focus states.
 - o Maintain proper color contrast.
 - o Use semantic HTML before ARIA.
- Performance
 - o Avoid large JavaScript bundles.
 - o Reduce unused CSS.
 - o Minimize render-blocking resources.
 - o Monitor Core Web Vitals continuously.

16. **Generic Engineering Best Practices**

- Follow progressive enhancement.
- Keep CSS specificity low.
- Avoid over-engineering solutions.
- Prefer clarity over cleverness.
- Document each component's purpose and usage.
- Maintain a clear folder structure.
- Track and manage technical debt.

17. **Quick Implementation Checklist**

- Semantic HTML structure implemented.
- Minimal vanilla JavaScript used.
- Lazy-loaded assets enabled.
- Modular block-based architecture followed.
- ESLint integrated locally and in CI.
- Stylelint integrated locally and in CI.
- Centralized design tokens implemented.
- Accessibility validated.
- CI quality gates enabled.

Development Design

Sites

Document Authoring (DA) — Authoring Model

1. Introduction to DA Authoring

1.1 What is Document Authoring (DA)?

Document Authoring (DA) is a web-based content authoring environment provided by Adobe at **da.live**. It is Adobe's document-based authoring option for **Edge Delivery Services (EDS)** and is best positioned as an alternative to traditional AEM page authoring for suitable EDS projects.

In DA, authors work in a document-style editor that resembles a simplified word processor. Content is authored as structured text, tables, images, and links, which EDS transforms into fully rendered web pages at delivery time.

1.2 How Authoring Differs from Traditional AEM

Aspect	Traditional AEM (6.x)	Document Authoring (DA)
Editor	AEM Sites Editor (Touch UI) with component dialogs	Web-based document editor at da.live
Content storage	JCR (Java Content Repository) with structured node trees	DA-managed documents rendered in an HTML-based structure for EDS
Component authoring	Drag component onto page, fill dialog fields, save to JCR	Type content and build tables in a document
Preview	AEM Preview mode or Publisher	EDS preview via <code>.page</code> URLs

1.3 Authoring Philosophy

The DA/EDS authoring philosophy centers on three principles:

1. **Content over configuration** — Authors focus on writing content, not configuring component properties. The content structure itself drives the rendering.

2. **Tables as the primary block-authoring pattern** — In standard document-style authoring, most blocks are represented as tables in the document. The table header names the block, and the rows contain the content or configuration. This is the main pattern authors need to learn.
 3. **Simplicity by design** — Complex component dialogs with dozens of fields are often replaced by intuitive table structures. Configuration that was previously spread across dialog tabs is either simplified, moved to styling conventions, or handled by the block's implementation at delivery time.
-

2. DA Authoring Model

2.1 Page Structure

A DA page is composed of **sections** separated by horizontal rules (---). Each section can contain:

- **Default content** — Text, headings, images, and links authored directly (not inside a table). This renders as standard HTML content.
- **Blocks** — Structured content authored as tables. Each table represents a block (component) that EDS processes and renders.

```
1 [Section 1]
2   Heading, paragraph, image      ← default content
3   ---                            ← section separator
4 [Section 2]
5   Block table (e.g. Accordion)   ← block content
6   ---
7 [Section 3]
8   Default content + Block table  ← sections can contain both
9
```

2.2 How Blocks Are Represented

In standard document-based authoring, a block is commonly authored as a **table** in the DA document. The structure typically follows a consistent pattern:

- **Row 1 (header):** Block name — identifies which block this is
- **Rows 2+:** Content rows — each row provides data for the block

Example — a simple block:

Columns	
Left column content	Right column content

Example — a block with key-value configuration:

Hero	
image	/media/hero-banner.jpg
title	Accelerating Scientific Discovery
subtitle	Explore our portfolio of solutions
cta-text	Learn More
cta-link	/products/overview

2.3 How Rows and Columns Map to Content

The meaning of rows and columns depends on the block's design:

Pattern A: Each row is a repeatable item Used when a block contains a list of similar items (e.g. accordion panels, cards, carousel slides).

Accordion	
Panel 1 Title	Panel 1 body content
Panel 2 Title	Panel 2 body content
Panel 3 Title	Panel 3 body content

Each row = one item. Column position determines the role (column 1 = title, column 2 = body).

Pattern B: Each row is a named property Used when a block has distinct configuration fields (e.g. hero, form container, embed).

Form Container	
action	/api/submit
method	POST
thankyou	/thank-you

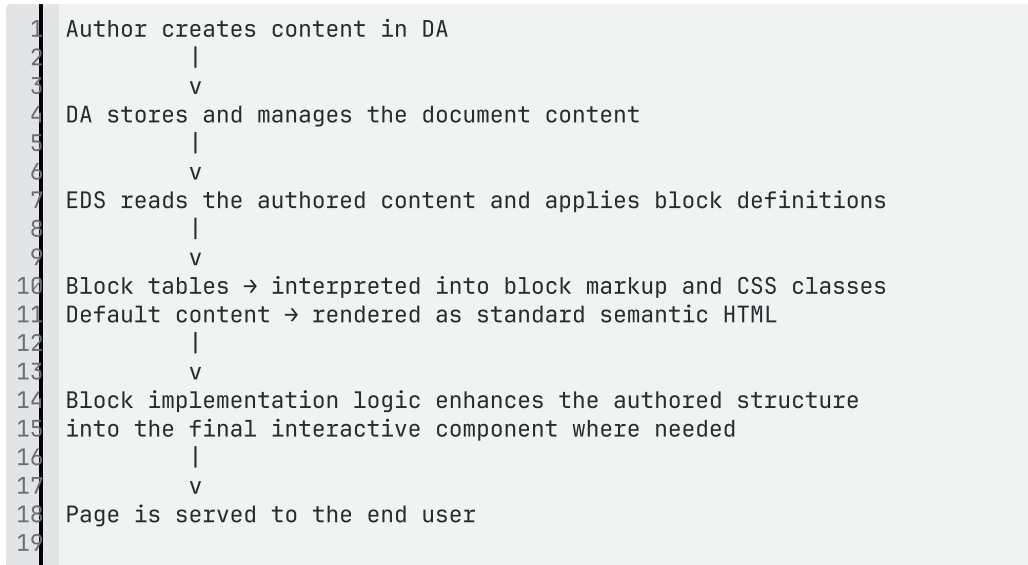
Each row = one property. Column 1 = property name, column 2 = value.

Pattern C: Grid or multi-column layout Used when content needs to be arranged in columns.

Columns (3)		
Column 1 content	Column 2 content	Column 3 content

Number of columns in the table = number of rendered columns.

2.4 How DA Content Becomes an EDS Page



At a high level, the block implementation is responsible for reading the authored structure and turning it into the final rendered component. The exact implementation can vary , but the main idea remains the same: authors create structured content and the implementation renders the final experience.

3. Variants and Configuration

3.1 How Variants Work

In traditional AEM, component variants are selected via a **Style System dropdown** or dialog field that writes a class to JCR. In DA, variants are authored directly in the **block name** using parentheses.

Syntax:

```

1 Block Name (variant)
2

```

Examples:

Author types	CSS classes applied	Rendered as
Accordion	.accordion	Default style

Accordion (icon-left)	<code>.accordion.icon-left</code>	Icon on the left
Accordion (classic-gray)	<code>.accordion.classic-gray</code>	Gray background variant
Hero (large)	<code>.hero.large</code>	Large hero variant
Cards (horizontal)	<code>.cards.horizontal1</code>	Horizontal card layout

Multiple variants can be combined:

Cards (horizontal, bordered)	<code>.cards.horizontal.bordered</code>
------------------------------	---

3.2 Section-Level Styling

Sections themselves can have metadata that controls their styling. A **Section Metadata** table at the end of a section applies configuration to the entire section:

Section Metadata	
style	dark-background

This adds the class `dark-background` to the section's `<div>`, allowing CSS to style the entire section (background color, text color, spacing, etc.).

3.3 Configuration Patterns

For blocks that require configuration beyond just content, there are two patterns:

Inline configuration — properties as rows in the same block table:

Hero	
image	<code>/media/banner.jpg</code>
title	Welcome
autoplay	true

Companion block — a separate configuration block:

Used when configuration is complex or shared. For example, the form system uses a separate `tfs-form-rules` block alongside the main `tfs-form` block.

tfs-form	
action	/api/submit

tfs-form-rules	
1-target	state
1-action	show
1-cond-1	country~contains~IN

4. Common Authoring Concepts

4.1 Blocks (Components)

In migration projects, many traditional AEM components are **re-modeled** as EDS **blocks**. A block is:

- Commonly authored as a **table** in the DA document
- Identified by the **table header** (block name)
- Rendered by a block implementation in JavaScript and CSS

AEM Concept	DA/EDS Equivalent
Component	Block
Component dialog	Table structure, guided authoring pattern, or Plugin UI for complex blocks
<code>cq:Component</code> resource type	Block definition / block name and implementation
Component HTL/JSP	Block implementation logic in JS
Component clientlib CSS	Block CSS file

This comparison is conceptual rather than strictly 1:1. Some AEM components map cleanly to a single block, while others need redesign or simplification for the EDS model.

4.2 Variants

See Section 3.1. Variants replace the AEM Style System. Instead of a dropdown in a dialog, the variant name is part of the block header. This is visible and explicit — authors always know which variant is in use.

4.3 Nested and Complex Content

DA tables support rich content within cells — including formatted text, images, links, and lists. For content that cannot be represented within a table cell (multi-column layouts, videos, other blocks), the **fragment reference pattern** is used:

1. Author creates the complex content as a separate DA page (a fragment)
2. In the parent block's table cell, author places a **link** to the fragment page
3. At render time, the block JS fetches the fragment's content and renders it inline

This is a reusable content pattern in DA that can serve a similar purpose to Experience Fragment or fragment inclusion patterns, without requiring the same AEM-specific product constructs.

4.4 References, Links, and Assets

Content Type	How to Author in DA
Internal page link	Type or paste the page path as a link
External URL	Type or paste the full URL as a link
Image	Drag and drop, paste, or reference according to the setup
PDF or document	Upload via DA or link to the managed asset, depending on setup
Fragment reference	Link to the fragment page path in a table cell

Assets can be handled directly in DA, but they may also be integrated with broader asset management patterns such as AEM Assets. DA should therefore not be described as a full DAM replacement; it is the authoring interface, while enterprise asset governance may still be handled separately.

4.5 Multi-Column and Structured Layouts

Multi-column layouts are authored using the **Columns** block:

Columns		
Content for column 1	Content for column 2	Content for column 3

The number of table columns determines the layout columns. Each cell can contain rich content.

4.6 Reusable Content

Reusable content patterns in DA can serve similar business purposes to familiar AEM constructs, but they should not be presented as exact product equivalents.

AEM Pattern	DA Equivalent
Experience Fragment	Reusable fragment page or referenced content pattern
Content Fragment	Structured content pattern such as spreadsheet/JSON or implementation-specific data model
Editable Template	Page pattern defined via metadata, section structure, and implementation rules
Shared component configurations	Spreadsheet-based or JSON-based shared configuration

These mappings are helpful as an orientation for AEM users, but they are not strict one-to-one replacements.

5. Comparison with Traditional AEM Authoring

5.1 Side-by-Side Comparison

Aspect	AEM 6.x Authoring	DA/EDS Authoring
Editor	AEM Sites Editor (Touch UI)	Web document editor (da.live)

Component placement	Drag from side panel onto parsys	Type content or build tables in document
Component configuration	Open dialog, fill fields across tabs	Content is the configuration — structured via table rows/columns
Variant selection	Style System dropdown in dialog	Variant name in block table header: Block (variant)
Nested components	Drag child components into parent parsys	Fragment reference pattern — link to fragment page
Content model	JCR nodes with typed properties	HTML tables with named rows/columns
Preview	AEM Preview mode	EDS preview URL
Publish	Replication to AEM Publisher	Publish via DA → live via EDS CDN
Permissions	AEM user/group ACLs	DA-level authoring access control, with preview/publish controls managed in the wider EDS model
Workflow	AEM Workflow engine	Lighter-weight review/publish controls and project-dependent approval patterns

5.2 What Authors Should Expect

Simpler day-to-day experience:

- No component dialogs with multiple tabs
- No drag-and-drop from a component side panel
- Content editing feels like editing a document

Different mental model:

- Components = tables. The table header is the component name.
- Configuration = rows in the table, not dialog fields
- Variants = part of the block name, not a separate dropdown
- No parsys concept — content flows naturally in the document

Familiar patterns retained:

- Rich text editing (bold, italic, links, lists, headings)
- Image insertion or asset reference patterns
- Preview before publish
- Page-level metadata

5.3 Complexity Comparison

Block Complexity	AEM Approach	DA Approach
Simple (text + image)	Component dialog: 2-3 fields	Table: 2-3 rows. Comparable effort
Medium (accordion, tabs)	Container component + child items, filters, model definitions	Single table with rows per item. Simpler
Complex (forms)	Multiple components, dialogs, server-side logic	Block tables + Plugin UI for guided authoring. Similar effort, different tooling

6. Authoring Experience and Governance

6.1 What Authors Control

Authors have direct control over:

- Page content — text, images, links, lists, headings
- Block selection — which blocks to use (by creating the appropriate table)
- Block variants — by specifying the variant in the block name
- Content order — by arranging blocks and content within sections
- Section structure — by using horizontal rules to define section boundaries

- Section styling — via Section Metadata block
- Page metadata — via Metadata block at the end of the document

6.2 What Is Controlled by Block Definitions

The following are governed by the development team, not by authors:

Governed by	Examples
Block JS (<code>decorate</code>)	How table content is transformed into rendered HTML, interactive behaviour, fragment fetching
Block CSS	Visual styling, spacing, colors, responsive behaviour per variant
Page-level scripts/styles	Global styles, fonts, navigation, footer
Spreadsheet data	Datasource options (countries, states), shared configuration

Authors cannot change how a block renders — only what content it renders. This ensures visual and behavioural consistency across the site.

6.3 How Consistency Is Maintained

Concern	Governance Mechanism
Visual consistency	Block CSS enforces design tokens (colors, typography, spacing)
Block usage guidance	DA Library panel shows available blocks with descriptions
Valid variants	Only variants with matching CSS render differently — unknown variants are harmless (no visual change)
Content structure	Block JS expects a specific table structure — incorrect tables degrade gracefully

Reusable patterns	Fragment pages provide shared content without duplication
Configuration data	Spreadsheets (JSON) centralize options and settings

6.4 Plugin-Assisted Authoring

For blocks with complex authoring requirements (e.g. forms with validation rules, field types, and conditional logic), a **DA Plugin** can provide a guided authoring experience:

- Author clicks the block in the DA Library panel
- A plugin UI presents a structured form/dialog
- Author fills in fields, selects options, configures behavior
- Plugin writes the correctly structured block table into the document

This provides a more guided authoring experience while maintaining the underlying DA content model. The plugin is an authoring convenience; the authored DA structure remains the source of truth.

6.5 Universal Editor (UE) Overlay with DA

DA can support an optional Universal Editor (UE) overlay for supported implementations.

At a high level:

- project definitions describe how supported blocks can be edited
- UE provides a more guided editing surface on top of the DA content model
- the underlying DA-authored structure remains the basis for delivery

This should be positioned as a UI enhancement for supported scenarios, not as a different delivery model.

6.6 Clarification: UE with AEM vs UE with DA

A common misconception is that using Universal Editor (UE) with DA provides the same authoring model as UE with **AEM as the content source. This is not the case.** UE adapts to the underlying content source, so the authoring depth, content model, and supported editing patterns differ.

UE is the editor shell — the content source determines what the editor can do.

Aspect	UE + AEM (as authoring source)	UE + DA (as authoring source)
Content storage	AEM repository / JCR-backed content model	DA-managed document-based content model
Content model	Rich AEM content structures and AEM-managed relationships	Document-based structures such as sections, tables, links, and supported guided editing patterns
Authoring behavior	Driven by AEM content definitions and AEM-managed authoring capabilities	Driven by DA content structures and the supported UE mapping for that project
Field/model complexity	Typically broader and more AEM-native	Typically simpler and aligned to DA's content model
Container / nesting model	Can rely on AEM-managed parent-child relationships	Usually aligned to DA's flatter document-oriented structures
Picker / reference behavior	Can leverage AEM-native selectors and references	Depends on the supported DA implementation and setup

So while the editor may look similar at the shell level, the behavior is not identical.

6.7 Summary — What Does Not Change Between UE + AEM and UE + DA

	Same?	Notes
Editor shell	Broadly similar	UE provides a consistent editing shell conceptually

Delivery goal	Yes	Both approaches ultimately support content delivered through EDS
Underlying content model	No	AEM-backed UE and DA-backed UE are built on different content models
Authoring depth and field behavior	No	Depends on the capabilities of the content source
Project definitions	Yes, but different in practice	Both need project definitions, but the structure and complexity differ
Assumed parity with classic AEM authoring	No	UE with DA should not be presented as the same as AEM-backed authoring

Block Library

Accordion

1. Block Overview

Property	Value
Block Name	Accordion
Maps to Existing	FAQ List (v1), FAQ Items, Accordion, Accordion Item
Description	A vertically stacked list of collapsible panels. Each panel is a title + body pair that expands and collapses on click.
Authoring Strategy	Flattened block — single table, each row is one collapsible panel
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Panel title — required. The clickable header for each panel. Plain text only — rich text formatting (bold, italic, links) is not supported in titles.

Panel body — required. The content revealed when the panel is expanded. Authored either as rich text directly in the table cell, or as a link to a fragment page for complex content.

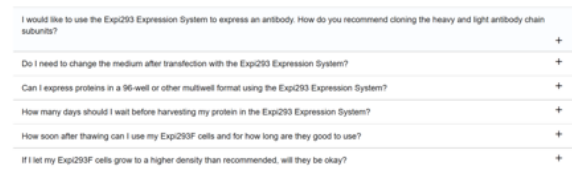
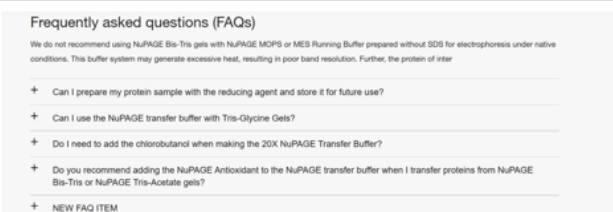
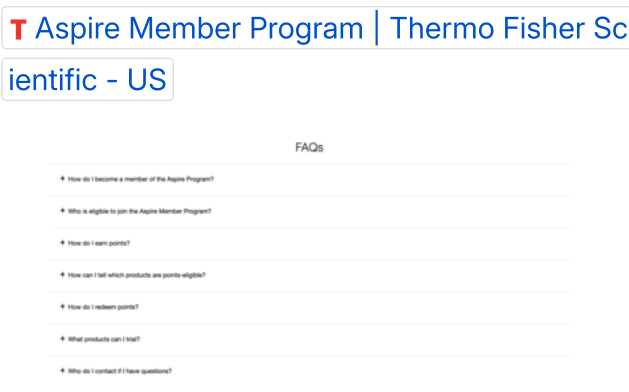
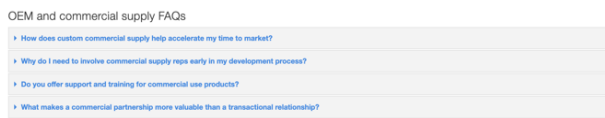
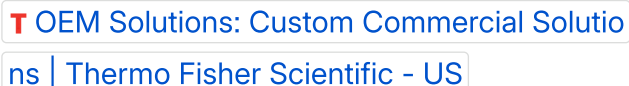
Author can add as many title + body pairs as needed. Each pair renders as one collapsible panel.

The accordion block must support the following content types within panel bodies:

- Rich text (paragraphs, headings, bold, italic, links, lists)
- Images
- Columns
- Tables
- Videos
- Product List

Simple content (rich text, images) is authored directly in the table cell. Complex content (columns, videos, forms, product list) is authored as a separate fragment page and referenced via a link.

3. Variants

Variant	Description	Reference
default		
icon-left	Plus sign is to the left of the accordion heading	
classic-small	alternate style	
classic-gray	Gray background with blue text. Normally occurs with a single accordion item.	 

Sample URLs showcasing content variants that can be included inside an accordion panel

T [Reducing Hazardous Materials and Waste | Thermo Fisher Scientific - US](#)

T [Advancing Neuronal Research | Thermo Fisher Scientific - US](#)

T [Attune Flow Cytometer Features | Thermo Fisher Scientific - US](#)

4. DA Block Table Contract

4.1 Table Structure

The accordion is authored as a **single block table** in the DA document. Each row represents one collapsible panel.

Column	Role	Content
Column 1	Panel Title	The clickable header text for the collapsible panel
Column 2	Panel Body	Rich text content directly, OR a link to a fragment page

4.2 Block Header and Variants

The first row of the table is the **block header** — it identifies the block and optionally specifies a variant. This row is not rendered as a panel.

Authors select variants by including the variant name in parentheses in the block table header row.

Author types in header row	Resulting CSS classes	Variant applied
Accordion	.accordion	Default
Accordion (icon-left)	.accordion.icon-left	Icon left
Accordion (classic-small)	.accordion.classic-small	Classic small

Accordion (classic-gray)	<code>.accordion.classic-gray</code> <code>c-gray</code>	Classic gray
-----------------------------	---	--------------

Content rows start from row 2 onward. Each content row is one collapsible panel.

4.3 Example — Simple Content (FAQ-style)

Accordion	
What is Western Blotting?	Western blotting is a technique used to detect specific proteins in a sample. It involves separating proteins by gel electrophoresis.
How does it work?	The process involves three steps: separation , transfer , and detection .
What equipment do I need?	You will need a gel electrophoresis system, transfer apparatus, and detection reagents. See our product guide .

4.4 Example — Fragment Content (Complex panels)

Accordion	
Product Specifications	/fragments/accordion/product-specs
Technical Details	/fragments/accordion/tech-details

4.5 Example — Mixed (Simple + Fragment)

Accordion (icon-left)	
What is it?	A simple text explanation with formatting and links .
Product Specifications	/fragments/accordion/product-specs

Frequently Asked Questions	Visit our FAQ page or contact support .
----------------------------	---

4.6 Authoring Summary

Concern	Approach
Block type	Flattened block — single table
Adding a panel	Author adds a row to the table
Removing a panel	Author deletes a row
Reordering panels	Author moves rows up/down
Simple content	Rich text authored directly in column 2
Complex content	Link to a fragment page in column 2
Variant selection	Variant name in block table header: Accordion (variant)

5. Authoring Patterns

5.1 Pattern 1 — Simple Content (Rich Text)

When to use: Panel body contains text, links, images, or lists — content that fits naturally in a DA table cell.

How to author: Type or paste content directly into column 2 of the block table. DA supports rich text formatting within table cells including bold, italic, links, lists, and inline images.

Suitable for: FAQ pages, text-only explanations, simple content with links.

5.2 Pattern 2 — Complex Content (Fragment Reference)

When to use: Panel body requires content that cannot be represented in a single table cell — such as multi-column layouts, embedded videos, product lists, or combinations of multiple blocks.

How to author:

1. Create a new DA page to hold the complex content (e.g. `/fragments/accordion/product-specs`)
2. Author the complex content on that page using any blocks — Columns, Video, Product List, etc.
3. In the accordion table, place a link to this fragment page in column 2

How the block identifies a fragment reference:

The block checks column 2 for all three of these conditions:

- The cell contains only a single link element
- The link points to an internal path (starts with `/`)
- No other text or elements are present in the cell

If all three are met → the block treats it as a fragment reference and fetches the content at render time. Otherwise → the cell content is rendered directly as the panel body.

Suitable for: Product specifications, comparison tables, video content, multi-column layouts inside a panel.

5.3 Pattern 3 — Mixed

A single accordion block can contain both simple and fragment panels. There is no restriction — each row is evaluated independently. Authors choose the appropriate pattern per panel based on content complexity.

A fragment page can also be referenced from multiple accordion panels across different pages. If the fragment content is updated, all pages referencing it reflect the change on next publish.

Tabs

1. Block Overview

Property	Value
Block Name	Tab List
Description	A tabbed container that organizes content into switchable panels. Each tab is a title + body pair. Clicking a tab reveals its panel and hides the others.
Authoring Strategy	Section-based — each tab panel is a DA section with <code>tab-label</code> metadata. A Tab List block acts as the controller.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Tab label — required. The text displayed on the tab button. Authored as `tab-label` in the Section Metadata of each tab panel section. Plain text only.

Tab body — required. The content displayed when the tab is active. Authored directly in the tab panel section using any blocks — cards, tables, columns, videos, product lists, images, text.

Author can add as many tab panel sections as needed. Each section with a `tab-label` renders as one tab.

The tabs block must support the following content types within tab panels:

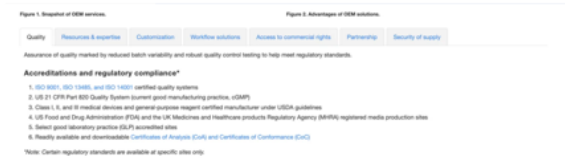
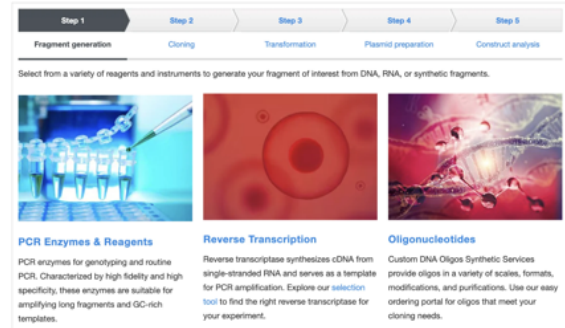
- Cards
- Tables
- Columns
- Videos
- Product Lists

- Images and rich text

Each content type is authored as a block directly inside the tab panel section — the same way blocks are authored in any regular section. No fragment pages are needed.

Mobile behaviour: On mobile devices, the tabbed interface collapses into an accordion layout for better usability.

3. Variants

Variant	Description	Reference
default		 T OEM Solutions: Custom Commercial Solutions Thermo Fisher Scientific - US
Classic-workflow		 T Molecular Cloning Essentials Thermo Fisher Scientific - US

Show All Option

The Tab List block supports an optional "Show All" feature. When enabled, a "Show All" button is prepended to the tab bar. Clicking it displays all tab panels simultaneously.

This is authored as a row in the Tab List block table:

Tab List	
show-all	true

4. DA Authoring Contract

4.1 Section-Based Pattern

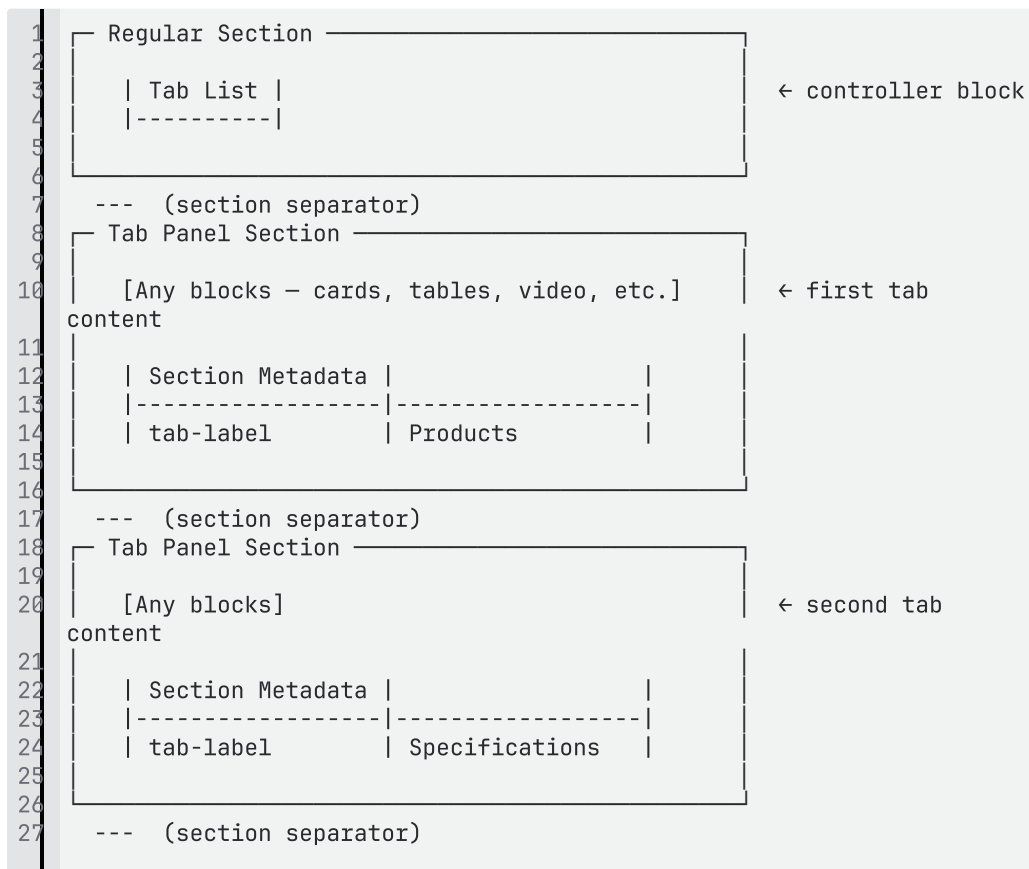
Tabs block uses a **section-based pattern**.

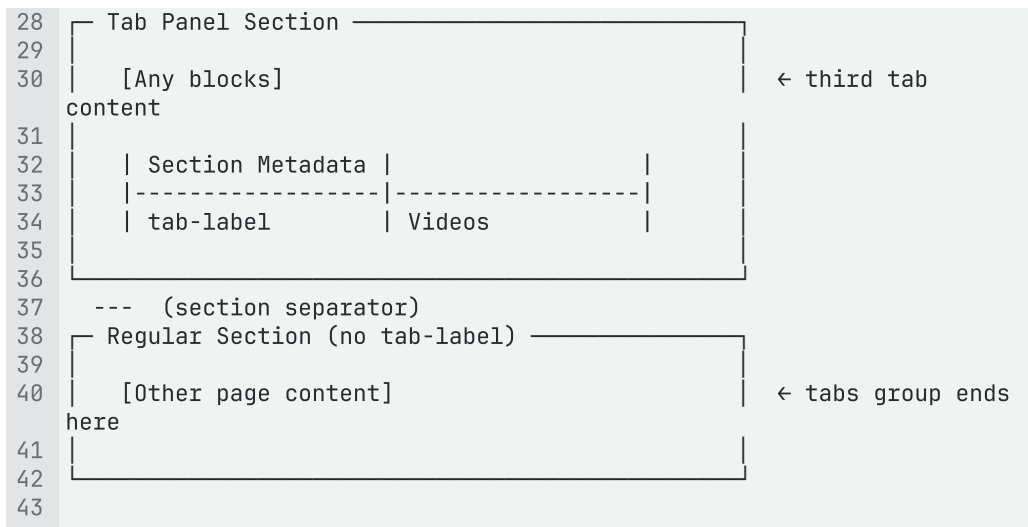
Two components work together:

Component	What It Is	What It Does
Tab List	A block placed inside a regular section	The controller — its JS discovers tab panel sections, generates horizontal tab buttons, and manages show/hide behaviour
Tab Panel	A regular DA section with <code>tab-label</code> in Section Metadata	The content — holds any blocks and provides the tab label

4.2 Page Structure

A tabs group in DA is structured as follows:





Key rules:

- The Tab List block must be in a section **before** the tab panel sections
- Each tab panel section must have a `tab-label` in its Section Metadata
- Consecutive sections with `tab-label` are grouped as tabs belonging to the Tab List above them
- A section without `tab-label` (or the end of the page) terminates the tabs group

4.3 Tab List Block Table

The Tab List block itself is a simple block table placed in the section before the tab panels. It acts as the controller and optionally holds configuration.

Authors select variants by including the variant name in parentheses in the Tab List block table header row.

Minimal (no configuration):

Tab List	
----------	--

With variant:

Tab List (classic-workflow)	
-----------------------------	--

With Show All option:

Tab List	
show-all	true

With variant and Show All:

Tab List (classic-workflow)	
show-all	true

4.4 Tab Panel Section Metadata

Each tab panel section uses the standard **Section Metadata** block to define the tab label. The Section Metadata table is placed at the end of the section (DA convention).

Section Metadata	
tab-label	Products

The `tab-label` value becomes the text on the tab button.

4.5 Complete Authoring Example

Below is a complete example of a 3-tab interface authored in DA — showing the full document structure as an author would see it.

Section 1 — Tab List controller:

Tab List	
----------	--

Section 2 — First tab panel (Products):

Cards	
Product A image	Product A description
Product B image	Product B description
Product C image	Product C description

Section Metadata	
tab-label	Products

Section 3 — Second tab panel (Specifications):

Table	
Specification	Value
Size	100ml
Weight	250g

Section Metadata	
tab-label	Specifications

Section 4 — Third tab panel (Videos):

Video	
source	https://www.youtube.com/watch?v=example

Section Metadata	
tab-label	Videos

Section 5 — Regular page content (tabs group ends):

Any content here is outside the tabs.

4.6 How the Tab List Block Discovers Tab Panels

At delivery time, the Tab List block JS:

1. Looks at sibling sections that follow the Tab List section
2. Identifies sections that have `tab-label` in their metadata (rendered as `data-tab-label` attribute on the section `<div>`)
3. Collects all consecutive tab-label sections as panels belonging to this Tab List

4. Stops when it encounters a section without `tab-label` or reaches the end of the page
5. Generates a horizontal tab bar with one button per panel, using the `tab-label` values as button text
6. Shows the first tab panel and hides the rest
7. Manages click and keyboard navigation to switch panels

The author does not need to wire anything — the discovery is automatic based on section position and metadata.

4.7 Authoring Summary

Concern	Approach
Block type	Section-based — Tab List controller + Tab Panel sections
Adding a tab	Author adds a new section with <code>tab-label</code> in Section Metadata
Removing a tab	Author deletes the tab panel section
Reordering tabs	Author moves sections up/down — tab order matches section order
Tab label	<code>tab-label</code> value in Section Metadata of each tab panel section
Tab content	Any blocks authored directly in the tab panel section
Variant selection	Variant name in Tab List block header: <code>Tab List (variant)</code>
Show All option	<code>show-all: true</code> row in Tab List block table

5. Authoring Patterns

5.1 Pattern 1 — Simple Tabs (Text and Images)

When to use: Each tab panel contains basic content — text, headings, images, or lists.

How to author: Add blocks or default content directly in each tab panel section. No special blocks needed — just type content or add images.

Suitable for: Product information tabs, FAQ tabs, overview pages.

5.2 Pattern 2 — Complex Tabs (Blocks Inside Panels)

When to use: Each tab panel contains structured block content — cards, data tables, videos, product lists, columns, or combinations.

How to author: Add any blocks inside the tab panel section, the same way you would add blocks to any regular page section. Each tab panel is a full section — all blocks are supported.

Suitable for: Product pages with tabs for specifications/videos/reviews, workflow pages with step-by-step content.

5.3 Pattern 3 — Mixed Tabs

A single tabs group can contain panels with different content types. One tab may have a Cards block, another may have a Table, and a third may have plain text. There is no restriction — each tab panel section is independent.

5.4 Workflow Tabs (classic-workflow variant)

When to use: Content represents a sequential workflow or process with numbered steps.

How to author: Use `Tab List (classic-workflow)` in the block header. Each tab panel represents one step in the workflow. The tab buttons render with numbered step indicators instead of plain text tabs.

Suitable for: Step-by-step guides, onboarding flows, experimental protocols.

5.5 Authoring Experience — What the Author Sees

At authoring time: The author sees vertically stacked sections in the DA document. The sections do not look like tabs — they appear as regular content sections one after another.

At delivery time: The Tab List block JS transforms these sections into a tabbed interface — generating horizontal tab buttons, hiding all panels except the active one, and managing click/keyboard navigation.

This is expected behaviour. The DA editor shows the content structure; the EDS delivery layer transforms it into the interactive tabbed component.

Hero

1. Block Overview

Property	Value
Block Name	Hero
Maps to Existing	Page Heading Hero Component
Description	A full-width banner that introduces the page with a background image, heading, and optional call-to-action.
Authoring Strategy	Flattened block — key-value pair table. Each row is a named property. Only include rows for fields that are needed.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

H1 Heading — derived automatically from page metadata (page title). Authors do **not** author the heading inside the hero block. The block JS reads the page title and renders it as the `<h1>` inside the hero.

Background image — optional. The hero background image. Authored by dropping an image into the value cell.

Foreground image — optional. A secondary image displayed in front of the background (e.g. product image, logo). Used with the `foreground-image` variant.

Subtitle — optional. Secondary text displayed below the heading. Plain text only.

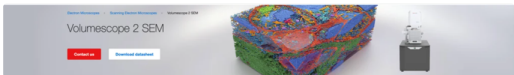
Description — optional. Supporting body copy below the subtitle. Supports rich text (bold, italic, links, lists).

CTA (Call-to-Action) — optional. 0, 1, or 2 CTA buttons. Each CTA has a text label, link URL, and style (primary or outline).

Breadcrumb overlay — optional. When enabled, a breadcrumb trail is displayed overlaid on the hero.

All fields are optional. Authors include only the rows they need — omitted rows mean that element is not rendered.

3. Variants

Variant	Description	Reference
default	Text is black and aligned to the left	
dark-background	Text is white	
center-align	Text is centered on the hero	
foreground-image	Includes both a background image and a foreground image	 T 3D SEM Volumescope 2 Scanning Electron Microscope Thermo Fisher Scientific - US
gradient-overlay	Dark gradient overlay on top of background image	 T Environmental Thermo Fisher Scientific - US
image-focal-center		

		T Carrier Screening Information The rmo Fisher Scientific - US
--	--	---

The Hero block supports multiple variants that can be **combined**. Authors list multiple variants separated by commas in the block header.

Combining Variants

Variants can be combined by listing them with commas in the block header:

Author types	CSS classes applied
Hero	.hero (default)
Hero (dark-background)	.hero.dark-background
Hero (dark-background, center-align)	.hero.dark-background.center-align
Hero (foreground-image, gradient-overlay)	.hero.foreground-image.gradient-overlay
Hero (dark-background, center-align, gradient-overlay)	.hero.dark-background.center-align.gradient-overlay

Authors select the combination that matches the desired visual treatment for the page.

4. DA Block Table Contract

4.1 Key-Value Structure

The Hero block uses a **key-value pair** pattern. Each row has a property name in column 1 and its value in column 2. Only include rows for fields that are needed — omit rows for fields that are not applicable.

4.2 Available Properties

Key (Column 1)	Value (Column 2)	Required	Description
image	Image (drag and drop)	Optional	Background image for the

			hero
image-alt	Text	Optional	Alt text for the background image
foreground-image	Image (drag and drop)	Optional	Foreground image — used with foreground-image variant
foreground-image-alt	Text	Optional	Alt text for the foreground image
subtitle	Plain text	Optional	Secondary text below the H1 heading
description	Rich text	Optional	Supporting body copy — supports bold, italic, links, lists
breadcrumb	true	Optional	Enables breadcrumb trail overlay on the hero
primary-cta	Link — [Text] (URL)	Optional	Primary CTA button
outline-cta	Link — [Text] (URL)	Optional	Secondary CTA button with outline style

4.3 Block Header and Variants

The first row of the table is the block header. It identifies the block and optionally specifies one or more variants.

Author types in header row	Resulting CSS classes
Hero	.hero
Hero (dark-background)	.hero.dark-background
Hero (dark-background, center-align)	.hero.dark-background.center-align
Hero (foreground-image, gradient-overlay)	.hero.foreground-image.gradient-overlay

Content rows (key-value pairs) start from row 2 onward.

4.4 How the Block JS Processes the Table

1. Reads the block header to determine variants
2. Iterates through each row — reads the key from column 1 and the value from column 2
3. Builds the hero HTML: background image as the hero backdrop, H1 from page metadata, subtitle, description, CTA buttons, and optional breadcrumb
4. If a key is not present, that element is simply not rendered

5. CTA Authoring

5.1 How CTAs Work

CTAs are authored as links in the value cell. The key name determines the button style.

Key	Button Style	Visual
primary-cta	Filled/solid button	Prominent, high-contrast action button
outline-cta	Outlined/bordered button	Secondary, less prominent action button

5.2 How to Author a CTA

The value cell contains a standard DA link — the link text becomes the button label and the URL becomes the button destination.

Key	What to type in value cell
primary-cta	Select text, add link → [Learn More](/products)
outline-cta	Select text, add link → [Watch Video](/video)

5.3 CTA Combinations

Authors can include 0, 1, or 2 CTAs:

No CTA: Simply omit both primary-cta and outline-cta rows.

One CTA (primary only):

primary-cta	Learn More
-------------	----------------------------

One CTA (outline only):

outline-cta	Watch Video
-------------	-----------------------------

Two CTAs:

primary-cta	Learn More
outline-cta	Watch Video

The primary CTA renders first (left), the outline CTA renders second (right).

6. Authoring Examples

6.1 Example 1 — Standard Hero with Dark Background

A common hero with a dark background image, subtitle, description, and a single CTA.

Reference: Standard product landing page hero with white text over a dark image.

Hero (dark-background)	
image	

image-alt	Laboratory researcher using microscope
subtitle	Accelerating Scientific Discovery
description	Explore our portfolio of life science research solutions designed to advance your work.
primary-cta	Explore Solutions

What renders:

- Full-width dark background image
- H1 heading pulled from page title (white text due to `dark-background` variant)
- Subtitle below heading
- Description paragraph
- One primary CTA button

6.2 Example 2 — Centered Hero with Dual CTA and Breadcrumb

A centered hero with gradient overlay, two CTA buttons, and breadcrumb navigation.

Reference: Campaign landing page with centered text, gradient for readability, and dual action buttons.

Hero (dark-background, center-align, gradient-overlay)	
image	
image-alt	Scientists collaborating in laboratory
subtitle	Protecting Sample Integrity
description	Learn how proper sample storage and handling can improve your research outcomes.

breadcrumb	true
primary-cta	Download Guide
outline-cta	Contact Sales

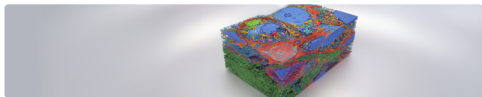
What renders:

- Full-width background image with dark gradient overlay
- Breadcrumb trail at the top of the hero
- H1 heading centered (from page title)
- Subtitle and description centered
- Two CTA buttons side by side — primary (filled) and outline (bordered)

6.3 Example 3 — Hero with Foreground Image

A hero with both a background and a foreground product image.

Reference: Product showcase page (e.g. 3D SEM Volumescope) with product image overlaid on the hero background.

Hero (foreground-image, dark-background)	
image	
image-alt	Abstract blue gradient background

foreground-image	
foreground-image-alt	Thermo Scientific Volumescope 2 SEM
subtitle	3D SEM Volumescope 2
description	Explore the next generation of scanning electron microscopy.
primary-cta	Request a Demo

What renders:

- Full-width background image
- Foreground product image positioned prominently (typically right-aligned)
- H1 heading from page title with white text
- Subtitle, description, and one primary CTA on the left

Text and Image block(Cards)

1. Block Overview

Property	Value
Block Name	Cards
Maps to Existing	Text and Image Component
Description	A repeatable content block that displays cards in a configurable grid. Each card can contain an image, title, subtitle, description, and a call-to-action. Supports multiple layout and visual treatment variants.
Authoring Strategy	Flattened block — 3-column table. Each row is one card.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Image — required. A visual associated with the card. Authored by dragging and dropping an image into column 1.

Title (H2) — optional. The card heading. Authored as bold text or heading in column 2.

Subtitle (H3) — optional. Secondary text below the title. Authored as text in column 2.

Description (rich text) — optional. Supporting body copy. Supports bold, italic, links, and lists. Authored in column 2.

CTA — optional. A call-to-action button with label, link, style, icon, and open-in-new-window option. Authored in column 3 using the CTA plugin.

Author can add as many cards as needed. Each row in the table renders as one card.

If a column is left empty, that element is not rendered. For example, a card with only an image and CTA (no text) will render as a clickable image card.

Adaptive Card Behavior

When a card contains only an image and a CTA link (no title or description), the block wraps the image in the CTA link, making the entire card clickable. No separate variant is needed — the block adapts based on which fields are present.

Background Color

The cards block supports configurable background colors applied to the entire block instance. Background color is authored as a variant in the block header.

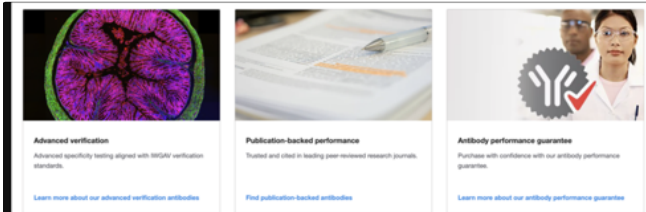
Color	Variant name	Description
White	(default — no variant needed)	White/transparent background
Light Grey	light-grey	Light grey background (#f5f5f5)
Blue	blue	Brand blue background
Dark	dark	Dark background with light text

Mobile Behavior

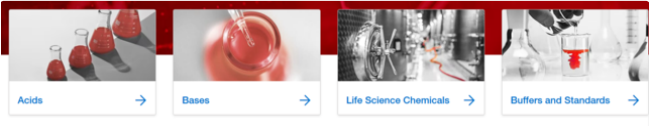
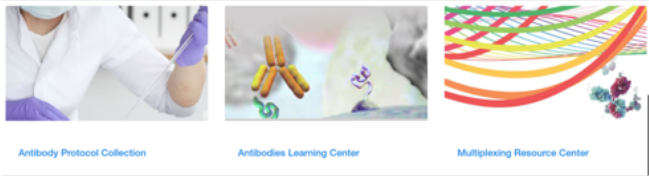
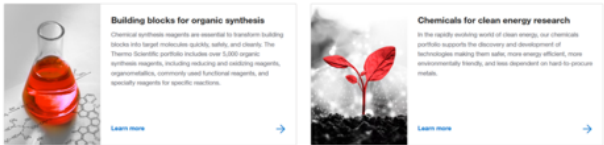
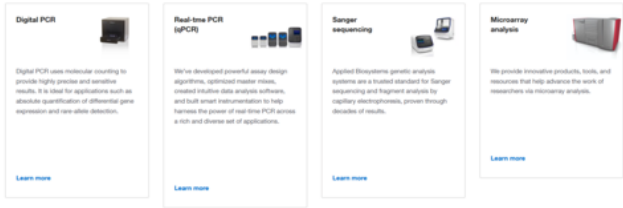
On mobile devices, cards stack vertically into a single-column layout regardless of the desktop grid variant.

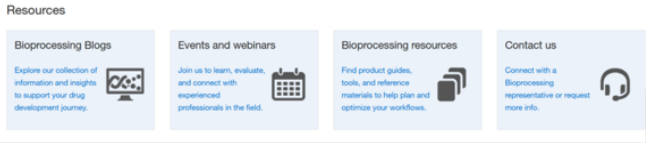
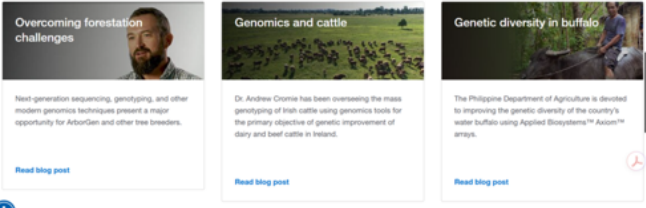
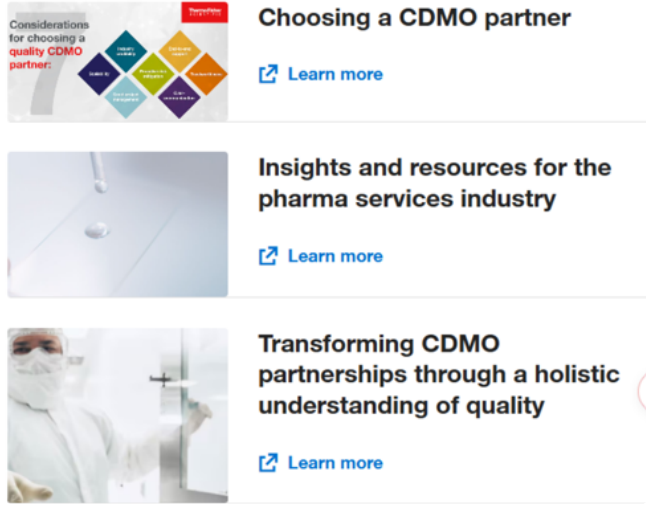
3. Variants

The Cards block supports multiple variants organized in two categories: **Layout** and **Visual Treatment**. Authors combine one layout variant with one or more visual treatment variants in the block header.

Variant	Description	Reference
default	Image at the top, gray border Default 3 col layout	 Antibodies Thermo Fisher Scientific - US

		 BenchStable Media Gibco BenchStable Media, available in our most common cell culture media formulations, are stable at room temperature, allowing for convenient and flexible storage without utilizing cold storage space. Explore Gibco BenchStable Media  Human Plasma-Like Medium Gibco Human Plasma-Like Medium (hPLM) is formulated to mimic the metabolic profile of human plasma, allowing for improved physiological relevance when studying cancer and disease. Explore Gibco Human Plasma-Like Medium  Media with GlutaMAX Supplement Media with Gibco GlutaMAX Supplement are standard cell culture formulations containing a stabilized form of L-glutamine that prevents degradation and buildup of ammonia. Explore GlutaMAX Media formulations
full-width-img-left	card is in full width and image left One column	Sustainability insights  Green Chemistry: Designing for a Sustainable Future In this episode of the Science With a Twist podcast, our host Vinod Mehandani welcomes John Warner, the co-founder, president, and CTO of the Warner-Babcock Institute for Green Chemistry, and co-founder of the non-profit, Beyond Benign. They talk about the role of green chemistry in sustainability, the importance of education, and why we need the right tools to build a world where the chemical building blocks of products used every day are healthier and safer for humans and the environment. Listen now  Get the facts. Find details on all green product claims for each of our greener product lines. We take sustainable product design seriously and believe all greener product claims should be transparently documented. For each of our greener product lines, we provide a green fact sheet that explains and substantiates the greener product claim(s) related to that product line. Browse complete list of greener products T Reducing Hazardous Materials and Waste Thermo Fisher Scientific - US
full-width-img-right	card is in full width and image right One Column	Supporting biopharma, biotech, and CDMOs We offer bioprocessing solutions for biopharma, biotech, and CDMOs to help streamline scale-up, navigate regulatory complexities, and tackle supply challenges, helping maintain consistent quality and productivity at every stage. Discover our industries →  T Bioprocessing Solutions Thermo Fisher Scientific - US
4-col	4 column layout	 Less waste and use of fewer resources When reducing waste in your lab, every little bit helps. Whether it's reducing, eliminating or reusing. Learn more  More energy efficient More energy efficient products help power your work and support lab sustainability. Learn more  Responsibly packaged and shipped Reducing packaging, replacing non-recyclable materials and shipping products at ambient temperatures when feasible. Learn more  Extending life of products and materials Innovating ways to extend the useful life of products and materials. Learn more T Reducing Hazardous Materials and Waste Thermo Fisher Scientific - US

clickable	Entire card is clickable with CTA link	
without-border	Without any borders or shadows	
img-left	Image to the left and text to the right 2 Column	 T Lab Chemicals Thermo Fisher Scientific - US
img-right	Image to the right and text to the left 2 Column	 T Why choose us? Thermo Fisher Scientific - US
feature-card	Product image in upper right corner	 T Applied Biosystems Thermo Fisher Scientific - US
overlay	Image as full-card background with text and CTA overlaid	 T Applied Biosystems Thermo Fisher Scientific - US

	Option for Selecting Text- White and Black	
card- resource Feature Product	Cards with Blue Background	 T Why choose us? Thermo Fisher Scientific - U S
title- over-img		 T Applied Biosystems Solutions for Agrigenomic s Thermo Fisher Scientific - US
list		 T Patheon Thermo Fisher Scientific - US

4. DA Block Table Contract

4.1 Table Structure

The Cards block is authored as a **3-column table** in the DA document. Each row represents one card.

Column	Role	Content
Column 1	Image	Card image — drag and drop into cell
Column 2	Text Content	Title (bold or heading), subtitle, description — rich text
Column 3	CTA	Call-to-action — authored using the CTA plugin

4.2 Block Header and Variants

The first row of the table is the **block header** — it identifies the block and specifies variants. This row is not rendered as a card.

Authors combine variants by listing them with commas. Select **one layout variant** and **one or more visual treatment variants** as needed.

Format: Cards (layout-variant, visual-variant, background-color)

Examples:

Author types in header row	Result
Cards	Default — 3-col, image top, gray border
Cards (4-col)	4 cards per row
Cards (4-col, feature-card)	4 cards per row with feature card look
Cards (2-col, img-left, without-border)	2-col, image left, no borders
Cards (overlay, overlay-light-text)	Image background with white text overlay
Cards (feature-card, blue)	Feature cards on brand blue background

Cards (fullwidth, img-left)	Full-width card, image on the left
Cards (4-col, clickable, dark)	4-col clickable cards on dark background

Content rows (cards) start from row 2 onward.

4.3 Layout Variants

Layout variants control the **grid structure** — how many cards per row. Select one layout variant. If none is selected, the default 3-column grid applies.

Variant	Value	Description
Default (3 col)	(none — baseline)	3-column grid
2 Column	2-col	2-column grid
4 Column	4-col	4-column grid
Full Width	fullwidth	1 card per row — full width

4.4 Visual Treatment Variants

Visual treatment variants control the **appearance** of each card — including image position, borders, overlay, and card style. These can be combined with a layout variant and with each other.

Image Position:

Variant	Value	Description
Default	(none — baseline)	Image at the top of the card
Image Left	img-left	Image positioned to the left, text to the right
Image Right	img-right	Image positioned to the right, text to the left

Card Style:

Variant	Value	Description
Without Border	<code>without-border</code>	No borders or shadows on cards
Clickable	<code>clickable</code>	Entire card wrapped in CTA link — card is fully clickable
Overlay	<code>overlay</code>	Image as full-card background, text + CTA overlaid
Overlay Light Text	<code>overlay-light-text</code>	White text on overlay variant
Overlay Dark Text	<code>overlay-dark-text</code>	Black text on overlay variant
Feature Card	<code>feature-card</code>	Product image in upper right corner, text prominent
Card Resource	<code>card-resource</code>	Blue background, feature product style
Title Over Image	<code>title-over-img</code>	Title text overlaid on the image
Classic Small	<code>classic-small</code>	Compact/alternate style
Classic Gray	<code>classic-gray</code>	Gray background with blue text, typically single item

4.5 How Layout and Visual Treatment Combine

Layout and visual treatment are **independent** — layout controls the grid, visual treatment controls the card appearance. They do not conflict.

What author wants	Layout	Visual	Block header
3-col standard cards	default	default	Cards
4-col feature cards	4-col	feature-card	Cards (4-col, feature-card)
3-col overlay with white text	default	overlay + overlay-light-text	Cards (overlay, overlay-light-text)
Full-width, image left, clickable, no borders	fullwidth	img-left + clickable + without-border	Cards (fullwidth, img-left, clickable, without-border)
2-col image left, blue background	2-col	img-left + blue	Cards (2-col, img-left, blue)
Full-width image right, dark background	fullwidth	img-right + dark	Cards (fullwidth, img-right, dark)

4.6 Authoring Summary

Concern	Approach
Block type	Flattened block — 3-column table
Adding a card	Author adds a row to the table
Removing a card	Author deletes a row

Reordering cards	Author moves rows up/down
Card image	Drag and drop into column 1
Card text	Rich text in column 2 — title (bold/heading), subtitle, description
Card CTA	Authored in column 3 using CTA plugin
Layout selection	Layout variant in block header
Visual treatment	Visual variant(s) in block header
Background color	Color variant in block header (light-grey, blue, dark)
Optional fields	Omit content in any column — that element is not rendered

5. CTA Authoring

5.1 Overview

Each card can have one CTA (call-to-action) button authored in column 3. The CTA is authored using the **CTA plugin** available in the DA library panel.

5.2 CTA Properties

The CTA plugin supports the following properties:

Property	Description
Label	The text displayed on the button (e.g. "Learn more")
Link	The destination URL
Style	Button style — Primary, Outline, Link
Icon	Icon displayed alongside the label (e.g. arrow, download, document, print)

Open in new window	Whether the link opens in a new browser tab
--------------------	---

5.3 How to Author a CTA

1. Place cursor in **column 3** of the card row
2. Open the **CTA plugin** from the DA library panel
3. Fill in the CTA properties: label, link, style, icon, open-in-new-window
4. Click **Add** — the plugin inserts the CTA content into the cell

The CTA plugin writes structured content into the cell that the block JS reads at delivery time to render the appropriate button with the correct style and icon.

5.4 CTA is Optional

If a card does not need a CTA, leave column 3 empty. The card will render without a button.

5.5 CTA Plugin — Standalone Use

The same CTA plugin can also be used outside the Cards block to create a standalone CTA button anywhere on the page. When used outside a table cell, the plugin creates a CTA block table instead of inserting inline content.

6. Authoring Examples

6.1 Example 1 — Default 3-Column Cards

Standard 3-column card layout with image, text, and CTA.

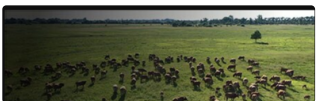
Cards		
	Building blocks for organic synthesis Chemical synthesis reagents are essential to transform building blocks into target molecules quickly, safely, and cleanly.	Learn more →
	Protein biology Explore our portfolio of	Learn more →

	protein biology products for expression, purification, and analysis.	
	Cell analysis Advanced tools and reagents for cell analysis workflows.	Learn more →

What renders: 3 cards side by side, each with image on top, title, description, and a CTA button at the bottom.

6.2 Example 2 — 4-Column Feature Cards with Blue Background

Feature cards in a 4-column grid on a blue background.

Cards (4-col, feature-card, blue)		
	Applied Biosystems Genetic analysis solutions	Explore →
	Invitrogen Cell biology reagents	Explore →
	Gibco Cell culture media	Explore →
	Ion Torrent Next-gen sequencing	Explore →

What renders: 4 feature cards per row with product images in the upper right corner, text prominent, on a blue background with white text.

6.3 Example 3 — Full Width Image Left

Single card per row with image on the left and text on the right.

Cards (fullwidth, img-left)		
	Lab Chemicals The Thermo Scientific portfolio includes over 5,000 organic synthesis reagents, including reducing and oxidizing reagents, organometallics, commonly used functional reagents, and specialty reagents for specific reactions.	Learn more →

What renders: Full-width card with image on the left side and title, description, CTA on the right side.

6.4 Example 4 — Overlay Cards with Light Text

Cards with image as full background and text overlaid.

Cards (overlay, overlay-light-text)		
	Life Sciences Research products for cell analysis, gene expression, and more.	Explore →
	Applied Sciences Customized kits and	Explore →

	solutions for production settings.	
	Clinical Diagnostics and translational research solutions.	Explore →

What renders: 3 cards with full-bleed background images. Title, description, and CTA are overlaid on the image with white text.

CTA

1. Block Overview

Property	Value
Block Name	CTA
Maps to Existing	CTA / Button Component
Description	A reusable call-to-action button supporting multiple visual styles and icons. Used both as a standalone block and inside other blocks (Cards, Hero, etc.).
Authoring Strategy	Flattened block (standalone) / Inline content (inside other blocks). Authored via the CTA plugin in both cases.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

Reusability

The CTA is designed as a **standardized component** used across the site:

- **Standalone** — CTA block placed directly on the page for inline actions, section-level actions, or independent call-to-actions
- **Inside other blocks** — CTA authored within Cards (column 3), Hero (`primary-cta` / `outline-cta` rows), and any other block that requires a call-to-action

The CTA plugin writes the same structured content format in all cases. This ensures consistent styling and behaviour across the entire site — regardless of where the CTA appears.

2. CTA Properties

Every CTA — whether standalone or inside another block — is defined by the same set of properties:

Property	Description	Default
----------	-------------	---------

Label	The text displayed on the button (e.g. "Learn more")	—
Link	The destination URL or content path	—
Style	Visual style of the button (see Section 3)	Primary
Icon	Icon displayed alongside the label (see Section 4)	None
Open in new window	Whether the link opens in a new browser tab	No (same window)

3. CTA Style Variants

The Style property controls the visual appearance of the CTA button.

Style	Value	Description
Primary	<code>primary</code>	Solid filled button — brand color background, white text. The most prominent action on the page.
Outline	<code>outline</code>	Border-only button — transparent background, brand color border and text. Secondary emphasis.
Link	<code>link</code>	Text link style with optional icon — no background, no border. Least prominent.

The red ‘play button’ (right arrow) is from classic. The blue play button to the left of the text is what we should use moving forward.

Additional styles will be added during implementation as encountered across the site.

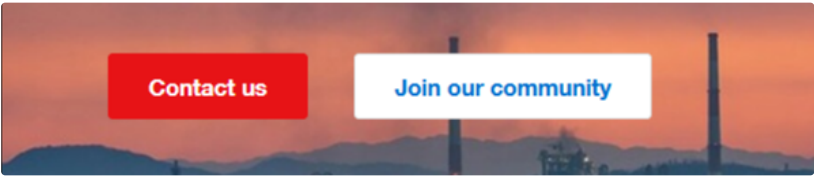


Watch the features of the CX7 Pro imaging platforms 

T HCS and HCA Platform Features | Thermo Fisher Scientific - US

-  Infographic: Five things to consider when improving ergonomics with biological safety cabinets
-  Video: Setting your stand height

T Biological Safety Cabinets Resources | Thermo Fisher Scientific - US



T Flow Measurement | Flow Meters | Flow Computers | Thermo Fisher Scientific - US

4. CTA Icons

The Icon property adds a visual indicator alongside the button label.

Icon	Value	Description
None	(empty)	No icon — label only
Arrow	<code>arrow</code>	Right-pointing arrow — used for navigation actions
Document	<code>document</code>	Document icon — used for document/resource links
Download	<code>download</code>	Download icon — used for downloadable

		assets
Print	print	Print icon — used for print actions

Additional icons will be added during implementation as encountered across the site.

Note: Per design direction, the blue play button (to the left of text) should be used — not the red classic right-arrow style.

5. DA Authoring — Standalone CTA

5.1 When to Use

Use the standalone CTA block when a call-to-action button is needed on its own — not inside a Cards, Hero, or other block. Common scenarios:

- Inline CTA within a content section
- Section-level action (e.g. "Contact Us" button after a content block)
- Independent call-to-action on the page

5.2 Block Table Structure

The standalone CTA is a **single-column block table** with one row. The CTA plugin writes structured content into the cell — the same content format used when CTA is authored inside other blocks (Cards, Hero, etc.).

CTA
(CTA plugin inserts content here)

The plugin writes label, link, style, icon, and external metadata as structured content inside the single cell. The content format is identical whether the CTA is standalone or inside another block.

5.3 How to Author

1. Add a **CTA block table** to the DA document (via library panel → Blocks → CTA)
2. Place cursor in the **content cell** of the table
3. Open the **CTA plugin** from the DA library panel
4. Fill in: Label, Link, Style, Icon, Open in new window
5. Click **Add** — the plugin inserts the CTA content into the cell

5.4 Which JS Decorates

The standalone CTA block table is decorated by `cta.js` (the CTA block's own JavaScript). It reads the CTA content from the cell and renders the appropriate button with the correct style, icon, and link behavior.

`cta.js` uses the same parsing logic that other block JS files (Cards, Hero) use to read CTA content — because the content format is the same everywhere.

6. DA Authoring — CTA Inside Other Blocks

6.1 When to Use

When a CTA is part of another block's content — such as a button on a card, or a call-to-action in a hero banner.

6.2 How It Works

The author places the cursor inside the **parent block's table cell** where the CTA belongs, then uses the CTA plugin to insert CTA content into that cell.

Parent Block	Where CTA goes	How to author
Cards	Column 3 of a card row	Place cursor in column 3 → use CTA plugin
Hero	Value cell of <code>primary-cta</code> or <code>outline-cta</code> row	Place cursor in value cell → use CTA plugin
Any future block	The designated CTA cell/row	Place cursor → use CTA plugin

6.3 Which JS Decorates

When the CTA is inside another block, the **parent block's JS** handles the rendering — not `cta.js`.

CTA location	Decorated by
Standalone CTA block table	<code>cta.js</code>
Inside Cards block	<code>cards.js</code>

Inside Hero block	<code>hero.js</code>
Inside any other block	That block's JS

There is no conflict because EDS block decoration is scoped — each block's JS only processes its own block element.

6.4 Consistent Content Format

The CTA plugin writes the **same structured content format** regardless of where the CTA is placed. This means:

- Every block's JS reads CTA content using the same parsing logic
- CTA styling and behaviour is consistent across the site
- Adding a new CTA style or icon option benefits all blocks immediately

This replaces the UE partials architecture (`_cta-fields.json` , `_dual-cta-fields.json` , `_button-types.json` , `_button-icons.json`) with a single plugin that serves all blocks.

7. CTA Plugin

7.1 Overview

The CTA plugin is a DA library panel tool that provides a guided authoring experience for creating CTA buttons. It replaces the AEM CTA Item dialog and the UE properties panel with a lightweight, consistent interface.

7.2 Plugin UI

When the author opens the CTA plugin, it presents the following dialog:

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

CTA
Close
Label *
Learn more
Link *
/products/overview
Style *
Primary
Icon

20

21

22

23

24

25

26

27

28

29

30

31

32

33

Arrow ▼

☐ Open in new window

Add to Page

Cancel

7.3 Plugin Fields

Field	Type	Required	Options
Label	Text input	Yes	—
Link	Text input	Yes	—
Style	Dropdown	Yes	Primary, Outline, Link
Icon	Dropdown	No	None, Arrow, Document, Download, Print
Open in new window	Checkbox	No	—

Breadcrumb

1. Block Overview

Property	Value
Block Name	Breadcrumb
Maps to Existing	Breadcrumb Component
Description	A navigational trail showing the current page's position within the site hierarchy. Links are generated dynamically based on the page's URL path.
Authoring Strategy	Metadata-driven — no block table authoring required. Controlled via page metadata and bulk metadata.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)
Variants	Default only

2. Authoring Criteria

No block table is authored for breadcrumbs. Authors do not create a breadcrumb block table on the page. Breadcrumbs are controlled entirely through metadata.

Breadcrumb links are generated dynamically based on the page's URL path. No manual link entry is required. The block JS reads the URL hierarchy and builds the navigational trail automatically.

Default behavior: Breadcrumbs are enabled by default (`true`). Authors only need to take action if they want to disable breadcrumbs on a specific page.

Page-level control: Authors can enable or disable breadcrumbs on individual pages using the `breadcrumbs` metadata property.

Bulk control: Breadcrumbs can be enabled or disabled across an entire subsection or site-wide using the bulk metadata spreadsheet.

Skip page from breadcrumb trail: A custom metadata property can be used to exclude a specific page from the breadcrumb navigation and adjust the starting breadcrumb position.

3. DA Authoring Contract

3.1 Page-Level Metadata

Breadcrumbs are controlled via the **Metadata** block at the end of the DA document.

Metadata	
breadcrumbs	true

Property	Values	Default	Description
breadcrumbs	true / false	true	Enables or disables the breadcrumb trail on the page

Since the default is `true`, authors only need to add this metadata row when they want to **disable** breadcrumbs:

Metadata	
breadcrumbs	false

If the `breadcrumbs` metadata row is not present, breadcrumbs are shown.

3.2 Bulk Metadata

For site-wide or subsection-level control, breadcrumbs can be managed via the **bulk metadata spreadsheet**. This avoids the need to set metadata on every individual page.

URL	breadcrumbs
/products/**	true
/landing-pages/**	false
/campaigns/**	false

Bulk metadata applies to all pages matching the URL pattern. Page-level metadata overrides bulk metadata if both are set.

3.3 Skip Page from Breadcrumb Trail

A page can be excluded from the breadcrumb navigation trail using a custom metadata property. When a page is marked to be skipped, it does not appear as a link in the breadcrumb trail of its child pages. The breadcrumb trail connects to the next visible ancestor instead.

Metadata	
breadcrumb-skip	true

3.4 How Breadcrumbs Are Generated

The breadcrumb block JS generates the navigational trail at delivery time:

1. Reads the current page's URL path
2. Splits the path into segments — each segment represents a level in the site hierarchy
3. For each segment, resolves the page title (from the page's metadata or navigation structure)
4. Renders the breadcrumb trail as a series of linked page titles separated by a delimiter
5. The current page (last segment) is displayed as plain text (not a link)

Example: For a page at `/products/life-science/pcr/thermal-cyclers`:

1	Home > Products > Life Science > PCR > Thermal Cyclers
2	

Each item except the last is a clickable link. "Thermal Cyclers" (current page) is displayed as text.

3.5 Authoring Summary

Concern	Approach
Enable breadcrumbs	Default — no action needed (enabled by default)
Disable on a page	Add <code>breadcrumbs: false</code> in page Metadata block
Enable/disable site-wide	Use bulk metadata spreadsheet
Skip a page from trail	Add <code>breadcrumb-skip: true</code> in page Metadata block

Breadcrumb links	Generated automatically from URL path — no manual authoring
Breadcrumb labels	Resolved from page titles — no manual authoring
Block table	Not needed — no breadcrumb block table on the page

Anchor list

1. Block Overview

Property	Value
Block Name	Anchor List
Maps to Existing	Anchor List Block Component
Description	An auto-generated navigation list that links to sections within the current page. The block scans the page for H2 headings and generates anchor links automatically.
Authoring Strategy	Flattened block — minimal table. Block content is auto-generated at delivery time. Author only provides an optional title.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Title — optional. A heading displayed above the anchor list (e.g. "On this page"). If omitted, the anchor list renders without a heading.

Anchor links are generated automatically. The block JS scans the page for all H2 headings at delivery time and builds a navigation list with one link per heading. No manual entry of links is required.

Authors do not need to create or maintain the link list. When H2 headings are added, removed, or reordered on the page, the anchor list updates automatically on next page load.

3. Variants

Variant	Description	Reference
default		

		Integrated CDMO-CRO Services Clinical trial CRO services CDMO & clinical trial supply services Lab instruments and equipment services Cell biology services Custom services Training services Enterprise services Enterprise-level lab informatics Financial and leasing services Partnering and licensing Food safety inspection product assurance services and solutions (PASS) T Services Thermo Fisher Scientific - US
Classic-horizontal		Cell & Gene Therapy Solutions Thermo Fisher Scientific - US

4. DA Block Table Contract

4.1 Table Structure

The Anchor List block is a **minimal table**. The block only needs a header row to identify itself. An optional title row can be added.

Column	Role	Content
Column 1	Key	Property name (title)
Column 2	Value	Property value

4.2 Block Header and Variants

The first row of the table is the block header.

Authors select variants by including the variant name in parentheses in the block table header row.

Author types in header row	Resulting CSS classes
----------------------------	-----------------------

Anchor List	<code>.anchor-list</code> (default — vertical)
Anchor List (classic-horizontal)	<code>.anchor-list.classic-horizontal</code>

4.3 How the Block Works at Delivery Time

1. Block JS scans the page for all H2 headings
2. For each H2, generates an anchor ID (based on the heading text) and creates a link
3. Renders the list of links inside the block — either vertically (default) or horizontally (classic-horizontal)
4. Clicking a link smooth-scrolls to the corresponding H2 section on the page

The author does not manage this list. It is fully automatic.

4.4 Authoring Summary

Concern	Approach
Block type	Flattened block — minimal table
Anchor links	Auto-generated from H2 headings — no manual authoring
Title	Optional — add a <code>title</code> row if a heading above the list is needed
Variant selection	Variant name in block header: <code>Anchor List (variant)</code>
Link maintenance	None — list updates automatically when H2 headings change

Column

1. Block Overview

Property	Value
Block Name	Columns
Description	Creates multi-column layouts for side-by-side content presentation. The number of table columns determines the number of rendered columns. Variants control width distribution.
Authoring Strategy	Flattened block — each table column maps directly to a rendered column. Rich text content authored directly in each cell.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

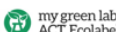
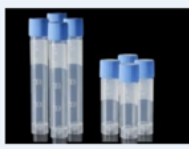
Column content (rich text) — required (at least one column). Each table column contains the content for that rendered column. Supports:

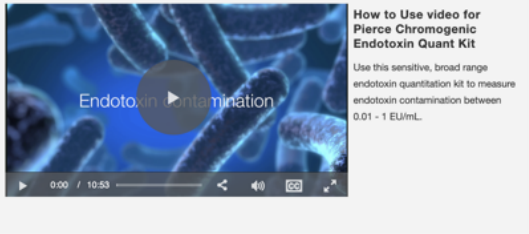
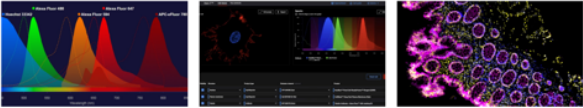
- Text (paragraphs, headings, bold, italic)
- Images
- Links and lists
- Any inline content that fits in a DA table cell

Author determines the number of columns by the number of table columns. A 2-column table renders a 2-column layout. A 3-column table renders a 3-column layout.

Width distribution is controlled via variants. If no variant is specified, columns are equal width.

3. Variants

Variant	Description	Reference
default	Two equally-sized columns	
60-40	Left column take sup 60% of total width, right column takes up 40% of width	Raising the bar on more sustainable product design Always with an eye on innovation, we strive to make the most of our resources and reduce the environmental footprint of our products, without compromising quality and performance. Leveraging our sustainable design framework and processes along with the PPI Business System, Thermo Fisher scientists apply Design for Sustainability principles to product design. We're on a journey to think differently about how we design, make and package our products—moving from linear processes to a more circular economy, where we eliminate waste and keep materials in use, either as a product, components or raw materials.  https://www.thermofisher.com/us/en/home/sustainable-design.html
40-60	Left column take sup 50% of total width, right column takes up 60% of width	 At Thermo Fisher, we prioritize transparency that enables our customers to make informed choices regarding the environmental impact of the products they select. To support, we have aligned our approach with industry standards, emphasizing the use of third-party ecolabels. Thermo Fisher is an early adopter and advocate of the My Green Lab ACT Ecolabel, the premier third-party ecolabel for laboratory products, encompassing consumables, chemicals, and equipment. Leveraging the ACT Ecolabel allows laboratories to make informed purchasing choices based on standardized and verified environmental data, enabling you to advance sustainability without sacrificing performance. ACT label program https://www.thermofisher.com/us/en/home/sustainable-design.html
profile	Clickable images with gray text underneath	 T Meet the Innovators Thermo Fisher Scientific - US
30-70		 T Protein Assays and Analysis Thermo Fisher Scientific - US

70-30		 T Protein Assays and Analysis Thermo Fisher Scientific - US
25-50-25	Middle column takes up 50% of total width, other two columns each take up 25% of width	 T Lab Centrifuges Thermo Fisher Scientific - US
3 equal columns	3 columns each of equal width	 T Fluorescent Secondary Antibodies Thermo Fisher Scientific - US

4. DA Block Table Contract

4.1 Table Structure

The Columns block maps **table columns directly to rendered columns**. Each cell in the table becomes one column in the rendered layout.

2-column layout:

Columns	
Left column content	Right column content

3-column layout:

Columns		
---------	--	--

Left column content	Middle column content	Right column content
---------------------	-----------------------	----------------------

4.2 Block Header and Variants

The first row of the table is the block header. It identifies the block and optionally specifies a width variant.

Author types in header row	Resulting layout
Columns	Equal width columns (number determined by table columns)
Columns (60-40)	Left 60%, right 40%
Columns (40-60)	Left 40%, right 60%
Columns (70-30)	Left 70%, right 30%
Columns (30-70)	Left 30%, right 70%
Columns (25-50-25)	Left 25%, middle 50%, right 25%
Columns (classic-profile)	Equal width with profile card styling

Content row starts from row 2. Each cell in the content row contains the content for that column.

4.3 Column Content

Each cell supports rich text content. Authors can place any combination of:

- Headings
- Paragraphs
- Bold, italic, links
- Images
- Lists
- Links that serve as CTAs (using CTA plugin)

4.4 Authoring Summary

Concern	Approach
Block type	Flattened block — table columns = rendered columns

Number of columns	Determined by number of table columns (2 or 3)
Width distribution	Variant in block header (60-40, 70-30, 25-50-25, etc.)
Column content	Rich text authored directly in each cell
Visual treatment	Variant in block header (classic-profile)
Default behaviour	Equal width columns when no width variant specified

Custom Table

1. Block Overview

Property	Value
Block Name	Data List
Maps to Existing	Table + Row Component (AEM 6.x)
Description	A spreadsheet-driven data listing block that automatically builds search, filters, pagination, and item cards from a single spreadsheet. Replaces the old AEM Table + Row approach with a single source of truth.
Authoring Strategy	Flattened block — key-value configuration table on the page. Content data maintained in a separate spreadsheet.
Authoring Source	DA (Document Authoring) — block configuration on page, data in spreadsheet
Delivery	Edge Delivery Services (EDS)

Key Concept

Authors maintain a single spreadsheet (the data source). The Data List block reads that spreadsheet's JSON endpoint and automatically:

- Extracts filter values from specified columns
- Builds the search bar and filter dropdowns
- Renders item cards (title, date, image, description, link)
- Provides pagination controls

Authors only edit the **spreadsheet** (for content/data) and the **block configuration table** (for display options). No per-item authoring on the page is needed.

2. Authoring Criteria

Spreadsheet — required. A DA spreadsheet (.xlsx) that contains all data items. Each row is one item. Each column is a field. The spreadsheet is served as JSON by EDS.

Block configuration — required. A Data List block table on the page that points to the spreadsheet and configures search, filters, and pagination options.

No per-item authoring on the page. All content items (events, resources, products, etc.) are managed in the spreadsheet. The page only contains the block configuration.

3. Spreadsheet Structure

3.1 Column Roles

Spreadsheet columns serve different purposes depending on how they are used:

Role	Purpose	Examples
Display	Fields shown on item cards	title, date, description, image, link
Filter	Fields used as filter facets — values become dropdown options	event-type, country, month
Search	Hidden field used for text search matching	search-keywords

3.2 Example Spreadsheet

title	date	end-date	location	description	link	image	event-type	country	month	search-keywords
SLA S 202 6	202 6- 02- 07	202 6- 02- 11	Boston, MA US	Short even	/events/slas-	/images/slas.jpg	Tradeshow	United States	February	SLA S, Auto

				t blurb	202 6					mati on
Bio Euro pe 202 6	202 6- 03- 15	202 6- 03- 17	Berli n, Ger man y	Ann ual biote ch conf eren ce	/eve nts/b io- euro pe	/ima ges/ bio- euro pe.jp g	Conf eren ce	Ger man y	Marc h	Bio, Euro pe, Biote ch

3.3 How Filters Are Derived

Filter dropdowns are **automatically populated** from the unique values in the specified filter columns. Authors do not manually define filter options.

For example, if `event-type` column contains: Tradeshow, Conference, Webinar, Workshop — the filter dropdown will show these four options automatically. When a new event type is added to the spreadsheet, it appears in the filter without any configuration change.

3.4 Spreadsheet Location

The spreadsheet is stored in DA as a `.xlsx` file and served by EDS as a JSON endpoint.

Example: A spreadsheet at `/data/events.xlsx` is accessible as JSON at `/data/events.json`.

The block configuration points to this JSON path.

3.5 Spreadsheet Organization — Best Practices

All data-list spreadsheets should be organized in a dedicated `/data` folder at the site root. This keeps data sources separate from authored pages and makes them easy to find, manage, and govern.

Recommended folder structure:

1	/data/	
2	events.xlsx	← Events and webinars listing
3	resources.xlsx	← Resource/document library
4	news.xlsx	← News and announcements
5	team.xlsx	← Team members
6	products-catalog.xlsx	← Product catalog data
7	training-courses.xlsx	← Training and courses
8		

Naming conventions:

Convention	Rule	Example
Location	All spreadsheets in <code>/data/</code> folder	<code>/data/events.xls</code> <code>x</code>
Naming	Lowercase, hyphen-separated, descriptive	<code>training-courses.xlsx</code> not <code>TC_Data.xlsx</code>
One spreadsheet per listing	Each Data List block points to its own spreadsheet	Events page → <code>/data/events</code> , Resources page → <code>/data/resources</code>
No page-path nesting	Spreadsheets live in <code>/data/</code> regardless of which page uses them	A page at <code>/products/life-science/events</code> still points to <code>/data/events</code>

Why `/data/` at the root:

- **Discoverable** — all data sources in one predictable location
- **Shared** — multiple pages can reference the same spreadsheet (e.g. a "Featured Events" widget on the homepage and the full Events page both point to `/data/events`)
- **Governed** — permissions and access control can be applied to the `/data/` folder as a whole
- **Clear separation** — authored pages live in their content hierarchy, data spreadsheets live in `/data/`

When to create sub-folders:

For large sites with many spreadsheets, organize by domain:

```

1 /data/
2   /events/
3     events-global.xlsx
4     events-america.xlsx
5     events-emea.xlsx
6   /resources/
7     resources-life-science.xlsx
8     resources-clinical.xlsx
9   /products/
10    products-catalog.xlsx
11    products-discontinued.xlsx

```

Sub-folders are recommended when the `/data/` folder exceeds 10–15 spreadsheets.

4. DA Block Table Contract

4.1 Table Structure

The Data List block is authored as a **key-value configuration table**. Each row sets one configuration option.

Column	Role	Content
Column 1	Key	Configuration property name
Column 2	Value	Configuration property value

4.2 Available Properties

Key	Value	Required	Default	Description
<code>data-source</code>	Path to JSON endpoint	Yes	—	Path to the spreadsheet JSON (e.g. <code>/events-data</code>)
<code>searchable</code>	<code>true</code> / <code>false</code>	No	<code>false</code>	Enables a text search bar above the listing
<code>filterable</code>	<code>true</code> / <code>false</code>	No	<code>false</code>	Enables filter dropdowns
<code>filter-columns</code>	Comma-separated column names	No (required if filterable is true)	—	Spreadsheet columns to use as filter facets (e.g. <code>event-</code>

				type, country, month)
pagination	true / false	No	false	Enables pagination controls
per-page	5 , 10 , 15 , 20 , 25	No	10	Number of items displayed per page
left-nav- filters	true / false	No	false	Places filters in a left rail instead of top bar

Only include rows for properties that are needed. Omitted properties use their default values.

4.3 Block Header

The first row is the block header identifying the block. No variants — the block has a single visual presentation.

Data List	
-----------	--

4.4 How the Block Works at Delivery Time

1. Block JS reads the configuration from the block table rows
2. Fetches the spreadsheet JSON from the `data-source` path
3. If `searchable` is true — renders a text search bar
4. If `filterable` is true — reads `filter-columns`, extracts unique values from those columns, and renders filter dropdowns
5. Renders item cards from the spreadsheet data (title, date, image, description, link)
6. If `pagination` is true — renders pagination controls with the configured `per-page` count
7. Search and filter interactions update the visible items in real-time without page reload

4.5 Authoring Summary

Concern	Approach
---------	----------

Block type	Flattened block — key-value configuration table
Data source	Spreadsheet in DA served as JSON by EDS
Adding an item	Add a row to the spreadsheet — no page edit needed
Removing an item	Delete the row from the spreadsheet
Bulk edits	Edit spreadsheet cells, save once
Filters	Auto-derived from spreadsheet column values
Search	Matches against search-keywords column
Configuration	Key-value rows in the block table on the page
Per-item authoring on page	None — all items managed in the spreadsheet

5. Authoring Workflows

5.1 Add a New Item

1. Open the spreadsheet in DA (e.g. `events-data.xlsx`)
2. Add a new row — fill in all fields (title, date, location, image, event-type, etc.)
3. Save the spreadsheet
4. The page renders the new item automatically — no page edit required

5.2 Remove an Expired Item

1. Open the spreadsheet
2. Delete the row
3. Save the spreadsheet
4. The item disappears from the listing. Filters update automatically — if that was the last item with a specific filter value, the value is removed from the dropdown.

5.3 Bulk Update

1. Open the spreadsheet
2. Edit multiple cells (e.g. change 20 dates, update descriptions, fix links)
3. Save once
4. All changes reflect on the page immediately

5.4 Add a New Filter Category

1. Add a new column to the spreadsheet (e.g. `region`)
2. Fill the column values for existing rows
3. Update the block configuration on the page — add the new column name to `filter-columns`
4. The new filter dropdown appears automatically with values from the column

5.5 Change Block Configuration

1. Open the page in DA
2. Edit the Data List block table — change any configuration value (e.g. change `per-page` from 10 to 20)
3. Save the page
4. Configuration change takes effect

6. Comparison: AEM (Table + Rows) vs EDS Data List

Concern	AEM Table + Rows	EDS Data List
Where data lives	Table config + many Row child components	One spreadsheet (single source)
Defining filters	Manual list in Table config + per-row mapping	Auto-derived from spreadsheet columns
Add new item	Add Row in AEM, map filters, publish	Add row in spreadsheet, save
Bulk edits	Edit each Row individually	Edit spreadsheet cells, save once
Page authoring	Heavy — many components per page	Light — one block table on page

Content maintenance	Per-item in AEM author	Spreadsheet only — no page edits
---------------------	------------------------	----------------------------------

Welcome to our Events and Webinars Hub. Easily search and register for upcoming events, webinars, and workshops. Find professional development opportunities, industry insights, and networking events. Start your search now and connect with experts and peers.



Q

Event Type ▾

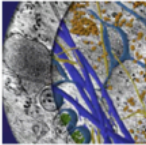
Country ▾

Month ▾

How AI Helps with Image Data Analysis in Life Sciences

December 4, 2025–April 1, 2026 | Global

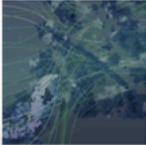
Discover how AI in life sciences streamlines image data analysis. Learn advanced denoising, AI-assisted annotation, and 3D segmentation with Amira Software. [Learn more](#)



Why data integration matters in Cryo-EM

January 13–April 15, 2026 | Global

Effectively managing cryo-EM data with CryoFlow Software. [Learn more](#)



T

Events and Webinars | Thermo Fisher Scientific - US

Sub Navigation

Map to Auto Navigation, Manual Navigation, Custom Navigation, Nav List Custom Manual ,Nav List

A secondary navigation bar for navigating within a section of the site. Default variant is horizontal tabs.

Authoring Criteria:

- Navigation mode – required. Author selects either auto or manual.
- Title - optional.

Manual mode:

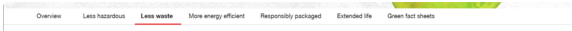
- Author adds link items directly in the block. Each row is a title + URL pair.
- The block renders those links as-is.

Auto mode:

- Author enters a root path.
- The block fetches query-index.json, filters for child pages under that path, and renders the links automatically.

Development Requirements

The block must detect nested link items and automatically render them as dropdowns with open/close toggle, click-outside dismiss, and keyboard navigation support.

Variant	Description	Reference
default	Nav displays as horizontal tabs	

Classic -left- nav	Nav displays in a column on the left side of the page.	Dulbecco's Modified Eagle Medium (DMEM) Dulbecco's Modified Eagle Medium F12 (DMEM F12) Minimum Essential Medium (MEM) Roswell Park Memorial Institute (RPMI) 1640 Medium Serum-Free Media (SFM) BenchStable Cell Culture Media Human Plasma-Like Medium Gibco MediaSelect Tool Gibco Media Bottle Cell Culture Bags for Media Storage Lab Armor Beads for Water Baths
Classic - without -red- hover	Nav displays without red hover.	Popular Invitrogen RNA products RNA Isolation Products RNA Stability & Storage RNA Interference Tools qPCR Directly from Cells Organic RNA Extraction RNase Control Key Applications MicroRNA Analysis RNAi Automated RNA Extraction Single-Cell Analysis RNA-Seq Sample Prep RNA Extraction for RT-PCR T RNA Purification by Invitrogen Thermo Fisher Scientific - US

		Protein Assays and Analysis ◦ Protein Assays ◦ Immunoprecipitation with Magnetic Beads ◦ Western Blotting ◦ ELISA-Related Products and Accessories ◦ ProQuantum High-Sensitivity Immunoassays ◦ Protein Interaction Analysis ◦ Protein Microarrays ◦ Protein and Enzyme Activity Assays ◦ Reporter Gene Assays ◦ Pro-Detect Rapid Assays T Protein Assays and Analysis Thermo Fisher Scientific - US
--	--	--

References url:--

[T Greener by design | Thermo Fisher Scientific - US](#)

[T Microarray Solutions Plant Resources | Thermo Fisher Scientific - US](#)

[T Thermo Scientific | Thermo Fisher Scientific - US](#)

[T TaqMan Real-Time PCR Assays | Thermo Fisher Scientific - US](#)

Basic Table

1. Block Overview

Property	Value
Block Name	Table
Maps to Existing	Text Component (used for table authoring)
Description	A configurable data table with styled headers, borders, alignment, sorting, and striping options. Supports merged cells (colspan/rowspan).
Authoring Strategy	Flattened block — the block header row provides the block name and styling variants. All subsequent rows are table content (headers + data).
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Table headers — The first content row (row 2 of the table) is treated as the table header row. Can be styled via variant (default, dark, none).

Table data rows — Each row after the header is a data row. Authors type content directly into cells.

Cell content — each cell supports rich text (bold, italic, links, lists, images).

All styling is controlled via variants in the block header. Authors do not need to configure properties separately — they combine variant names to get the desired table appearance.

3. Variants

The Table block supports multiple variants that can be combined. Variants cover header styling, borders, alignment, row colours, and special features.

3.1 Header Style

Variant	Description
default (no variant needed)	Light gray background header row
<code>dark</code>	Charcoal background, white text header row
<code>no-header</code>	No header styling — first row rendered as regular data

3.2 Border Style

Variant	Description
default (no variant needed)	Bordered — cell borders visible
<code>borderless</code>	No cell borders

3.3 Text Alignment

Variant	Description
default (no variant needed)	Left-aligned text in all body cells
<code>center</code>	Center-aligned text in all body cells

3.4 Visual Treatment

Variant	Description
default	Standard table
<code>striped</code>	Alternate row background colours
<code>first-column-colour</code>	First column has grey background colour

sortable-table	Column headers are clickable for sorting
elisa-kit	Custom green + blue design

Additional variants will be incorporated during implementation as encountered across the site.

Variant	Description	Reference															
default		T Luminex System Accessories Thermo Fisher Scientific - US <table><tr><th>Product</th><th>Description</th><th>Cat. No.</th><th>Unit Size</th></tr><tr><td>INTELLIFLEX Calibration Kit</td><td>The INTELLIFLEX Calibration Kit is for use with the Luminex xMAP INTELLIFLEX and Luminex xMAP INTELLIFLEX DR-SE systems and is used to calibrate the optics of these systems. The calibration kit contains all reagents needed for calibration of these systems.</td><td>IFXCALK20</td><td>20 reactions</td></tr><tr><td>INTELLIFLEX Performance Verification Kit</td><td>The Performance Verification Kit contains all reagents needed for verifying the calibration of INTELLIFLEX systems.</td><td>IFXPVERK20</td><td>20 reactions</td></tr></table>	Product	Description	Cat. No.	Unit Size	INTELLIFLEX Calibration Kit	The INTELLIFLEX Calibration Kit is for use with the Luminex xMAP INTELLIFLEX and Luminex xMAP INTELLIFLEX DR-SE systems and is used to calibrate the optics of these systems. The calibration kit contains all reagents needed for calibration of these systems.	IFXCALK20	20 reactions	INTELLIFLEX Performance Verification Kit	The Performance Verification Kit contains all reagents needed for verifying the calibration of INTELLIFLEX systems.	IFXPVERK20	20 reactions			
Product	Description	Cat. No.	Unit Size														
INTELLIFLEX Calibration Kit	The INTELLIFLEX Calibration Kit is for use with the Luminex xMAP INTELLIFLEX and Luminex xMAP INTELLIFLEX DR-SE systems and is used to calibrate the optics of these systems. The calibration kit contains all reagents needed for calibration of these systems.	IFXCALK20	20 reactions														
INTELLIFLEX Performance Verification Kit	The Performance Verification Kit contains all reagents needed for verifying the calibration of INTELLIFLEX systems.	IFXPVERK20	20 reactions														
		<table><tr><th>Air-drying</th><th></th><th>Lyophilization</th></tr><tr><td>Process: ~2 hours</td><td></td><td>Process: ~24 hours</td></tr><tr><td>Equipment: Heating oven</td><td></td><td>Equipment: Lyophilizer</td></tr><tr><td>Product shipment and storage at room temperature</td><td></td><td>Product shipment and storage at room temperature</td></tr><tr><td>Excipients and manual included</td><td></td><td>Lyophilization service provider recommendation</td></tr></table> T Air-Drying and Lyophilization for Assay Stabilization and Sustainability Thermo Fisher Scientific - US	Air-drying		Lyophilization	Process: ~2 hours		Process: ~24 hours	Equipment: Heating oven		Equipment: Lyophilizer	Product shipment and storage at room temperature		Product shipment and storage at room temperature	Excipients and manual included		Lyophilization service provider recommendation
Air-drying		Lyophilization															
Process: ~2 hours		Process: ~24 hours															
Equipment: Heating oven		Equipment: Lyophilizer															
Product shipment and storage at room temperature		Product shipment and storage at room temperature															
Excipients and manual included		Lyophilization service provider recommendation															
striped	Alternate row background colors	<table><tr><th>Component</th><th>Amount</th></tr><tr><td>SuperScript III/RTaseOUT Enzyme Mix</td><td>100 µl</td></tr><tr><td>2X First-Strand Reaction Mix (contains 10 mM MgCl2, and 1 mM each dNTP)</td><td>500 µl</td></tr><tr><td>Annealing Buffer</td><td>50 µl</td></tr><tr><td>Oligo(dT20 (50 µM)</td><td>50 µl</td></tr><tr><td>Random hexamers</td><td>50 µl</td></tr></table> T SuperScript III First-Strand Synthesis SuperMix Thermo Fisher Scientific - US	Component	Amount	SuperScript III/RTaseOUT Enzyme Mix	100 µl	2X First-Strand Reaction Mix (contains 10 mM MgCl2, and 1 mM each dNTP)	500 µl	Annealing Buffer	50 µl	Oligo(dT20 (50 µM)	50 µl	Random hexamers	50 µl			
Component	Amount																
SuperScript III/RTaseOUT Enzyme Mix	100 µl																
2X First-Strand Reaction Mix (contains 10 mM MgCl2, and 1 mM each dNTP)	500 µl																
Annealing Buffer	50 µl																
Oligo(dT20 (50 µM)	50 µl																
Random hexamers	50 µl																

first-column -colour	First column grey background color	<table><tr><td></td><td>Vanquish Horizon Tandem UHPLC System</td><td>Vanquish Flex Tandem UHPLC System</td><td>Vanquish Core Tandem HPLC System</td></tr><tr><td>Pump</td><td>2 Vanquish Binary Pump H</td><td>2 Vanquish Binary Pump F or 2 Vanquish Quaternary Pump F or 1 Vanquish Dual Pump F</td><td>2 Vanquish Binary Pump C/CN or 2 Vanquish Quaternary Pump C/CN or 1 Vanquish Dual Pump C/CN</td></tr><tr><td>Autosampler</td><td>Vanquish Split Sampler HT</td><td>Vanquish Split Sampler FT</td><td>Vanquish Split Sampler C or Vanquish Split Sampler CT</td></tr><tr><td>Extended sample capacity (optional)</td><td colspan="3">Vanquish Charger (optional)</td></tr><tr><td>Column compartment</td><td colspan="2">Vanquish Column Compartment H</td><td>Vanquish Column Compartment C</td></tr><tr><td>Virtually zero dead volume system connections</td><td colspan="3">Viper UHPLC Fittings Kits</td></tr><tr><td>Detectors</td><td colspan="3">1 Detector Visit our HPLC and UHPLC Detectors page</td></tr></table>		Vanquish Horizon Tandem UHPLC System	Vanquish Flex Tandem UHPLC System	Vanquish Core Tandem HPLC System	Pump	2 Vanquish Binary Pump H	2 Vanquish Binary Pump F or 2 Vanquish Quaternary Pump F or 1 Vanquish Dual Pump F	2 Vanquish Binary Pump C/CN or 2 Vanquish Quaternary Pump C/CN or 1 Vanquish Dual Pump C/CN	Autosampler	Vanquish Split Sampler HT	Vanquish Split Sampler FT	Vanquish Split Sampler C or Vanquish Split Sampler CT	Extended sample capacity (optional)	Vanquish Charger (optional)			Column compartment	Vanquish Column Compartment H		Vanquish Column Compartment C	Virtually zero dead volume system connections	Viper UHPLC Fittings Kits			Detectors	1 Detector Visit our HPLC and UHPLC Detectors page											
	Vanquish Horizon Tandem UHPLC System	Vanquish Flex Tandem UHPLC System	Vanquish Core Tandem HPLC System																																				
Pump	2 Vanquish Binary Pump H	2 Vanquish Binary Pump F or 2 Vanquish Quaternary Pump F or 1 Vanquish Dual Pump F	2 Vanquish Binary Pump C/CN or 2 Vanquish Quaternary Pump C/CN or 1 Vanquish Dual Pump C/CN																																				
Autosampler	Vanquish Split Sampler HT	Vanquish Split Sampler FT	Vanquish Split Sampler C or Vanquish Split Sampler CT																																				
Extended sample capacity (optional)	Vanquish Charger (optional)																																						
Column compartment	Vanquish Column Compartment H		Vanquish Column Compartment C																																				
Virtually zero dead volume system connections	Viper UHPLC Fittings Kits																																						
Detectors	1 Detector Visit our HPLC and UHPLC Detectors page																																						
		T Vanquish Tandem LC Systems Thermo Fisher Scientific - US																																					
sortable-table	Table with sorting	<table><tr><th>Type</th><th>Title</th><th>Categories</th></tr><tr><td>Application note (2011)</td><td>Bacterial analysis using the Attune Acoustic Focusing Cytometer</td><td>Attune/Attune NxT, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability</td></tr><tr><td>Application note (2011)</td><td>Detection of human mesenchymal stem cells using the Attune Acoustic Focusing Cytometer</td><td>Attune/Attune NxT, cell cycle, flow cytometer/flow cytometry, immunophenotyping, stem cell research</td></tr><tr><td>Application note (2012)</td><td>Monitoring neurite morphology and synapse formation in primary neurons for neurotoxicity assessments and drug screening</td><td>ArrayScan, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, neuroscience, viability</td></tr><tr><td>Application note (2012)</td><td>Microbiological applications using the Attune Acoustic Focusing Cytometer</td><td>Attune/Attune NxT, cell health, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability</td></tr><tr><td>Application note (2013)</td><td>Multiplexed mitosis and apoptosis analysis</td><td>antibodies, apoptosis, ArrayScan, cell cycle, cell proliferation, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, particles</td></tr><tr><td>Application note (2014)</td><td>High content cell cycle screening: Pairing FUCCI technology with Thermo Scientific HCS Studio 2.0 Software and FCS Express Image cytometry</td><td>ArrayScan, cancer, cell cycle, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader</td></tr></table>		Type	Title	Categories	Application note (2011)	Bacterial analysis using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability	Application note (2011)	Detection of human mesenchymal stem cells using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, cell cycle, flow cytometer/flow cytometry, immunophenotyping, stem cell research	Application note (2012)	Monitoring neurite morphology and synapse formation in primary neurons for neurotoxicity assessments and drug screening	ArrayScan, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, neuroscience, viability	Application note (2012)	Microbiological applications using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, cell health, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability	Application note (2013)	Multiplexed mitosis and apoptosis analysis	antibodies, apoptosis, ArrayScan, cell cycle, cell proliferation, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, particles	Application note (2014)	High content cell cycle screening: Pairing FUCCI technology with Thermo Scientific HCS Studio 2.0 Software and FCS Express Image cytometry	ArrayScan, cancer, cell cycle, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader															
Type	Title	Categories																																					
Application note (2011)	Bacterial analysis using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability																																					
Application note (2011)	Detection of human mesenchymal stem cells using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, cell cycle, flow cytometer/flow cytometry, immunophenotyping, stem cell research																																					
Application note (2012)	Monitoring neurite morphology and synapse formation in primary neurons for neurotoxicity assessments and drug screening	ArrayScan, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, neuroscience, viability																																					
Application note (2012)	Microbiological applications using the Attune Acoustic Focusing Cytometer	Attune/Attune NxT, cell health, flow cytometer/flow cytometry, fluorescent dyes, microbiology, viability																																					
Application note (2013)	Multiplexed mitosis and apoptosis analysis	antibodies, apoptosis, ArrayScan, cell cycle, cell proliferation, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader, particles																																					
Application note (2014)	High content cell cycle screening: Pairing FUCCI technology with Thermo Scientific HCS Studio 2.0 Software and FCS Express Image cytometry	ArrayScan, cancer, cell cycle, drug discovery, fluorescence microscopy/fluorescence imaging, fluorescent dyes, high content analysis, microplate reader																																					
		T Cell Viability, Proliferation and Cell Cycle Information Thermo Fisher Scientific - US																																					
		S																																					
elisa-kit	Custom green + blue design	<table><tr><th>7 steps for Instant ELISA kits</th><th>17 steps for conventional ELISA kits</th></tr><tr><td>1. Rehydration of standard and sample wells on plate</td><td>1. Washing of coated plates</td></tr><tr><td>2. Sample addition</td><td>2. Reconstitution of standard protein</td></tr><tr><td>3. Incubation</td><td>3. Addition of diluent to standard wells</td></tr><tr><td>4. Washing</td><td>4. Titration of standard curve</td></tr><tr><td>5. Addition of TMB substrate</td><td>5. Addition of sample diluent</td></tr><tr><td>6. Addition of stop solution</td><td>6. Sample addition</td></tr><tr><td>7. Evaluation of results</td><td>7. Dilution of biotin conjugate</td></tr><tr><td></td><td>8. Addition of biotin conjugate</td></tr><tr><td></td><td>9. Incubation</td></tr><tr><td></td><td>10. Preparation of streptavidin-HRP conjugate</td></tr><tr><td></td><td>11. Washing</td></tr><tr><td></td><td>12. Addition of streptavidin-HRP conjugate</td></tr><tr><td></td><td>13. Incubation</td></tr><tr><td></td><td>14. Washing</td></tr><tr><td></td><td>15. Addition of TMB substrate</td></tr><tr><td></td><td>16. Addition of stop solution</td></tr><tr><td></td><td>17. Evaluation of results</td></tr></table>		7 steps for Instant ELISA kits	17 steps for conventional ELISA kits	1. Rehydration of standard and sample wells on plate	1. Washing of coated plates	2. Sample addition	2. Reconstitution of standard protein	3. Incubation	3. Addition of diluent to standard wells	4. Washing	4. Titration of standard curve	5. Addition of TMB substrate	5. Addition of sample diluent	6. Addition of stop solution	6. Sample addition	7. Evaluation of results	7. Dilution of biotin conjugate		8. Addition of biotin conjugate		9. Incubation		10. Preparation of streptavidin-HRP conjugate		11. Washing		12. Addition of streptavidin-HRP conjugate		13. Incubation		14. Washing		15. Addition of TMB substrate		16. Addition of stop solution		17. Evaluation of results
7 steps for Instant ELISA kits	17 steps for conventional ELISA kits																																						
1. Rehydration of standard and sample wells on plate	1. Washing of coated plates																																						
2. Sample addition	2. Reconstitution of standard protein																																						
3. Incubation	3. Addition of diluent to standard wells																																						
4. Washing	4. Titration of standard curve																																						
5. Addition of TMB substrate	5. Addition of sample diluent																																						
6. Addition of stop solution	6. Sample addition																																						
7. Evaluation of results	7. Dilution of biotin conjugate																																						
	8. Addition of biotin conjugate																																						
	9. Incubation																																						
	10. Preparation of streptavidin-HRP conjugate																																						
	11. Washing																																						
	12. Addition of streptavidin-HRP conjugate																																						
	13. Incubation																																						
	14. Washing																																						
	15. Addition of TMB substrate																																						
	16. Addition of stop solution																																						
	17. Evaluation of results																																						
		T ELISA Kits and ELISA Components Thermo Fisher Scientific - US																																					

3.5 Combining Variants

Authors combine variants with commas in the block header:

Author types	Result
Table	Default — light gray header, bordered, left-aligned
Table (dark)	Dark header, bordered, left-aligned
Table (striped, bordered)	Striped rows, bordered cells
Table (dark, borderless, center)	Dark header, no borders, centered text
Table (striped, first-column-colour)	Striped rows with highlighted first column
Table (sortable-table, dark)	Sortable columns with dark header

4. DA Block Table Contract

4.1 Table Structure

The Table block uses the block table itself as the content. The structure is:

- **Row 1** — Block header (block name + variants). Not rendered as table content.
- **Row 2** — Table column headers. Rendered as `<thead>` with styling from header variant.
- **Row 3+** — Table data rows. Rendered as `<tbody>` rows.

Row	Role
Row 1	Block header — Table (variants)
Row 2	Column headers
Row 3 onward	Data rows

4.2 Block Header

The first row identifies the block and specifies all styling via variants.

Format: Table (variant, variant, variant)

All configuration is in the block header — no separate key-value configuration rows needed.

4.3 Merged Cells

DA supports merging cells natively. Authors select cells and use the merge option in the DA editor.

- **Horizontal merge (colspan)** — merges cells across columns in the same row
- **Vertical merge (rowspan)** — merges cells across rows in the same column

Merged cells are preserved in the rendered HTML output. The block JS does not alter merge attributes.

4.4 Sortable Table Behaviour

When the `sortable-table` variant is used:

- Column headers become clickable
- Clicking a header sorts the table rows by that column
- Clicking again reverses the sort direction
- Sorting is handled client-side by the block JS

4.5 Authoring Summary

Concern	Approach
Block type	Flattened block — block header + table content rows
Header style	Variant in block header (default, dark, no-header)
Borders	Variant in block header (default bordered, borderless)
Text alignment	Variant in block header (default left, center)
Row striping	Variant in block header (striped)
Sorting	Variant in block header (sortable-table)

Merged cells	DA native merge — colspan and rowspan supported
Adding rows/columns	Author edits the table directly in DA
Cell content	Rich text — bold, italic, links, lists, images

5. Authoring Examples

5.1 Example 1 — Default Table

A standard table with light gray header and bordered cells.

Table			
Feature	Model A	Model B	Model C
Weight	2.5 kg	3.0 kg	1.8 kg
Capacity	100 ml	250 ml	50 ml
Temperature Range	-20°C to 60°C	-40°C to 80°C	-10°C to 40°C

What renders: A standard bordered table with light gray header row containing "Feature", "Model A", "Model B", "Model C". Data rows below with left-aligned text.

5.2 Example 2 — Dark Header, Striped Rows

A table with dark header styling and alternating row colours.

Table (dark, striped)			
Product	Cat. No.	Size	Price
TaqMan Assay	4331182	100 rxns	\$299
SYBR Green Master Mix	4367659	200 rxns	\$199

PowerUp SYBR Green	4368577	500 rxns	\$449
-----------------------	---------	----------	-------

What renders: Dark charcoal header with white text. Body rows alternate between white and light gray backgrounds for easier reading.

Testimonial

1. Block Overview

Property	Value
Block Name	Testimonial
Description	Displays customer quotes with attribution. Adapts layout based on what content the author provides.
Authoring Strategy	Flattened block — key-value pair table. Each row is a named property. Only include rows for fields that are needed.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

2. Authoring Criteria

Quote (rich text) —The testimonial quote and any supporting description. Supports bold, italic, and links.

Author attribution (rich text) — Name, title, and organization of the person being quoted.

Author image — A small headshot displayed inline next to the author name. If omitted, the testimonial renders without an image.

CTA link — A link to the full customer story or video. Authored using the CTA plugin for consistent styling and icons.

Note: CTA should use only standard CTA icons. Legacy red icons from Classic are not supported.

3. Variants

Variant	Description	Reference
default		"When all is said and done, it outperforms [the rest] in that it can analyze 35,000 cells per second with acoustic focusing technology. Its accuracy is not diminished even if you conduct analysis at a high flow rate of 100-1,000 μL/min, which drastically shortens the time required for an experiment."  Dr. Takaashi Satoh, IPReC Assistant Professor Osaka University, Japan T Attune Flow Cytometer Features Thermo Fisher Scientific - US

"Like magic, science takes practice, but the end results are so satisfying."

Read the customer story and discover how Martin Pelletier, PhD overcame his challenge to screen for multiplex biomarkers from a small sample volume and to perform large scale tests with high reproducibility in his translational research.

Martin Pelletier, PhD Researcher, Division of Infectious and Immune Diseases at the CHU de Québec-Laval University Research Center
Associate Professor, Department of Microbiology-Infectious Diseases, and Immunology
Faculty of Medicine, Université Laval, Canada

[Read customer story](#) 

"The Attune Xenith Flow Cytometer is a great spectral instrument with high-speed acquisition. Our core facility staff and research customers who are already familiar with the Attune NxT Flow Cytometer are able to expand their panels and transition seamlessly to the Attune Xenith, while using software that feels familiar. Researchers with experience on other spectral cytometry platforms in our facility are pleased with the increased speed and clog-resistance of Attune Xenith. Additionally, our staff benefit from the large fluid tanks, well-protected SIP, and walk-away shutdown that reduce the amount of daily maintenance time needed to keep the Attune Xenith ready for our researchers. The Attune Xenith is a good, solid cytometer platform capable of tackling a wide range of cytometry assays in one instrument. We feel it would be a welcome addition to a busy flow core."

- Kathryn Fox, PhD, SCYM(ASCP)CM - Flow Lab Technical Manager and Dagna Sheerar,
SCYM(ASCP)CM - Flow Cytometry Director
University of Wisconsin Carbone Cancer Center, Flow Cytometry Laboratory

"We've been very comfortable working with Thermo Fisher Scientific for the last four years. We have a collaboration arrangement that goes back over three and a half years and has taken us from initial, in some cases, accidental/serendipitous discoveries, to a point where the technology we're working on is being adopted by internationally respected reference laboratories overseas. That's very unusual for a group based in somewhere as geographically isolated as Western Australia. This collaboration is a very unusual one, at least, for us locally and has been a very productive one."



- Tim Inglis, BM, DM, PhD, FRCPath, FRCPA, DTM&H
School of Medicine
University of Western Australia, PathWest Laboratory Medicine

[Watch researcher's story](#) 

4. DA Block Table Contract

4.1 Table Structure

The Testimonial block uses a **key-value pair** pattern. Each row has a property name in column 1 and its value in column 2. Only include rows for fields that are needed.

Column	Role	Content
Column 1	Key	Property name
Column 2	Value	Property value

4.2 Available Properties

Key (Column 1)	Value (Column 2)	Description
----------------	------------------	-------------

quote	Rich text	The testimonial quote — supports bold, italic, links
attribution	Rich text	Author name, title, and organization
image	Image (drag and drop)	Author headshot — small image displayed next to attribution
cta	CTA content (via CTA plugin)	Link to full customer story or video

4.3 Block Header

The first row is the block header. Only one variant exists (default), so the header is simply:

Testimonial	
-------------	--

4.4 How the Block Adapts

The block JS renders different layouts based on which fields are present:

Fields provided	Layout
quote + attribution	Text-only testimonial — quote with attribution below
quote + attribution + image	Testimonial with author headshot next to attribution
quote + attribution + cta	Testimonial with action link at the bottom
quote + attribution + image + cta	Full testimonial — quote, author image, attribution, and CTA

4.5 Authoring Summary

Concern	Approach
Block type	Flattened block — key-value table
Quote	Rich text in quote row

Attribution	Rich text in <code>attribution</code> row
Author image	Drag and drop into <code>image</code> row value cell
CTA	Authored using CTA plugin in <code>cta</code> row value cell
Optional fields	Omit rows that are not needed — block adapts layout
Variant	Default only — layout adapts based on content

5. CTA Authoring

The testimonial CTA follows the same pattern as all other blocks. The author places cursor in the `cta` value cell and uses the **CTA plugin** to add the call-to-action.

The CTA plugin provides:

- Label (e.g. "Read full story", "Watch video")
- Link (URL to customer story or video)
- Style (Primary, Outline, Link)
- Icon (Arrow, Document, Download, etc.)
- Open in new window

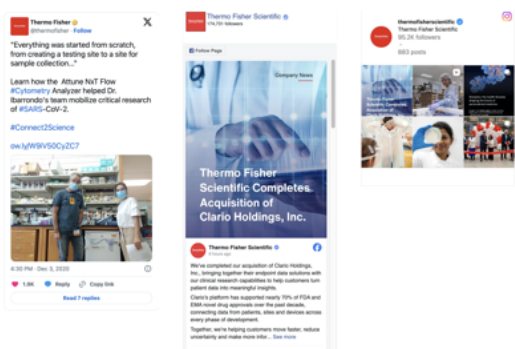
The CTA content format is identical to CTA used in Cards, Hero, and standalone CTA blocks.

Embed

Embeds third-party content by URL. The block detects the platform and renders the appropriate embed.

Authoring Criteria

- **URL** – required. A link to the third-party content.

Variant	Description	Reference
default	This is an example of where we would use the embed block – the author pastes the social media URL and the block automatically detects the platform and renders the appropriate embed.	 Social Media Thermo Fisher Scientific - US

Immersive

1. Block Overview

Property	Value
Block Name	Immersive
Description	A single EDS block for embedding interactive 3D tours, calculators, forms, quizzes, configurators, and other rich experiences directly on the page. Authors select an experience type and provide a URL/identifier — all rendering logic is handled by the block.
Authoring Strategy	Flattened block — key-value pair table. Author selects variant (experience type) and provides one URL or identifier. Minimal authoring surface.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)

Design Principle

Authors should never paste iframe code, script tags, or raw HTML. The block handles all embed construction, asset loading, and rendering internally. The author's only responsibility is selecting the experience type and providing the URL/identifier given to them by the immersive team or vendor.

2. Authoring Criteria

Experience type (variant) — required. Selected via the block header. Determines how the block renders the content.

URL or identifier — required. A single URL or ID that points to the experience. What this looks like depends on the content category:

Content Category	What the author provides	Provided by
3D Tour	Experience JSON URL	Internal immersive team
Web Experience	Bundle URL (JS entry point)	Internal immersive team
External Embed	Experience ID or URL	External vendor / project team

That's it. The author provides one variant and one URL. Everything else — rendering, asset loading, iframe construction, viewer initialization — is handled by the block.

3. Variants

The variant identifies the type of immersive experience. It determines the rendering approach used by the block JS.

Variant	DA Block Name	Description
3d-tour	Immersive (3d-tour)	Interactive 3D product tour with rotatable model, hotspots, side menu, camera perspectives, animations, and AR support
calculator	Immersive (calculator)	Interactive calculator or cost savings tool
selection-tool	Immersive (selection-tool)	Cross-reference or product selection tool
learning-lab	Immersive (learning-lab)	Interactive learning lab or learning center
virtual-experience	Immersive (virtual-experience)	Virtual lab or platform experience

	experience)	
quiz	Immersive (quiz)	Quiz or assessment
game	Immersive (game)	Marketing game or gamification experience
configurator	Immersive (configurator)	Product configurator
form	Immersive (form)	External form experience
landing-page	Immersive (landing-page)	Interactive landing page

Authors select the experience type from this list. The block determines how to render it internally based on the variant.

4. DA Block Table Contract

4.1 Table Structure

The Immersive block is a **minimal key-value table**. One row for the URL/identifier. That's the entire authoring surface.

Column	Role	Content
Column 1	Key	Property name (url)
Column 2	Value	URL or identifier for the experience

4.2 Block Header and Variant

The first row is the block header. The variant (experience type) is specified in parentheses.

Author types in header row	Content category
Immersive (3d-tour)	3D Tour
Immersive (calculator)	Web Experience

Immersive (selection-tool)	Web Experience
Immersive (learning-lab)	Web Experience
Immersive (virtual-experience)	Web Experience
Immersive (quiz)	Web Experience
Immersive (game)	Web Experience
Immersive (configurator)	Web Experience
Immersive (form)	External Embed
Immersive (landing-page)	Web Experience

4.3 Available Properties

Key (Column 1)	Value (Column 2)	Required	Description
url	URL or identifier	Yes	The experience JSON URL, bundle URL, or external experience ID/URL

Only one property row is needed. The variant in the block header tells the block JS how to interpret and render the URL.

4.4 How the Block Works at Delivery Time

The block JS reads the variant and URL, then applies the appropriate rendering strategy:

Variant category	Block JS action
3D Tour	Creates canvas element + UI overlay → loads viewer engine from config sheet → passes experience JSON URL to viewer

Web Experience	Creates a mount div → loads the experience's JS/CSS bundle from the URL → bundle handles all rendering
External Embed	Constructs an iframe from the ID/URL → handles resize and cross-domain communication

All loading is asynchronous — the immersive content does not block page rendering.

4.5 Authoring Summary

Concern	Approach
Block type	Flattened block — key-value table (one row)
Experience type	Variant in block header
Experience URL/ID	Single <code>url</code> row in the table
Rendering logic	Handled entirely by block JS — author provides only the URL
Raw HTML / iframes	Never authored directly — block constructs embeds internally
Config and assets	Managed by immersive team and dev team — not by content authors

5. Content Categories

5.1 3D Tour

Built by: Internal immersive team **Rendered via:** Native canvas element using a shared 3D viewer engine

How it works:

- Author provides the Experience JSON URL
- Block JS reads viewer engine paths from a shared config sheet (maintained by dev team)

- Viewer engine initializes the 3D scene from the experience JSON
- Features: rotatable 3D model, hotspots, camera views, animations, side menu, AR support

Experience JSON (maintained by immersive team) defines:

- 3D model path and environment settings
- Hotspot names, positions, and linked content page URLs
- Camera views and perspectives
- Animations and trigger behaviour
- CTA buttons and links
- Translation keys for localization

Hotspot popup content: Authored as standalone chromeless DA pages (no header/footer).

Loaded into modals at runtime by the viewer engine. Editable independently from the immersive block.

Config sheet (maintained by dev team):

- 3D tour viewer engine JS and CSS bundle paths
- Dev and prod resource paths
- Updated when the immersive team ships a new viewer version — no code deployment needed
- Authors do not interact with this

5.2 Web Experience

Built by: Internal immersive team **Rendered via:** Experience's own JS/CSS bundle loaded dynamically

How it works:

- Author provides the Bundle URL (path to the experience's JS entry point)
- Block JS creates a mount div and loads the bundle
- The experience's own code handles all rendering and interaction

Includes: Calculators, learning labs, virtual experiences, selection tools, quizzes, games, configurators, interactive landing pages.

5.3 External Embed

Built by: External vendors **Rendered via:** iframe constructed by the block

How it works:

- Author provides the experience ID or URL
- Block JS constructs the iframe internally from the identifier
- Block handles domain logic, iframe resize, and cross-domain communication

Hosted on: Thermo Fisher subdomains and third-party domains (e.g. Kaon)

Note: External iframe content loads after the EDS page — expected latency from cross-domain calls. Native rendering (3D tour, web experience) provides full analytics tracking; iframe-based embeds may have limited tracking.

6. Authoring Examples

6.1 Example 1 — 3D Product Tour

An interactive 3D tour of the Volumescope 2 SEM.

Immersive (3d-tour)	
url	/content/dam/experiences/volumescope-tour.json

What renders: A full-width interactive 3D viewer with a rotatable product model, clickable hotspots showing product features, camera perspective buttons, a side menu, and AR launch button on supported devices.

6.2 Example 2 — Cost Savings Calculator

An interactive calculator tool.

Immersive (calculator)	
url	/content/dam/experiences/cost-savings-calculator/bundle.js

What renders: An interactive calculator where users input their lab parameters and see estimated cost savings. Fully rendered from the JS bundle — no iframe.

6.3 Example 3 — Product Selection Tool

A cross-reference or product finder tool.

Immersive (selection-tool)	
url	/content/dam/experiences/antibody-selector/bundle.js

What renders: A guided product selection tool where users answer questions or apply filters to find the right product for their application.

6.4 Example 4 — Learning Lab

An interactive learning centre.

Immersive (learning-lab)	
url	/content/dam/experiences/pcr-learning-lab/bundle.js

What renders: An interactive educational experience with step-by-step modules, animations, and knowledge checks.

6.5 Example 5 — External Vendor Embed (Kaon)

An externally hosted interactive experience.

Immersive (configurator)	
url	kaon-experience-12345

What renders: An iframe-based product configurator hosted by the external vendor. The block constructs the iframe from the ID, handles sizing, and manages cross-domain communication.

6.6 Example 6 — Quiz

An interactive quiz or assessment.

Immersive (quiz)	
url	/content/dam/experiences/genomics-quiz/bundle.js

What renders: A multi-step quiz with questions, scoring, and results. Users interact directly on the page — no navigation away from the content.

PDP Document Display

1. Block Overview

Property	Value
Block Name	PDP Document Display
Maps to Existing	PDP Document Display - Parent Component
Description	Integrates an external Product Data Page (PDP) documents application into EDS. The external app provides document browsing and filtering based on product SKUs, document IDs, and document types. EDS acts as a lightweight orchestration layer.
Authoring Strategy	Flattened block — key-value pair table. Author provides product and document configuration. Block JS handles external script loading, config passing, and container management.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS)
Instance Limit	One instance per page

Responsibility Separation

Layer	Responsibility
EDS Block	Captures author input, creates container, loads external script, passes configuration
External Application	Document fetching, filtering, UI rendering, user interactions

2. Authoring Criteria

SKU — required (unless Document IDs provided). Product SKU(s) used to fetch documents. Supports comma-separated values for multi-SKU.

Document IDs — optional. Specific document identifiers for targeted display.

Document Types — optional. Filters the document listing by type (e.g. spec, manual, guide, certificate, SDS).

Sort By — optional. Default sort order for results.

Expanded — optional. Whether result accordions display expanded or collapsed on load.

Authors configure these fields — the external application uses them to fetch and display the relevant documents.

3. Variants

Variant	DA Block Name	Description
default	PDP Document Display	Standard document display

Note: The corpccommons base component defines two style variants (`red-border` and `black-border`). These were not configured in TFS template policies and do not appear to be actively used. If confirmed needed by the client, they will be implemented as block variants: `PDP Document Display (red-border)` .

4. DA Block Table Contract

4.1 Table Structure

The PDP Document Display block uses a **key-value pair** pattern. Each row is a configuration property.

Column	Role	Content
Column 1	Key	Configuration property name

Column 2	Value	Configuration property value
----------	-------	------------------------------

4.2 Available Properties

Key (Column 1)	Value (Column 2)	Description
<code>sku</code>	Product SKU(s) — comma-separated for multiple	Product SKU used to fetch associated documents
<code>document-ids</code>	Comma-separated document IDs	Specific document identifiers for targeted display
<code>document-types</code>	Comma-separated types (e.g. <code>spec</code> , <code>manual</code> , <code>guide</code>)	Filters results by document type
<code>sort-by</code>	Sort field name	Default sort order for results
<code>expanded</code>	<code>true</code> / <code>false</code>	Show result accordions expanded on page load

Only include rows for properties that are needed. Omitted properties use default values.

4.3 Document Type Values

The same document type string values used in the current AEM implementation are used in EDS — no taxonomy change. Values passed to `window.documentsConfiguration()` remain identical.

Examples: `spec`, `manual`, `guide`, `certificate`, `SDS`, `protocol`

Ownership of the document type taxonomy sits with the external application team. The EDS block passes whatever values are authored.

4.4 Authoring Summary

Concern	Approach
Block type	Flattened block — key-value configuration table

Product SKU	Author enters SKU(s) in the <code>sku</code> row
Document filtering	Author specifies types in <code>document-types</code> row
Specific documents	Author enters IDs in <code>document-ids</code> row
External app URL	Centrally governed — not authored per page (see Section 6)
Rendering	Handled entirely by the external application
Instance limit	One per page

5. Architecture and Integration

5.1 Architecture Flow

1. Author configures block (SKU, document types, etc.) in DA
2. EDS block JS reads authored values from the block table
3. Block creates a container `<div>` with a fixed ID (`pdp-document-display-content`)
4. Block fetches the external application URL from the siteConfig sheet
5. Block waits for visibility (lazy load via IntersectionObserver) — or loads immediately if above-the-fold
6. External JS is loaded dynamically
7. Block constructs the configuration object:

```

1 {
2   sku: "ABC-12345",
3   documentIds: ["DOC-001", "DOC-002"],
4   documentTypes: ["spec", "manual"],
5   sortBy: "date",
6   expanded: true,
7   containerId: "pdp-document-display-content"
8 }
9
```

8. Block invokes: `window.documentsConfiguration(config)`
9. External application fetches data and renders UI into the container

5.2 Container ID Strategy

Since only one instance of this block is permitted per page, the `containerId` is a **fixed predictable value** set by the block JS — no dynamic generation required.

The block assigns a consistent ID (`pdp-document-display-content`) to the render container and passes it to `window.documentsConfiguration()` . This keeps the integration contract simple and stable.

6. Configuration Governance

6.1 External Application URL

The application URL is **centrally managed** via a siteConfig sheet using a key-value structure. Authors do not configure or see this URL.

siteConfig sheet entry:

Key	Value
pdp-application-url	/store/v2/documents/static/conditionalLoadAssets.js

The EDS block fetches this sheet at runtime and reads the URL. This ensures:

- URL is governed centrally — not author-editable
- URL changes are a spreadsheet edit — no code deployment needed
- Consistent across all pages using this block

6.2 Why Centrally Governed

The external script URL is an infrastructure concern, not a content concern. Allowing authors to configure it per page would:

- Create security risk (arbitrary script URLs)
- Lead to inconsistency across pages
- Make updates require per-page edits

7. Error Handling

The EDS block carries forward proven error handling patterns from the existing AEM implementation.

7.1 Script Load Failure / Missing Global Function

The block uses a polling mechanism (`checkPDPReady`) that checks every 200ms for up to 30 attempts (6-second timeout). If the external application does not respond within this window,

the block renders an error message in the container.

7.2 Multiple Instances on Same Page

A detection function identifies duplicate instances on page load and replaces the duplicate with a clear error message. Only one instance per page is supported.

7.3 Invalid or Empty SKU / Document Results

Block JS guards against invoking `window.documentsConfiguration()` if both SKU and Document IDs are absent. The existing CSS rule suppressing the "no results" message from end users is carried forward to the block stylesheet.

8. Performance and Security

8.1 Loading Strategy

The block follows EDS recommended loading best practices:

- **Below-the-fold placement** — lazy initialization using IntersectionObserver. External JS loads when the block enters or nears the viewport.
- **Above-the-fold placement** — immediate load without deferral.

The load strategy is determined by the block's typical placement on PDP pages. Both patterns are supported.

Benefits:

- Improves Core Web Vitals
- Reduces initial page load time
- External script loaded only once (duplicate load prevention)

8.2 CSP (Content Security Policy)

The EDS CSP approach is **nonce-based with** `strict-dynamic` as documented at aem.live/docs/csp. Under this model:

- Scripts dynamically loaded by trusted EDS scripts are automatically covered through the trust chain
- No separate nonce or domain whitelisting is required for the external PDP application script
- The centrally governed application URL (via siteConfig) ensures the script source is not author-editable, reducing risk of unintended or malicious URLs

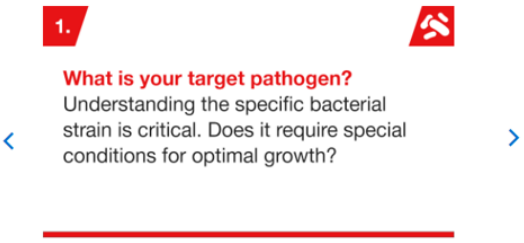
No custom CSP configuration is anticipated for this block beyond the standard EDS setup.

Carousel

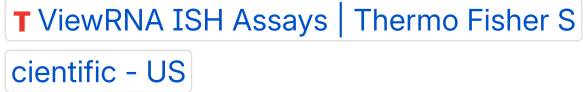
A block for displaying a rotating set of content slides (images, text) with navigation controls

Authoring Criteria

- Image – required. The slide image.
- Richtext – optional. Supporting text and call-to-action link per slide.

Variant	Description	Reference
default	Single-slide carousel with previous/next navigation	5 considerations & questions to ask when selecting your peptone mix  T Boost Consistency and Yields in Your Veterinary Vaccine Production with an Optimized Peptone Mix Thermo Fisher Scientific - US
three-col	Carousel displaying three columns of content per slide	 T Polymerase Chain Reaction (PCR) Thermo Fisher Scientific - US  T Life in the Lab Thermo Fisher Scientific - US

Includes thumbnail
navigation strip
below
No previous/next
navigation



Product Selection Guide

1. Overview

The **Product Selection Guide component** integrates with an external Feature Collection service

Component acts as a lightweight wrapper that orchestrates parameter extraction and HTTP communication with the external service, returning the pre-rendered HTML response directly into the page.

[T Nalgene Bottle, Carboy and Vial Selection Guide](#) | [Thermo Fisher Scientific - US](#)

2. Current AEM Implementation

- Authors configure the component using **featureCollectionId** (mandatory)
- The component relies on the **featureCollectionId** to fetch and render the appropriate product selection guide.
- At runtime, the component uses a **Sling Model (server-side)** to:
 - Read authored properties
 - Extract request parameters (filters, paging, etc.)
 - Derive contextual information such as:
 - locale (language/country)
 - device type (mobile vs desktop)

The Sling Model then:

1. Constructs a request to the external **Feature Collection service**
2. Passes:
 - featureCollectionId
 - query parameters
 - headers/cookies (locale, user-agent, etc.)
3. Performs a **server-side HTTP call**
4. Receives a response in the form of **HTML (including CSS and JS)**
5. Directly injects this HTML into the AEM page

3. Key Concern

The current implementation relies on **HTML passthrough from an external service**, which introduces significant concerns:

- External HTML may include **uncontrolled CSS and JavaScript**
- Increased risk of:
 - render-blocking resources
 - layout shifts
 - degraded Core Web Vitals (LCP, TBT)(Largest Contentful Paint, Total Blocking Time)
- Limited control over markup, accessibility, and styling
- Tight coupling between backend response and frontend rendering.

This pattern is **not suitable for EDS**, where performance, control, and predictable rendering are critical.

4. Proposed Approach – JSON + EDS Block

Replace HTML passthrough with a **structured JSON contract**, and let the **EDS block handle rendering (decoration)**.

- **Backend → Provides data (JSON)**
- **EDS Block → Handles rendering (HTML, CSS, JS)**

EDS Block Behavior

The **product-selection-guide block** will:

- Read authored configuration:
 - featureCollectionId
 - optional display settings
- Read runtime context from the URL:
 - filters
 - paging
 - view type
- Determine locale using EDS-supported mechanisms (URL structure or metadata)
- Call the **Feature Collection service (JSON endpoint)**
- Render:
 - product listing (grid/list)
 - filters and controls
 - pagination
 - empty and error states

- The external Feature Collection service should expose a **structured JSON API** instead of returning pre-rendered HTML.

This establishes a clear separation of responsibilities:

- **Backend → Business logic and data**
- **EDS Block → Rendering and interaction**

Adopting a JSON-based contract is critical because:

- Prevents uncontrolled injection of external CSS and JavaScript
- Ensures **full control over markup, styling, and behavior within EDS**
- Enables optimized rendering, lazy loading, and better Core Web Vitals
- Improves maintainability by decoupling backend output from frontend structure

EDS blocks should **consume structured data, not pre-rendered HTML**.

5. Pros and Cons

Aspect	Current AEM (HTML Passthrough)	Proposed (JSON + EDS Block)
Performance (Core Web Vitals)	Risky due to external CSS/JS injection	Strong control with optimized rendering
Control over UI	Low (backend defines HTML)	High (EDS controls DOM and styling)
Maintainability	Tightly coupled to backend output	Clean separation of data and presentation
Authoring Experience	Simple, based on featureCollectionId	Same model retained
Flexibility	Limited	High (UI changes independent of backend)
Accessibility & Semantics	Depends on external HTML	Fully controllable in EDS
EDS Alignment	Not aligned	Fully aligned with EDS principles

6. Summary

The current implementation uses a **server-side HTML passthrough model**, where AEM acts as a proxy to an external service and renders its response directly. While functional, this introduces performance and maintainability challenges.

The recommended approach is to adopt a **JSON-driven integration with an EDS block**, where:

- The backend provides structured data
- The EDS block handles rendering and interaction

This results in:

- Improved performance and Core Web Vitals
- Full control over markup and styling
- A scalable, maintainable architecture aligned with EDS best practices

Ownership & Integration Boundaries

Ownership Matrix

Component	Owner	Responsibility
Product Selection Guide Block (EDS)	Adobe / Implementation Team	Block JS/CSS, rendering (grid/list/filters/pagination/empty/error states), URL parameter handling, locale detection, UX interactions
Feature Collection Service (JSON API)	TFS	Exposes structured JSON endpoint, business logic, filtering, sorting, pagination logic, product data
JSON API Contract	Joint (Adobe + TFS)	Request/response schema agreed during implementation

API Authentication (if needed)	TFS provides credentials; Adobe configures (in Edge Worker if API can't be accessed from browser)	Depends on API authentication.
---------------------------------------	---	--------------------------------

Key Dependency: TFS Must Deliver JSON API

The EDS integration **requires TFS to expose a structured JSON endpoint** from the Feature Collection service. The current HTML passthrough endpoint cannot be used in EDS.

FeaturedCollection

1. Overview

The **Featured Collection component** is a server-side content inclusion component that pulls external content into the page at render time.

2. Current AEM Implementation

- Authors configure the component by providing **Featured Collection URL** (mandatory) that resolves to external featured content.

Example:

```
/search/browse/promotions/{promocode}?selector=noheader,nofooter
```

- At runtime, the component uses **Server-Side Includes (SSI)** to fetch and render external content.

Flow:

1. The component reads:
 - Authored **featuredCollectionUrl**
 - Incoming request query parameters
2. The component constructs an SSI include:
 - Appends request query parameters to the authored URL
3. The server executes the SSI directive:
 - Makes an HTTP call to the external service
 - Fetches the response
4. The response (HTML) is:
 - Injected directly into the page
 - Rendered as part of the server response

The external service returns:

- Pre-rendered HTML
- Associated CSS references
- JavaScript bundles
- Fully composed UI

The component does not control rendering and simply outputs the response.

3. Why HTML Injection is Not Recommended

The current approach relies on **injecting external HTML (with CSS and JS)**, which introduces key challenges:

3.1 Performance Impact

- External CSS may block rendering → impacts **LCP**
- External JS may block execution → impacts **TBT**
- Increased risk of layout shifts and delayed rendering

3.2 Lack of Control

- Markup structure is controlled externally
- Styling is not fully governed by the site
- Difficult to enforce:
 - design consistency
 - accessibility standards
 - semantic structure

3.3 Tight Coupling

- Frontend depends on backend HTML structure
- Backend changes can break UI
- Limits flexibility for UI evolution

3.4 EDS Compatibility

- **SSI is not supported in EDS**
- Injecting HTML with embedded CSS/JS is **not aligned with EDS architecture**
This pattern is **not suitable for EDS** due to performance and architectural constraints.

4. Recommended EDS Approach – JSON + Block Decoration

Replace SSI-based HTML inclusion with:

- **Backend → Provides structured data (JSON)**
- **EDS Block → Handles rendering (HTML, CSS, JS)**

Authors continue to provide: Featured collection identifier or URL

The **featured-collection block** will:

- Read authored configuration from the page
- Read relevant query parameters from the URL

- Construct a request to the backend JSON API
- Fetch structured data
- Render and Apply
 - controlled CSS
 - minimal, optimized JavaScript
- The backend should expose a **JSON API** instead of returning HTML.

The response should include:

- collection metadata
- item list (promotions, categories, etc.)
- images and navigation links
- optional paging or filtering data

This ensures:

- Backend handles **business logic and data**
- EDS handles **presentation and interaction**

This approach ensures:

- No uncontrolled CSS/JS injection
- Reduced payload size (JSON vs HTML)
- Ability to:
 - lazy load content
 - defer non-critical logic
 - optimize rendering

Aligns with EDS best practices for **Core Web Vitals and performance**

Media Formulation

1. Executive Summary

The **Media Formulation** feature on [12320 - DMEM, low glucose, pyruvate, HEPES | Thermo Fisher Scientific - US](#) displays detailed scientific composition data — ingredients, molecular weights, concentrations, SKUs, references, and related products. Each product is currently served by a single AEM JSP component that queries a Microsoft SQL Server database at request time using a product ID passed as a URL selector (e.g. `media-formulation.48.html`).

No individual page is authored per product — a single dynamic template serves all of them.

This document defines the current state, the proposed EDS architecture, the technical implementation steps, and the ownership responsibilities between TFS and Adobe.

2. Current Architecture

2.1 Component Overview

The media formulation feature is implemented as an AEM JSP component (`mediaformulation.jsp`) backed by a custom OSGi service and tag library.

Modules involved:

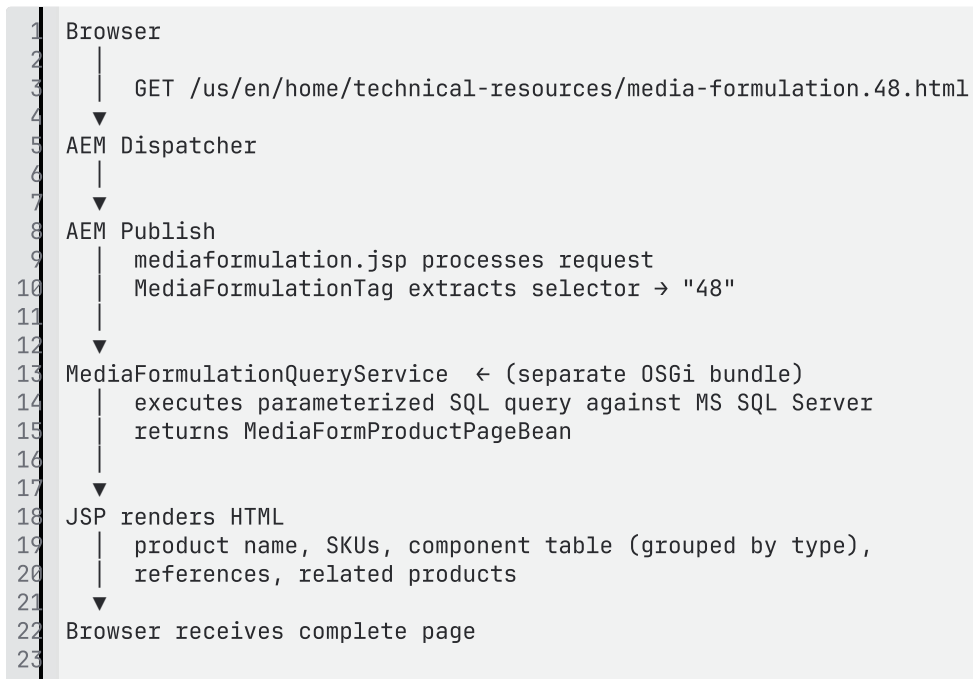
Module	Role
<code>lifetech-web-ui</code>	JSP template — renders the HTML table, SKUs, references, related products
<code>lifetech-web-taglib</code>	<code>MediaFormulationTag.java</code> — extracts URL selector, invokes the query service, binds data bean to JSP context
<code>lifetech-web-service-mediaformulation</code>	Data model beans — <code>MediaFormSKU</code> , <code>MediaFormComponentBean</code> , <code>MediaFormReferenceBean</code>

(*separate bundle* — not in corp-de-ce-tf-wcm repo)

MediaFormulationQueryService — OSGi service that executes the SQL Server query and returns MediaFormProductPageBean

Note: The corp-de-ce-tf-wcm repository does not contain the MediaFormulationQueryService implementation or MediaFormProductPageBean. These are present only as imported interfaces in the taglib. The actual SQL query logic resides in a **separate OSGi bundle installed in AEM**. This bundle will need to be located and reviewed before the API can be implemented.

2.2 Data Flow



2.3 URL Pattern

URL	Behaviour
/technical-resources/media-formulation.html	Base landing page — no product data (no selector)
/technical-resources/media-formulation.48.html	Product page — selector 48 passed to query service

/technical-resources/media-
formulation.{id}.html

Pattern for all products —
thousands of IDs

The product ID (selector) is the sole key used to look up all product data from the database.

2.4 Data Model

Data returned by the service for a single product:

```
1 Product
2   |— name
3   |— catalogDescription
4   |— additionalDescription
5   |— SKUs[] (list of catalog numbers)
6   |— mediaFormComponentTypes[]
7       |— type (e.g. "Amino Acids")
8           |— components[]
9               |— productName
10              |— molecularWeight (0 → blank)
11              |— concentration (0 → "confidential")
12              |— molarity (0/NaN → "n/a")
13   |— references[] (citation strings)
14   |— relatedProducts[]
15       |— productID
16       |— productName
17       |— skus[]
18
```

Note: This data model is **inferred** from the JSP rendering logic (`mediaformulation.jsp`) and the available DTO classes (`MediaFormComponentBean` , `MediaFormSKU` , `MediaFormReferenceBean`). The actual SQL schema and full field list have not been confirmed as the `MediaFormulationQueryService` implementation resides in a separate OSGi bundle not present in this codebase. The model must be validated against the actual DB schema before the JSON API contract is finalized — see dependency D2 and D3.

3. Proposed Architecture

3.1 Approach

The proposed solution uses the **JSON2HTML** service provided by Adobe EDS to render product pages at the edge — without creating individual pages per product and without client-side data fetching.

Reference: [JSON2HTML for Edge Delivery Services](#)

3.2 How JSON2HTML Works

JSON2HTML is an Adobe EDS edge worker service that intercepts incoming page requests, fetches data from a JSON API, and renders the HTML response using a **Mustache template** — entirely at the edge, before the response reaches the browser.

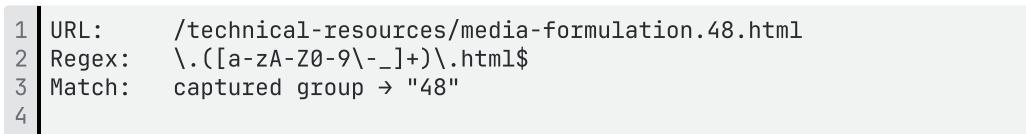


Key characteristics:

- The browser receives **fully rendered HTML** on the first byte — no JavaScript data fetching, no skeleton loaders
- A **single Mustache template** serves all products — no per-product authoring required
- CLS (Cumulative Layout Shift) = 0 is structurally guaranteed because all content is in the initial HTML
- The product ID is extracted from the existing URL format using a regex.

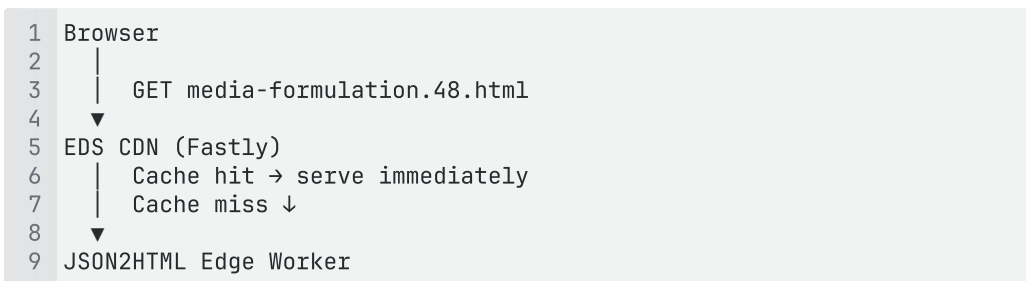
3.3 URL Routing Strategy

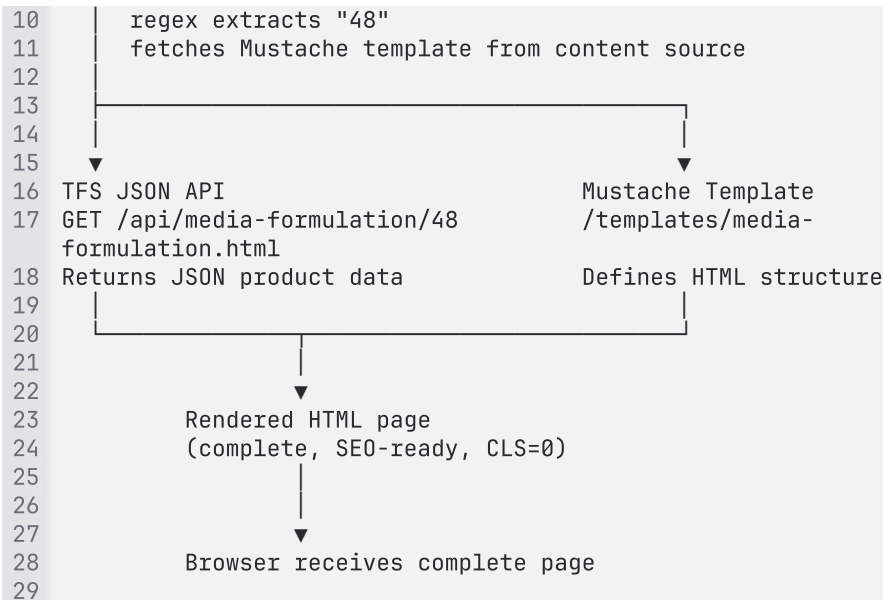
JSON2HTML uses a regex pattern configured per overlay to identify which URLs it handles and to extract the product ID:



This regex matches any URL with a selector (product ID) and extracts it cleanly.

3.4 High-Level Architecture





4. Technical Details

4.1 Add Overlay to Content Source

EDS uses the **Config Service** (repoless setup) to register content origins and overlays.

The AEM instance is registered as the content origin for the site, and the JSON2HTML overlay is registered as an interceptor for the media formulation URL pattern:

```

1 # Register AEM as content origin
2 curl -X POST \
3   "https://admin.hlx.page/config/{org}/{site}/content" \
4   -H "Authorization: token ${ADMIN_TOKEN}" \
5   -H "Content-Type: application/json" \
6   -d '{
7     "origin": {
8       "url": "https://author-p{programId}-e{envId}.adobecloud.com",
9       "type": "aem"
10    },
11    "overlay": {
12      "url":
13      "https://json2html.adobecloud.workers.dev/<ORG>/<SITE>/<BRANCH>",
14      "type": "markup"
15    }
16  }'
```

4.2 Register JSON2HTML

The JSON2HTML overlay is registered via its own Config Service endpoint.

```

1 POST
2 https://json2html.adobecloud.workers.dev/config/<ORG>/<SITE>/<BRANCH>
3 Authorization: token <your-admin-token>
4 Content-Type: application/json
5 [
6   {
7     "url":
8     "https://json2html.adobecloud.workers.dev/<ORG>/<SITE>/<BRANCH>",
9     "type": "markup"
10  }
```

```

6      "path": "/technical-resources/media-formulation\\.([a-zA-Z0-9\\-_
  _]+)\\.html",
7      "endpoint": "https://<some-endpoint/path>/{id}.json",
8      "regex": "/[^/]+$/",
9      "template": "/templates/media-formulation.html",
10     "headers": {
11       "X-API-Key": "your-api-key-here",
12       "Accept": "application/json"
13     },
14     "forwardHeaders": [
15       "Authorization",
16     ]
17   }
18 ]
19

```

4.3 Mustache Template

A single Mustache template (`/templates/media-formulation.html`) defines the HTML structure for pages. It is stored in AEM and served via the registered content source.

Mustache is a logic-less template language — all data formatting (e.g. `concentration = 0` → `"confidential"`) is done in the API response before the template receives it.

Template structure:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <title>{{name}} - Media Formulations | Thermo Fisher
Scientific</title>
5   <meta name="description" content="{{catalogDescription}}">
6   <link rel="canonical" href="/us/en/home/technical-resources/media-
formulation.{{productId}}.html">
7 </head>
8 <body>
9   <header></header>
10  <!-- Edge worker fills this - template does not need to know about
header content -->
11  <main>
12    <!-- Breadcrumb - empty wrapper; breadcrumb.js decorate() builds
trail from window.location -->
13    <div class="section">
14      <div class="breadcrumb"><div><div></div></div></div>
15    </div>
16    <!-- Product heading and description -->
17    <div class="section">
18      <h1>{{name}}</h1>
19      <p>{{catalogDescription}}</p>
20      <p>{{additionalDescription}}</p>
21    </div>
22    <!-- Single media-formulation block - contains all product content
-->
23    <!-- CSS uses descendant selectors to style each sub-section -->
24    <div class="section">
25      <div class="media-formulation">
26        <div><div>
27          <!-- SKUs -->
28          <div class="skus">
29            <p><strong>Catalog Number(s):</strong></p>
30            {{#skus}}<a
href="http://www.thermofisher.com/order/catalog/product/{{.}}">{{.}}

```



```

31 </a> {{/skus}}
32 </div>
33 <!-- Components table -->
34 <table>
35   <thead>
36     <tr>
37       <th>Components</th>
38       <th>Molecular Weight</th>
39       <th>Concentration (mg/L)</th>
40       <th>mM</th>
41     </tr>
42   </thead>
43   <tbody>
44     {{#componentTypes}}
45     <tr class="type-header">
46       <td colspan="4"><strong>{{productType}}</strong></td>
47     </tr>
48     {{#components}}
49     <tr>
50       <td>{{productName}}</td>
51       <td>{{molecularWeight}}</td>
52       <td>{{concentration}}</td>
53       <td>{{molarity}}</td>
54     </tr>
55   </tbody>
56 </table>
57 <!-- References (conditional) -->
58 {{#references.0}}
59 <div class="references">
60   <h2>References:</h2>
61   {{#references}}<p>{{.}}</p>{{/references}}
62 </div>
63 {{/references.0}}
64 <!-- Related Products (conditional) -->
65 {{#relatedProducts.0}}
66 <div class="related-products">
67   <h2>Related Products:</h2>
68   {{#relatedProducts}}
69     <p>
70       <a href="/us/en/home/technical-resources/media-
71 formulation.{{productId}}.html">{{productName}}</a><br>
72       Cat No: {{#skus}}{{.}} {{/skus}}
73     </p>
74   </div>
75   {{/relatedProducts}}
76 </div></div>
77 </div>
78 </div>
79 </main>
80 <footer></footer>
81 <!-- Edge worker fills this - template does not need to know about
82 footer content -->
83 </body>
84 </html>
85

```

4.4 Block Structure — Media Formulation

EDS blocks are identified by their CSS class names. Each block can optionally have a JavaScript file (`decorate()` function) and a CSS file for styling.

A new EDS block is only needed when a separate `decorate()` JavaScript function is required. Since JSON2HTML renders all product content (SKUs, component table, references, related products) server-side at the edge, there is nothing left for any JS to do. A single `media-formulation` block with one CSS file is sufficient — CSS descendant selectors handle styling of each sub-section inside the block.

CSS approach:

```
1 /* media-formulation.css */
2 .media-formulation table { ... }
3 .media-formulation .skus { ... }
4 .media-formulation .references { ... }
5 .media-formulation .related-products { ... }
6 .media-formulation .type-header td { ... }
7
```

EDS file structure:

```
1 /blocks/
2   media-formulation/
3     media-formulation.css  ← single CSS file, styles all sub-sections
4 /templates/
5   media-formulation.html   ← single Mustache template for all
6   products
```

4.5 Data Source Strategy — TFS Decision Required

Regardless of where the product data lives, the rendering solution remains **JSON2HTML** — the only difference is what the JSON API reads from. This is a business decision for TFS.

Two options are available:

Option A — Keep SQL Server (Recommended if DB is actively managed)

The existing SQL Server database continues to be the source of truth. The JSON API connects to the DB and returns product data as JSON for a given product ID. No data migration is required.

TFS should choose this option if:

- The DB is updated regularly by internal systems or teams
 - Other downstream applications (internal tools, integrations, reporting) also consume data from the same DB
 - The authoring/update workflow for formulation data is already established around the DB
-

If TFS decides to retire the DB dependency for this feature, product formulation data could be maintained in spreadsheets instead. EDS exposes spreadsheet data as a JSON API automatically .

TFS should consider this option if:

- The DB is the only consumer of this formulation data (no other downstream dependencies)
- The team maintaining the data prefers a simpler, spreadsheet-based workflow over DB administration
- TFS wants to reduce infrastructure dependencies and operational overhead of the DB connection

Important: This is entirely a TFS business decision. TFS knows best — who updates the DB, how frequently, and which downstream systems depend on it. Adobe's role is to highlight this option; the choice rests with TFS.

In both cases, JSON2HTML is the rendering solution. The Mustache template, URL routing, block structure, and EDS configuration remain identical. Only the API implementation changes.

	Option A — SQL Server	Option B — Spreadsheets
Data source	Microsoft SQL Server (existing)	EDS Spreadsheets
Data migration	None	Full migration of all product records to sheets
Downstream impact	None — other DB consumers unaffected	Must confirm no other system reads from this DB table
Authoring workflow	Existing DB update process unchanged	Editors update spreadsheet rows directly
Rendering	JSON2HTML + Mustache	JSON2HTML + Mustache (identical)

4.6 API Middleware — JSON Data Source

(Applicable if Option A — SQL Server is chosen)

JSON2HTML requires a JSON API endpoint that accepts a product ID and returns structured product data. This API acts as the replacement for `MediaFormulationQueryService` in the current AEM architecture.

This API must be built and owned by TFS.

TFS is best positioned to own this API because:

- TFS owns the SQL Server database, credentials, and schema
- TFS understands the data access patterns, security requirements, and downstream consumers
- TFS controls the DB infrastructure, connection management, and SLA
- Adobe has no visibility into or access to the TFS database

Adobe's role is to consume the API endpoint from the JSON2HTML worker. The hosting platform, technology stack, and implementation approach are TFS's choice — it can be a REST API on any stack TFS already operates (Node.js, Java Spring, .NET, etc.).

Endpoint contract (TFS to implement):

```
1 GET {tfs-api-base-url}/media-formulation/product/{productId}
2
```

Expected JSON response:

```
1 {
2   "productId": "48",
3   "name": "DMEM, High Glucose",
4   "catalogDescription": "Dulbecco Modified Eagle Medium...",
5   "additionalDescription": "For use in cell culture...",
6   "skus": ["11965092", "11965118"],
7   "componentTypes": [
8     {
9       "productType": "Amino Acids",
10      "components": [
11        { "productName": "Glycine", "molecularWeight": "75.03",
12          "concentration": "30.0", "molarity": "0.4" },
13        { "productName": "L-Arginine HCl", "molecularWeight":
14          "210.66", "concentration": "confidential", "molarity": "n/a" }
15      ]
16    }
17  ],
18  "references": ["1. Dulbecco R and Freeman G (1959) Virology
19    8:396."],
20  "relatedProducts": [
21    { "productId": "49", "productName": "RPMI 1640 Medium", "skus":
22      ["11875093"] }
23  ]
24 }
```

Data formatting rules (applied in API, not in Mustache):

Field	Rule
concentration	DB value 0 → return string "confidential"
molarity	DB value 0 or NaN → return string "n/a"
molecularWeight	DB value 0 → return empty string ""
references	Pre-number each entry (e.g. "1. Dulbecco...") — Mustache cannot compute indexes

API contract requirements:

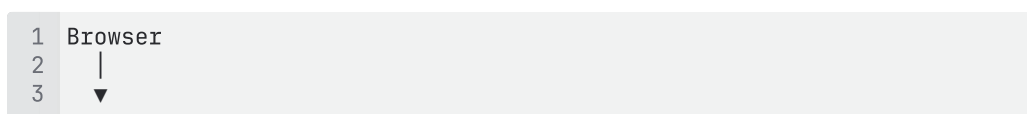
Status	Condition
200	Product found — return JSON
404	Product ID not found in database
400	Invalid product ID format
503	Database unavailable

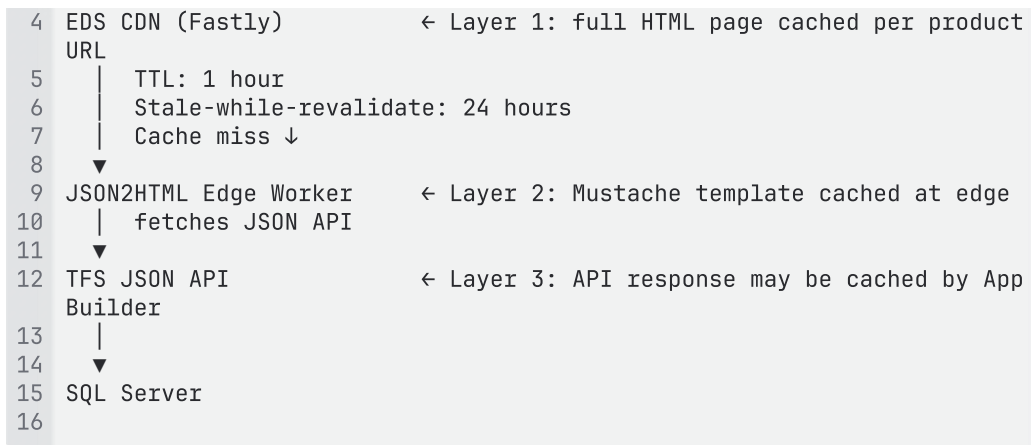
Note: The SQL query logic from `MediaFormulationQueryService` (which resides in a separate OSGi bundle not present in `corp-de-ce-tf-wcm`) must be reverse-engineered or obtained from TFS before this API can be implemented. This is a key prerequisite. This API is only required if TFS chooses **Option A (SQL Server)** — see section 4.5.

4.7 Caching & Cache Invalidation

Because product data is fetched dynamically from the TFS JSON API, caching is critical — both to protect the API from request volume and to ensure users are not served stale formulation data after a DB update.

Cache layers:





Cache invalidation — when a product record is updated in the DB:

The key question for TFS is: *how quickly must the website reflect a DB change?*

With a 1-hour TTL, stale content can serve for up to 1 hour after a DB update. If near-real-time updates are required, a cache purge mechanism must be implemented:

```

1  DB record updated
2  → TFS system fires a webhook to Adobe cache-purge endpoint
3  → Adobe purges the specific product URL from the CDN
   (e.g. purge media-formulation.48.html only)
4  → Next request re-renders via JSON2HTML with fresh API data
5  → Updated content live within seconds of purge
6
7

```

If near-real-time is not a requirement and hourly staleness is acceptable, no purge mechanism is needed — the TTL alone is sufficient.

TFS must confirm: What is the acceptable staleness window for formulation data changes?

This determines whether a cache purge integration is in scope.

Staleness Acceptable	Approach	Complexity
Up to 1 hour	TTL-only caching — no additional work	Low
Near real-time (< 5 min)	DB change triggers cache purge webhook	Medium
Immediate	TTL = 0, no CDN caching — every request hits API	Not recommended — high API load

4.8 Error Handling & Fallback Behavior

The following defines the expected behavior when the TFS JSON API is unavailable or returns an error. This must be agreed between TFS and Adobe as part of the API contract.

Scenario	API Response	User Experience	CDN Caches Response?
Product found	HTTP 200 + JSON	Page renders normally	Yes — per TTL
Product ID not found in DB	HTTP 404	EDS serves the site's standard 404.html error page	Yes — briefly (5 min) to protect against crawlers
Invalid product ID format	HTTP 400	EDS serves 404.html	No
API timeout / DB unavailable	HTTP 503	EDS serves a generic error page or the last cached version if stale-while-revalidate is active	No

Stale-while-revalidate as a resilience mechanism:

With `stale-while-revalidate: 86400` (24 hours) configured in the JSON2HTML overlay, if the API returns a 503, the CDN can continue to serve the last successfully cached HTML for up to 24 hours while retrying in the background. This means a temporary API outage is invisible to end users for pages that were previously cached.

5. Runtime Ownership & Integration

The solution has a clear split in ownership between TFS and Adobe:

Component	Owner	Description
JSON API endpoint	TFS	Built, hosted, and maintained entirely by

		TFS. TFS owns the DB, credentials, and business logic — Adobe has no access to the database. Technology stack and hosting platform are TFS's choice. This is the single most critical dependency for the entire migration.
SQL Server access & credentials	TFS	DB connection, credentials, schema, and query logic remain fully with TFS.
JSON2HTML service	Adobe	Adobe-managed edge worker. Adobe configures and maintains this service.
Mustache template	Adobe	Developed by Adobe. Stored in AEM content source. Registered with JSON2HTML overlay.
EDS Config Service registration	Adobe	Adobe registers the content origin and JSON2HTML overlay via Config Service API.
EDS block CSS	Adobe	CSS-only styling files for all media-formulation blocks.

Integration point:

The sole integration point between TFS and Adobe is the **JSON API**. Adobe's JSON2HTML worker calls the TFS-provided API endpoint — this is the only interface between the two

systems. Adobe does not touch the database, the query logic, or the hosting infrastructure.

Both parties need to agree and sign off on the following before implementation begins:

1. **API endpoint URL** — the base URL TFS will expose the API on
 2. **JSON response schema** — as defined in section 4.6; TFS must conform to this contract exactly
 3. **Authentication mechanism** — how the JSON2HTML worker authenticates to the TFS API (e.g. API key in request header, IP allowlist, mutual TLS)
 4. **SLA** — response time target (recommended p95 < 200ms);
 5. **Availability** — expected uptime; section 4.8 covers what happens on API outage
-

6. Dependencies

The migration cannot proceed to implementation without the following:

#	Dependency	Owner	Blocker for
D0	Data source decision — TFS to confirm whether product data stays in SQL Server (Option A) or is migrated to spreadsheets (Option B) — see section 4.5	TFS	All implementation
D1	JSON API endpoint — TFS to build, host, and expose an API that accepts a product ID and returns JSON conforming to the schema in section 4.6. Required only if Option A (SQL Server) is chosen. Adobe cannot proceed with JSON2HTML configuration or end-to-end testing until this	TFS	JSON2HTML configuration, Mustache template testing, end-to-end validation

	endpoint is available in a staging environment.		
D2	SQL Server schema confirmation — full table structure, FK relationships, indexes	TFS (DBA)	API implementation
D3	<code>MediaFormulationQueueService</code> source code — the separate OSGi bundle not present in <code>corp-de-ce-tf-wcm</code> — needed to understand exact SQL query logic	TFS	API implementation
D5	Config Service credentials — org/site identifier and admin token for the repoless EDS setup	Adobe	Config Service registration

Product List

1. Summary

The Product List component displays product catalogs with pricing on web pages, allowing users to view and purchase products.

References:

[T Celleste Image Analysis Software | Thermo Fisher Scientific - US](#)

[T EVOS LED Light Cubes | Thermo Fisher Scientific - US](#)

[T Molecular Biology Bulk Reagents and Lab Supplies for Sample Preparation | Thermo Fisher Scientific - US](#)

[T Transfected Cell Analysis | Thermo Fisher Scientific - US](#)

Key fact: Product data (names, sizes, pricing, availability) is NOT authored. It is fetched dynamically at runtime from the Product Microservice based on the SKU list configured by the author. This block defines WHAT to show (SKUs + columns). The Edge Worker + Microservice handle HOW to render it.

Block Overview

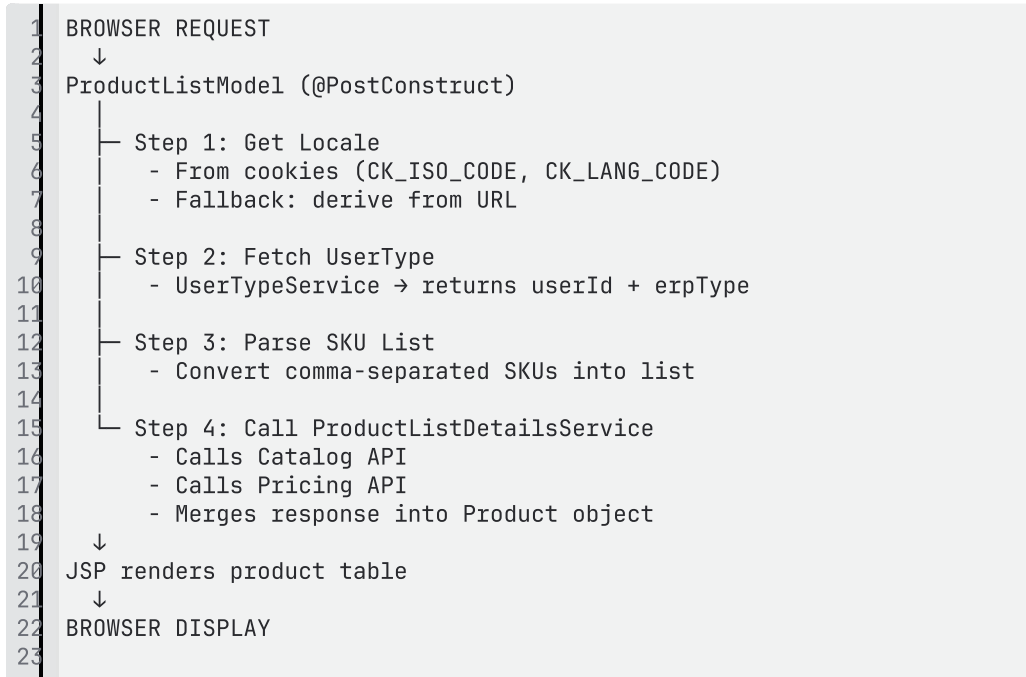
Property	Value
Block Name	Product List
Authoring Strategy	Flattened block — key-value configuration table. Author provides SKU list and column display preferences. Product data is fetched dynamically at runtime — not authored.
Authoring Source	DA (Document Authoring)
Delivery	Edge Delivery Services (EDS) via Edge Worker

2. Current State

The Product List component is a flexible, data-driven AEM component that renders product information in a table format. It:

- Fetches product details via Catalog Microservice API based on SKU list
- Retrieves pricing via Pricing Access API with user-specific pricing
- Renders a dynamic HTML table with configurable columns (SKU, Name, Size, Price, Quantity)
- Supports Add-to-Cart via client-side JavaScript
- Handles multiple price access types (RequestQuote, OrderNow, NoPrice, etc.)

Data Flow (Current)



Service Layer (Current)

ProductListDetailsServiceImpl performs:

- UserType retrieval
- OAuth token generation via AuthTokenProvider
- Catalog API call
- Pricing API call
- Consolidation into a single Product response

Frontend (Current)

- Only renders data provided by backend
- Handles Add-to-Cart: validates quantity, calls mini-cart
- No API calls from browser

3. EDS Approach

EDS does not support AEM-style server-side rendering. The published HTML is static.

To support dynamic product data while keeping SEO and security intact, a **two-layer architecture** is proposed:

1. Edge Worker (Presentation Layer)

- Reads product-list block from page
- Extracts SKU list and configuration
- Calls Product Microservice
- Renders HTML using returned JSON
- Injects final HTML into response

2. Product Microservice (Data Layer)

- Handles OAuth token generation
- Resolves user context (UserType)
- Calls: Catalog API, Pricing API
- Merges responses
- Returns normalized JSON

Product List Block (Authoring)

The EDS Product List block allows authors to configure:

- SKU list (comma-separated)
- Column selection: Size, Price, Quantity, Add-to-Cart, PDP Link

This ensures flexibility similar to the current AEM component while keeping authoring simple.

Important Design Decision

- Business logic is NOT implemented in Edge Worker
- Edge should remain thin (rendering only)
- Microservice should handle all backend logic
- Microservice can be implemented using: App Builder, AWS Lambda, or Node.js service

4. Architecture

```
1 EDS Page (Product List Block)
2   ↓
3 Edge Worker
4   - Read SKUs and column config
5   - Call Microservice
```

```
6 - Render HTML from JSON response
7 ↓
8 Product Microservice
9 - Auth Token
10 - UserType
11 - Catalog API
12 - Pricing API
13 - Merge response
14 ↓
15 Backend APIs (existing)
16
```

Additional Notes

- Add-to-Cart remains client-side (same as current)

Recommendation

Adopt Edge Worker + Product Microservice pattern:

- Keeps API logic secure and outside browser
- Maintains SEO via server-side rendering at edge
- Keeps edge layer simple and maintainable
- Reuses existing backend logic effectively

5. Client-Side Behaviour

5.1 Quantity Validation

Client-side quantity validation is handled within the EDS block JS, same as the current implementation. The existing validation logic from `lineItem.js` will be ported:

- Minimum/maximum quantity limits
- Integer-only checks
- Non-zero validation
- Error messaging on invalid input

This runs in the browser — no server call needed for validation.

5.2 Analytics (Product Impressions & Add-to-Cart)

In the current AEM implementation, product impressions and Add-to-Cart events are pushed to `digitalData` / the AEM data layer. In EDS, there is no AEM data layer.

Analytics tracking will be implemented directly within the block JS:

- **Product impressions** — pushed when product rows are rendered and visible to the user
- **Add-to-Cart events** — pushed when the user clicks the Add to Cart button

The event structure will be kept consistent with existing analytics expectations where applicable. This aligns with the EDS approach where tracking is handled at the block level.

5.3 Add-to-Cart

Add-to-Cart remains client-side (same as current):

1. User enters quantity and clicks Add to Cart
2. Block JS validates quantity (see 5.1)
3. Block JS calls the Mini-Cart service
4. Mini-Cart service handles cart operations and stock validation
5. Block JS updates the UI (success/error feedback)

6. Integration Contract

Aspect	Specification
Request	SKU list + locale/country + user context (cookie/token if authenticated)
Response	Normalized JSON — array of product objects with name, SKU, size, price, availability, price access type
Authentication	API Key in request header (X-API-Key) or mTLS — exact mechanism depends on deployment model, network setup, and security standards
SLA Target	< 500ms (p95 response time) for typical request (5–20 SKUs)
Timeout	2 seconds at the Edge Worker — if microservice does not respond within 2s, fallback is triggered
Method	POST (SKU list in request body) or GET (SKUs as query parameter) — to be agreed

The integration contract between the Edge Worker and the Product Microservice will be defined and agreed jointly during implementation.

7. Cache Strategy

User Type	Strategy	Details
Anonymous users	Edge-cached	Cache key based on country + SKU list. Short TTL (5–15 minutes). Price changes reflect within TTL window. Refreshed on expiry.
Logged-in users	All caches bypassed	Every request fetches live, personalized pricing from microservice. No edge caching for authenticated requests.
Fallback	Stale-while-revalidate	If microservice is temporarily unavailable, serve last cached response (anonymous users only) to avoid complete failure.

Final TTL is a business decision — trade-off between performance (longer TTL) and pricing freshness (shorter TTL). Will be agreed during implementation.

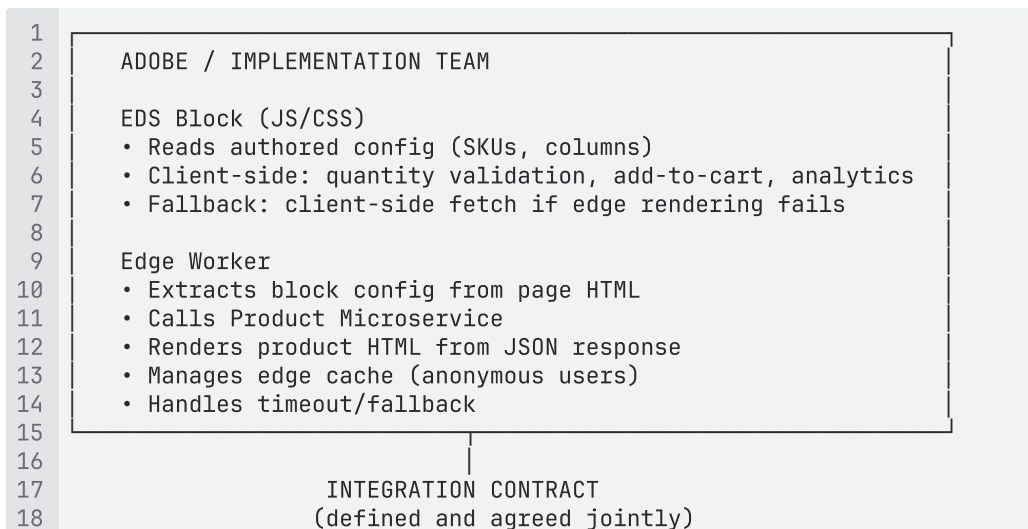
8. Ownership and Boundaries

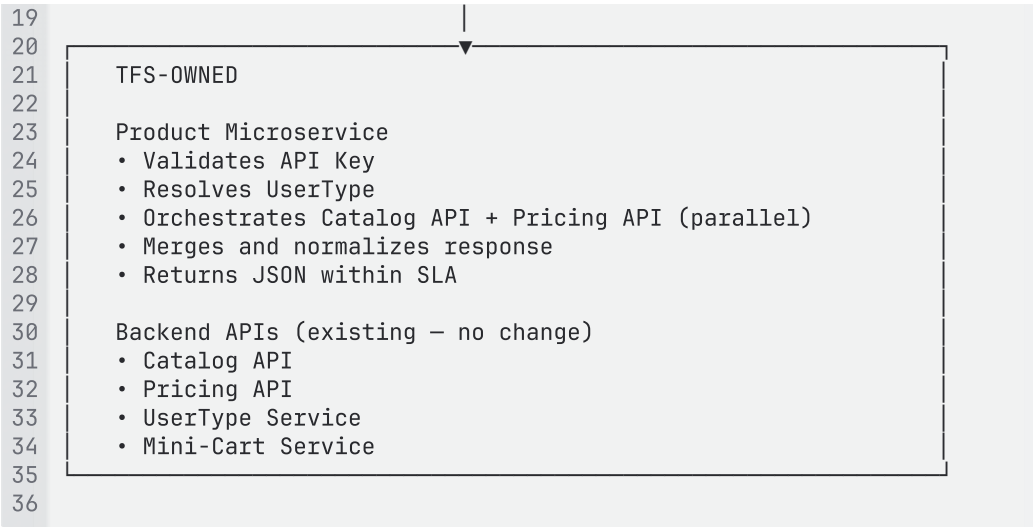
Ownership Matrix

Component	Owner	Responsibilities
Product List Block (EDS)	Adobe / Implementation Team	Block JS/CSS, client-side rendering (fallback), quantity validation, analytics

		event push, add-to-cart UX
Edge Worker	Adobe / Implementation Team	Extract SKU/column config from page HTML, call Product Microservice, render product HTML table, inject into response, manage cache, handle timeout/fallback
Product Microservice	TFS	UserType resolution, Catalog API calls, Pricing API calls, response merging, normalized JSON response, SLA ownership
Mini-Cart Service	TFS (existing backend)	Cart operations, stock validation, order management
Integration Contract	Joint (Adobe + TFS)	Request/response schema agreed during implementation

Integration Boundaries





9. DA Block Table Contract

9.2 Table Structure

The Product List block uses a **key-value pair** pattern. Each row is a configuration property.

Column	Role	Content
Column 1	Key	Configuration property name
Column 2	Value	Configuration property value

9.3 Available Properties

Key (Column 1)	Value (Column 2)	Required	Default	Description
sku-list	Comma-separated SKU IDs	Yes	—	Product SKUs to display (e.g. 16096040, 15596018, A33251, 17909)
show-size	true / false	No	true	Show or hide the Size

				column
show-price	true / false	No	true	Show or hide the List Price column
show-quantity	true / false	No	true	Show or hide the Quantity input column
show-add-to-cart	true / false	No	true	Show or hide the Add to Cart button column
show-pdp-link	true / false	No	false	Show or hide a link to the Product Detail Page

Only include rows for properties that need to differ from defaults. If all columns should be shown, only the `sku-list` row is needed.

9.4 How the Block Works at Delivery Time

1. Edge Worker reads the Product List block from the page HTML
2. Extracts `sku-list` and column configuration from the block table rows
3. Determines locale from URL structure
4. Calls the Product Microservice with SKU list, locale, and user context
5. Receives normalized JSON response (product name, SKU, size, price, availability, price access type)
6. Renders an HTML product table based on column configuration
7. Injects rendered HTML into the response before delivering to the browser
8. Client-side JS handles Add-to-Cart interactions, quantity validation, and analytics

9.5 Authoring Summary

Concern	Approach
Block type	Flattened block — key-value configuration table

SKU list	Author enters comma-separated SKU IDs
Column visibility	Author sets boolean rows (true/false) for each column
Product data	NOT authored — fetched dynamically from Product Microservice
Pricing	NOT authored — fetched dynamically with user-specific pricing
Add-to-Cart	Client-side — handled by block JS
Rendering	Edge Worker renders HTML from microservice JSON response
Default behaviour	All columns shown. Only <code>sku-list</code> row is strictly required.

Video and Video playlist

1. Block Overview

TFS uses Brightcove Video Cloud as its video hosting platform. The EDS implementation uses **two separate blocks**:

Block	Purpose	Author Fields
Video	Embed a single Brightcove video	Account, Video ID, Display Mode, conditional fields
Video Playlist	Embed a Brightcove playlist	Account, Playlist ID, Player, Display Mode, conditional fields

Why Separate Blocks

- **Simplified authoring** — authors directly choose between video or playlist without conditional fields or complex dialogs
- **Clear authoring intent** — explicit selection reduces ambiguity
- **Different rendering structures** — single player vs rail/list UI requiring different DOM and styling
- **Different Brightcove configurations** — video and playlist may require different player configurations and script loading
- **Independent styling** — playlist-specific layouts (rails, scrolling) handled independently from video rendering

References

[T Real-Time PCR | Thermo Fisher Scientific - US](#)

[T 3D SEM | Volumescope 2 | Scanning Electron Microscope | Thermo Fisher Scientific - US](#)

[T Greener by design | Thermo Fisher Scientific - US](#)

[T Gibco Cell Culture Basics | Thermo Fisher Scientific - US](#)

2. Authoring Criteria

2.1 Video Block

Account — required. The Brightcove account (friendly name). Selected via plugin dropdown.

Video ID — required. The Brightcove video ID.

Display Mode — required. Determines how the video appears on the page (Inline, Button, Link, Thumbnail, Teaser).

Conditional fields (based on display mode):

- Button/Link mode: CTA text
- Teaser mode: Title, Description, Image

2.2 Video Playlist Block

Account — required. The Brightcove account (friendly name). Selected via plugin dropdown.

Playlist ID — required. The Brightcove playlist ID.

Player — required. The player configuration (Default, Playlist with right rail, Playlist with bottom rail). Selected via plugin dropdown.

Display Mode — required. Determines how the playlist appears on the page.

Conditional fields (based on display mode):

- Button/Link mode: CTA text
- Teaser mode: Title, Description, Image

2.3 Authoring Principle

Authors see **friendly account names** (CBD, PGH, FEI) — never Brightcove account IDs, player IDs, or script URLs. The Brightcove configuration sheet handles name-to-ID resolution at runtime.

All authoring is done via the **Brightcove plugin** in the DA library panel. Authors do not manually create or edit block tables for video content.

3. Display Modes (Variants)

Both blocks support the same set of display modes. The display mode is written as the **variant** in the block header by the plugin.

Display Mode	Variant	Behavior	Author Configures
Inline	(default — no variant)	Player renders directly on page and plays in place	Account + Video/Playlist ID only
Button	button	A CTA button is displayed. Clicking opens the player in a modal overlay.	+ CTA text
Link	link	A text link is displayed. Clicking opens the player in a modal overlay.	+ CTA text
Thumbnail	thumbnail	Video thumbnail displayed. Clicking opens the player in a modal overlay.	Account + Video/Playlist ID only (thumbnail auto-fetched from Brightcove)
Teaser	teaser	A card with thumbnail, title, and description. Clicking opens the player in a modal overlay.	+ Title + Description + optional Image

4. DA Block Table Contract — Video Block

4.1 Table Structure

The Video block uses a key-value pair pattern. The display mode is the variant in the block header.

Column	Role	Content
Column 1	Key	Property name
Column 2	Value	Property value

4.2 Available Properties

Key	Value	Required	Description
<code>account</code>	Account friendly name (e.g. CBD, PGH, FEI)	Yes	Resolved to Brightcove account ID via config sheet
<code>video-id</code>	Brightcove video ID	Yes	The specific video to embed
<code>cta-text</code>	Button/link label text	Only for Button/Link mode	Text displayed on the CTA
<code>title</code>	Teaser card title	Only for Teaser mode	Heading displayed on the teaser card
<code>description</code>	Teaser card description	Only for Teaser mode	Supporting text on the teaser card
<code>image</code>	Teaser card image	Optional for Teaser mode	Custom thumbnail — if omitted, auto-fetched from Brightcove

4.3 Block Header by Display Mode

Display Mode	Block Header
Inline	<code>Video</code>

Button	Video (button)
Link	Video (link)
Thumbnail	Video (thumbnail)
Teaser	Video (teaser)

5. DA Block Table Contract — Video Playlist Block

5.1 Table Structure

Same key-value pattern as Video block.

5.2 Available Properties

Key	Value	Required	Description
account	Account friendly name	Yes	Resolved to Brightcove account ID via config sheet
playlist-id	Brightcove playlist ID	Yes	The specific playlist to embed
player	Player type: default, right-rail, bottom-rail	Yes	Determines playlist layout — resolved to player ID via config sheet
cta-text	Button/link label text	Only for Button/Link mode	Text displayed on the CTA
title	Teaser card title	Only for Teaser mode	Heading on teaser card
description	Teaser card description	Only for Teaser mode	Supporting text on teaser card

image	Teaser card image	Optional for Teaser mode	Custom thumbnail
-------	-------------------	--------------------------	------------------

5.3 Block Header by Display Mode

Display Mode	Block Header
Inline	Video Playlist
Button	Video Playlist (button)
Link	Video Playlist (link)
Thumbnail	Video Playlist (thumbnail)
Teaser	Video Playlist (teaser)

6. Brightcove Plugin — Authoring Experience

6.1 Overview

The Brightcove plugin is available in the DA library panel. It provides guided authoring for both Video and Video Playlist blocks in a single plugin with two tabs.

6.2 Plugin Dialog

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

Brightcove

Close

[Video] [Playlist]

Account *

CBD ▼

(options: CBD, PGH, FEI – from config sheet)

Video ID *

6293625135001

Display Mode *

Teaser ▼

(options: Inline, Button, Link, Thumbnail, Teaser)

— Conditional fields (Teaser selected) —

Title

28

Are you ready to Connect?

29

30

31

Description

32

The Connect Platform delivers a secure...

33

34

35

Image (optional)

36

[Browse/Upload]

37

38

39

40

41

Add to Page

42

43

44

Cancel

45

46

47

6.3 Playlist Tab (Additional Field)

When the **Playlist** tab is selected, an additional **Player** dropdown appears:

1

Player *

2

3

Playlist with bottom rail ▼

4

5

(options: Default, Playlist with right rail,

6

Playlist with bottom rail – from config sheet)

7

6.4 Conditional Fields by Display Mode

Display Mode selected	Additional fields shown
Inline	None
Button	CTA Text
Link	CTA Text
Thumbnail	None
Teaser	Title, Description, Image (optional)

6.5 What the Plugin Outputs

The plugin writes the **complete block table** to the DA document — including the header row with the correct variant.

Author selection	Plugin writes
------------------	---------------

Video + Inline	Video block table with account + video-id rows
Video + Button	Video (button) block table with account + video-id + cta-text rows
Video + Teaser	Video (teaser) block table with account + video-id + title + description + image rows
Playlist + Inline	Video Playlist block table with account + playlist-id + player rows
Playlist + Button	Video Playlist (button) block table with account + playlist-id + player + cta-text rows

6.6 Edit Existing Block

When the author selects an existing Video or Video Playlist block and opens the Brightcove plugin:

- Plugin reads the existing block table
- Pre-populates all fields (account, ID, display mode, conditional fields)
- Author modifies what's needed
- Clicks **Update** → plugin rewrites the block table

7. Multi-Video Grid Layout

When multiple videos need to be displayed in a grid layout (e.g. 3 videos per row, 4 videos per row), authors use **Section Metadata** to apply a grid style to the section containing the video blocks.

7.1 How to Author

1. Place multiple Video blocks in the same section
2. Add Section Metadata at the end of the section with a grid style

7.2 Section Grid Styles

Section Metadata style	Layout
------------------------	--------

3-col	3 videos per row
4-col	4 videos per row
2-col	2 videos per row

7.3 Example — 3 Videos in a Row

```

1 | Video (thumbnail) | |
2 | account | CBD |
3 | video-id | 6293625135001 |
4
5 | Video (thumbnail) | |
6 | account | CBD |
7 | video-id | 7382946251001 |
8
9 | Video (thumbnail) | |
10 | account | CBD |
11 | video-id | 8491057362001 |
12
13 | Section Metadata | |
14 | style | 3-col |
15

```

What renders: Three video thumbnails side by side in a 3-column grid. Clicking any thumbnail opens the video in a modal overlay.

7.4 Note

The 3-col, 4-col, 2-col section styles are generic — they work for any blocks placed in a section, not just videos. This is the DA/EDS equivalent of AEM's Layout Container column count.

Further details on Brightcove integration can be referenced from the provided URL. [Brightcov](#)

e

Dynamic Merchandising Offer

1. Overview

The offer system delivers personalized promotional content via Adobe Target / AEP Edge Network, with fallback to default offer content when personalized offers are unavailable.

The EDS implementation consists of two blocks:

Block	Purpose
Dynamic Offer	Smart container on content pages — holds one placement code and a default offer path. At runtime, receives personalized offer from Target or falls back to default.
Default Offer	Fragment page that stores the raw HTML of the fallback offer. Used when Target does not return a personalized offer.

Key principle: Marketing team continues authoring offers in their Marketing AEM instance. No migration of offer authoring is required. Only the default offer Experience Fragments from the TFS On-Prem instance need to be migrated to EDS fragment pages.

2. Assumptions

#	Assumption
1	Offers are authored on the Marketing Cloud instance and pushed to Target/AEP from the Marketing environment.
2	The Marketing Cloud instance and the TFS AEM instance maintain separate code repositories — no shared codebase.
3	Offer HTML is delivered as Experience Fragment HTML. Styling is applied through CSS and JS loaded from the global header (<code>dm-offers/dm-offer-style.css</code> and <code>dm-offers/preload.min.js</code>).
4	<code>dm-offer-style.css</code> and <code>preload.min.js</code> will be loaded via the EDS global header/footer or via <code>delayed.js</code> . These are

	responsible for offer styling and fallback behaviour.
5	The Adobe cloud pipeline (Marketing AEM → Target → AEP Edge → Alloy SDK → DOM injection) remains unchanged. Only the page-side implementation changes for EDS.
6	Placement codes are a known, finite set managed centrally (e.g. <code>mk_fp_01</code> , <code>mk_fp_02</code> , <code>hb</code> , <code>lb</code> , <code>pc</code> , <code>fad_s_1</code>).
7	Default offers are not region/language specific — stored under a common path. In EDS, default offer fragments will follow a centralized structure at <code>/fragments/default-offers/</code> .
8	PDP, Commerce (cart), Search, and other systems consume offers directly from AEP/Target. No offers are created for them in the TFS AEM instance. No impact on these systems.
9	No offers are authored directly in the TFS AEM instance — only default XF-based offer HTML is stored there.
10	Images in offers are served from Dynamic Media (dm-images.thermofisher.com).
11	Offer placement in the Header and Footer comes from the Header/Footer HTML — the Dynamic Offer block is not used for header/footer offers.
12	AEP/Target requires country, language, and user type information for decisioning. This data is derived from the EDS page URL structure and browser cookies.

3. Current State Architecture

Flow

1. Marketing team creates and manages offer content as Experience Fragments in Marketing AEM (Cloud)
2. Raw HTML of the default offer is copied from Marketing AEM Publish to TFS AEM On-Prem as an Experience Fragment
3. The Dynamic Merchandising Offer component is authored on the page — configured with a Placement Code and Default Offer XF path

4. On page load, Alloy SDK (AEP Web SDK) makes a client-side call to `edge.adobedc.net/ee/v1/interact`
5. AEP Edge routes the request to Adobe Target for offer decisioning
6. If Target returns a personalized offer → Alloy SDK injects the Experience Fragment HTML into the placement container
7. If Target returns no offer → Default offer is displayed

Key Technical Finding

The injected Experience Fragment HTML is clean and lightweight:

- Zero scripts, zero stylesheets, zero iframes
- Composed entirely of divs, images, links, text, and hidden inputs
- Not problematic for EDS pages — can be safely injected at runtime

4. EDS Solution — Recommended Approach

Consume Offers from Marketing Team

1. Marketing team continues authoring XFs in their AEM instance
2. Automation pushes offer HTML from Marketing AEM to AEP
3. AEP stores offer as HTML payload (current method — no change)
4. Target returns full HTML in AEP response
5. EDS page renders the returned HTML string

What Changes for EDS

Concern	Current (AEM)	EDS
Offer authoring	Marketing AEM instance	Marketing AEM instance — no change
Offer delivery	Target → Alloy SDK → DOM injection	Target → Alloy SDK → DOM injection — no change
Default offer storage	XF in TFS AEM On-Prem	Default Offer fragment in DA
Page-level block	Dynamic Merchandising Offer component	Dynamic Offer block (DA table)

Placement container	<code><div id="tf-cq-mk_fp_01"></code>	Block renders same container ID
Styling	dm-offer-style.css via global header	Same CSS loaded via EDS delayed.js

What Does NOT Change

- Marketing team offer authoring workflow
- AEP/Target pipeline and decisioning
- Alloy SDK client-side behaviour
- Offer HTML content structure
- Other systems (PDP, Commerce, Search) that consume from AEP/Target

What Needs Migration

Only the default offer Experience Fragments from TFS AEM On-Prem need to be migrated to EDS fragment pages:

- Path: `/content/experience-fragments/tfsite/us/en/site/dm-offers/`
- Each XF becomes a Default Offer fragment page in DA

5. Block Definitions

5.1 Dynamic Offer Block

Property	Value
Block Name	Dynamic Offer
Purpose	Smart container on content pages. Holds one placement code and default offer path. At runtime, receives Target offer or shows default.
Authoring Strategy	Flattened block — key-value table. One block per offer placement.
Instance per page	Multiple — one per placement position needed on the page

5.2 Default Offer Block

Property	Value
Block Name	Default Offer
Purpose	Stores raw HTML of the fallback offer content. Used when Target does not return a personalized offer.
Authoring Strategy	Embed-style block — author pastes raw HTML from Marketing AEM into the block. HTML is preserved as-is.
Location	Fragment pages at <code>/fragments/default-offers/</code>
Instance per fragment	One per fragment page

6. DA Block Table Contract — Dynamic Offer

6.1 Table Structure

The Dynamic Offer block uses a **key-value pair** pattern. Each block represents **one offer placement**.

Column	Role	Content
Column 1	Key	Property name
Column 2	Value	Property value

6.2 Available Properties

Key	Value	Required	Description
<code>placement-code</code>	Placement code string	Yes	Identifies the offer position. Selected via plugin dropdown. Determines the container ID

			used by Alloy SDK for Target injection.
default-offer	Link to default offer fragment	Yes	Path to the Default Offer fragment page that holds the fallback HTML.

6.3 Placement Codes

Placement Code	Friendly Name	Description
mk_fp_01	Feature Panel Left	Left feature panel on content pages
mk_fp_02	Feature Panel Right	Right feature panel on content pages
hb	Header Banner	Full-width header banner
lb	Lightbox	Lightbox popup offer
pc	Promo Cards	Promotional cards section
fad_s_1	Find A Distributor	Find a Distributor section offer

6.4 Runtime Flow

1. Page loads → Dynamic Offer block JS creates a container `<div>` with ID derived from placement code (e.g. `tf-cq-mk_fp_01`)
2. Alloy SDK (loaded via global header) detects the container
3. Alloy SDK calls AEP Edge → Target for offer decisioning
4. If Target returns personalized offer HTML → Alloy SDK injects it into the container
5. If Target returns nothing within timeout → block JS fetches the default offer fragment's `.plain.html` and injects it as fallback

6.5 Authoring Summary

Concern	Approach
Block type	Flattened block — key-value table
One block per placement	Yes — each Dynamic Offer block handles one placement code
Placement code	Selected via plugin dropdown (inserts value into cell)
Default offer	Link to default offer fragment page
Personalized offer	Delivered by Target at runtime — no authoring needed
Multi-offer layout	Use Section Metadata for grid (see Section 9)

7. DA Block Table Contract — Default Offer

7.1 Purpose

The Default Offer block stores raw HTML content from Marketing AEM. This HTML includes inline styles, CSS classes, nested divs, hidden inputs, and data attributes. It cannot be authored as standard DA content — it must be stored and rendered as-is.

7.2 Fragment Location

Default Offer fragments are stored at:

```

1 /fragments/default-offers/
2   mk-fp-01      ← Default for Feature Panel Left
3   mk-fp-02      ← Default for Feature Panel Right
4   hb           ← Default for Header Banner
5   lb           ← Default for Lightbox
6   pc           ← Default for Promo Cards
7   fad-s-1      ← Default for Find A Distributor
8
```

7.3 Block Structure

The Default Offer block on the fragment page holds the raw HTML:

Default Offer	
---------------	--

(raw HTML pasted from Marketing AEM)	
--------------------------------------	--

The HTML is preserved as-is — including inline styles, CSS classes, data attributes, and nested div structures. The fragment's `.plain.html` serves this content unchanged.

7.4 Authoring Workflow

1. Marketing team creates/updates offer in Marketing AEM Author (Cloud) — **unchanged**
2. Raw HTML is copied from Marketing AEM Publish — **unchanged**
3. Author opens the corresponding Default Offer fragment page in DA (e.g. `/fragments/default-offers/mk-fp-01`)
4. Pastes the raw HTML into the Default Offer block
5. Saves and publishes the fragment

7.5 Why Raw HTML

The default offer HTML from Marketing AEM contains:

- Inline styles specific to offer styling
- CSS classes consumed by `dm-offer-style.css`
- Nested div structures matching Target-delivered offers
- Hidden inputs and data attributes for tracking
- Dynamic Media image URLs

This HTML must render identically to Target-delivered offers (styled by the same CSS).

8. Placement Code Plugin

8.1 Purpose

A small plugin that provides a dropdown for placement code selection. Prevents authors from mistyping placement codes.

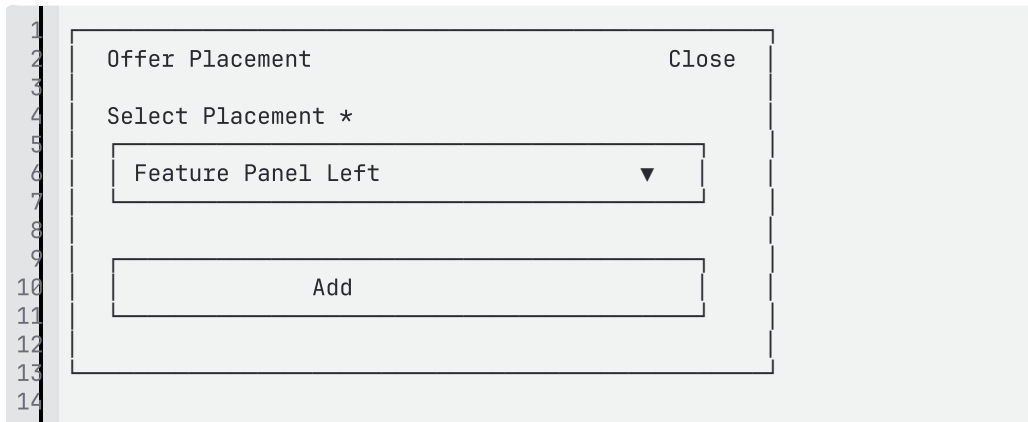
8.2 How It Works

1. Author places cursor in the value cell of the `placement-code` row
2. Opens the **Placement Code plugin** from the DA library panel
3. Plugin shows a dropdown with friendly names:
 - Feature Panel Left
 - Feature Panel Right

- Header Banner
- Lightbox
- Promo Cards
- Find A Distributor

4. Author selects → plugin inserts the placement code value (e.g. `mk_fp_01`) into the cell

8.3 Plugin Dialog



8.4 Scope

The plugin only inserts the placement code value — nothing else. Default offer path is authored manually by the author (adding a link to the fragment page).

9. Multi-Offer Layout

When multiple offers need to be displayed side by side (e.g. Feature Panel Left + Feature Panel Right in a 2-column layout), authors use **Section Metadata** to apply a grid style.

9.1 Example — Two Offers Side by Side

```

1 | Dynamic Offer | |
2 | placement-code | mk_fp_01 |
3 | default-offer | [/fragments/default-offers/mk-fp-01]
  | (/fragments/default-offers/mk-fp-01) |
4
5 | Dynamic Offer | |
6 | placement-code | mk_fp_02 |
7 | default-offer | [/fragments/default-offers/mk-fp-02]
  | (/fragments/default-offers/mk-fp-02) |
8
9 | Section Metadata | |
10 | style | 2-col |
11

```

What renders: Two offer containers side by side. Each receives its own personalized offer from Target independently, or falls back to its own default.

9.2 Section Grid Styles

Section Metadata style	Layout
2-col	2 offers side by side
3-col	3 offers in a row
4-col	4 offers in a row

Each Dynamic Offer block operates independently — its own Target call, its own fallback. Section CSS handles the visual arrangement.

10. Authoring Examples

10.1 Example 1 — Single Offer (Feature Panel)

Dynamic Offer	
placement-code	mk_fp_01
default-offer	/fragments/default-offers/mk-fp-01

What happens at runtime: Block creates container with ID `tf-cq-mk_fp_01`. Target delivers a personalized offer into it. If Target returns nothing, block fetches the default offer fragment and displays it.

10.2 Example 2 — Two Offers Side by Side

```

1 | Dynamic Offer | |
2 | placement-code | mk_fp_01 |
3 | default-offer | [/fragments/default-offers/mk-fp-01]
  | (/fragments/default-offers/mk-fp-01) |
4
5 | Dynamic Offer | |
6 | placement-code | mk_fp_02 |
7 | default-offer | [/fragments/default-offers/mk-fp-02]
  | (/fragments/default-offers/mk-fp-02) |
8
9 | Section Metadata | |
10 | style | 2-col |
11

```

What renders: Two offer panels displayed in a 2-column grid. Each operates independently with its own placement code and fallback.

10.3 Example 3 — Default Offer Fragment (Author View)

The Default Offer fragment page at `/fragments/default-offers/mk-fp-01` :

Default Offer	
<pre><div class="dm-offer- feature-panel"> <div class="dm-offer- text">Summer Sale - 20% Off</div> </div></pre>	

What this is: The raw HTML from Marketing AEM, pasted as-is. Preserved with all inline styles, classes, and structure. Served via `.plain.html` when the block needs fallback content.

11. Open Questions

#	Question	Owner
1	Is Interact Client Context still needed on EDS pages, or is it part of legacy?	TFS to confirm
2	What page-level data is required on EDS pages for Target decisioning beyond country, language, and user type?	TFS to confirm

Header and Footer -Design

Forms Solution

TFS Forms – Current State Overview for Sites + EDS Migration

1. Purpose

This page documents the **current state of the TFS forms implementation** and the **key discovery findings** related to migrating forms to **Sites + Edge Delivery Services (EDS)** *without* AEM Adaptive Forms.

It is intended to be the **foundation document**. Detailed design for each customization (rule editor, dynamic dropdowns etc.) will be captured in separate follow-up pages.

2. Context & Scope

- **Current platform:** AEM 6.4.8.4(Sites), **no Adaptive Forms**, heavy customization on top of core components
 - **Target platform:** AEM Sites + Edge Delivery Services (EDS), **no AEM Forms license**
 - **Scope of this page:**
 - Describe the existing forms landscape and architecture
 - Capture key assumptions for migration
 - Summarize discovery findings that impact EDS design.
-

3. Current Forms Landscape (AEM 6.5)

3.1 Form technology

- TFS **does not use Adaptive Forms**.
- Forms are built using **customized core components** and custom logic.
- There is **significant custom behavior** around:
 - Rule handling (show/hide logic, etc.)
 - Integration with marketing systems via middleware
 - Session ID generation and tracking.

3.2 Form volume and distribution

- Approximately **50k+ forms** are present in the existing site.
- Forms vary across:
 - Classic UI and Touch UI implementations
 - Different business units / use cases

- Different integration patterns (but common middleware).

3.3 Classic vs Touch forms

- There are currently **two types of forms**:
 - **Classic forms**
 - **Touch forms**
 - These differ in **how the `formSessionID` is generated and managed**.
-

4. Submission Architecture & Middleware

4.1 Middleware: `aem-datahub-formprocessor`

- A dedicated middleware, `aem-datahub-formprocessor`, is responsible for **handling form submissions**.
- Main responsibilities:
 - Receive submission payloads from TFS forms
 - Process and **forward data to downstream systems** such as:
 - Eloqua
 - Marketo
 - Other marketing / data platforms as configured
 - Apply any TFS-specific business rules required during submission.

4.2 Submission flow (current)

- On form submit:
 - The **browser calls the middleware directly** (no application gateway in between for this flow).
 - The middleware handles routing and integration logic.
- During discovery, it was confirmed that:
 - This **pattern of direct browser → middleware calls is expected to remain** for EDS as well.
 - No additional application gateway is required for the planned EDS frontend.

4.3 Middleware in the migration

- **Assumption for migration:**

The **middleware remains unchanged** as much as possible.

- EDS-based forms will submit to **the same middleware endpoints**.
- EDS changes focus on:
 - How the form is rendered
 - How data is prepared on the client side

- Ensuring that required fields (e.g., IDs, session IDs) are correctly included.

5. Session Handling (`formSessionID`)

5.1 Current state

- **Classic** and **Touch** forms currently use **different logic** to manage `formSessionID`.
- This divergence introduces complexity in:
 - Tracking submissions
 - Troubleshooting
 - Migration planning.

5.2 Target state for EDS

- For the migration, the goal is to **standardize on a single session handling flow**:
 - `formSessionID` **will be generated on page render**.
 - The generated `formSessionID` will be stored in a **hidden form field**.
 - When the user submits the form, `formSessionID` will be included in the payload to the middleware.
- Benefits:
 - One unified pattern across all EDS forms
 - Simplifies both implementation and debugging
 - Easier alignment with downstream systems.

6. Discovery Findings – Forms Complexity & Customization

6.1 Overall complexity

- TFS forms are a **complex and heavily customized** area of the implementation:
 - Many custom behaviors beyond standard core components
 - Tight coupling with internal data sources and middleware requirements
 - High volume (50k+ forms), increasing migration impact.
- Because **Adaptive Forms are neither used nor licensed**, all advanced behavior must be:
 - Implemented via **custom development**, OR
 - Re-thought for EDS constraints.

6.2 Implication: need for App Builder / backend services

- To replicate or improve the **dynamic dialog behavior** in an EDS + DA world:
 - **App Builder-based services** will be needed , details of Integrations are mentioned on [Fo](#)
[rm Authoring in DA/EDS](#) .

7. Migration Assumptions & Constraints (Forms)

The following are **key assumptions and constraints** identified so far for the migration to Sites + EDS:

- **No Adaptive Forms / AEM Forms:**

- Forms will be built using **custom EDS blocks** and/or document-based forms patterns.
- The **built-in AEM Forms rule editor / features are not available**.

- **Middleware remains in place:**

- `aem-datahub-formprocessor` will continue to handle submissions.
- No major changes expected in middleware contract initially.

- **Browser → middleware direct calls:**

- EDS frontend will continue to submit directly to middleware (no new gateway layer).

- **Unified session handling:**

- Single pattern for `formSessionID` (generated at page render, persisted in a hidden field, submitted with the form).

- **Custom development required for advanced behavior:**

- Show/hide rules, dynamic dropdowns, and external ID lookups must be implemented via:
 - EDS block logic
 - App Builder services
 - DA plugins
 - Edge Worker

Form Authoring in DA/EDS

1. Overview

This document describes the solution design for form authoring in the Thermo Fisher Scientific (TFS) AEM-to-EDS migration. It covers the full authoring lifecycle: from how authors configure and assemble forms in Document Authoring (DA), through the two custom DA Library Panel plugins built to support this, to the App Builder serverless integration layer, and finally to the runtime EDS block JavaScript that renders and processes forms on the published page.

The AEM implementation involved richly configured touch-UI dialog components, three real-time AJAX calls from within the dialog, 11 distinct action types with conditional configuration panels, and a proprietary rule editor for show/hide logic. The EDS architecture replaces all of this with a document-native model: block tables authored in DA, plugin-assisted configuration, and a serverless App Builder integration layer.

Scope of this document:

- Form container block configuration (Plugin 1) — DA Library Panel plugin + 3 App Builder integrations
- Form field block authoring (Plugin 2) — DA Library Panel plugin + 9 EDS block JavaScript files (one per field block type)
- Multi-step form authoring model
- App Builder integration actions (3)
- EDS block JavaScript runtime responsibilities
- Block naming conventions and schemas

2. Current State — AEM Form Authoring

2.1 Component Architecture

Forms in AEM are built using a suite of core form components under the `formcommons/components/form` namespace:

Component	Resource Type
Form Container	<code>formcommons/components/form/container/v1/container</code>
Form Input	<code>formcommons/components/form/input</code>

Form Options	<code>formcommons/components/form/options</code>
Form Button	<code>formcommons/components/form/button</code>
Form Hidden	<code>formcommons/components/form/hidden</code>
Form Upload	<code>formcommons/components/form/upload</code>
Form Textarea	<code>formcommons/components/form/textarea</code>
Form Recaptcha	<code>formcommons/components/form/recaptcha</code>
Form XF Inclusion	<code>formcommons/components/form/xf-inclusion</code>

All configuration is stored as JCR node properties. The Form Container and each field component have their own touch-UI dialog, with tabs for Properties, Constraints, and Accessibility.

2.2 Form Container Dialog

The Form Container dialog is the most complex authoring interface in the form system. Its key characteristics:

- **Action Type multi-select:** Authors choose one or more of 11 action types. Selecting an action type reveals a conditional configuration panel specific to that type.
- **Regional config:** Several action types (Eloqua, GCMS, Email, CORA, FS BIO) support regional override rows, each repeatable via a multiframe component.
- **Conditional field visibility:** A dedicated Rule Editor toolbar button on each field component opens a visual interface for building show/hide rules.

2.3 Action Types

Code	Name	Key Config Fields
1001	Eloqua	<code>eloqua-form-name</code> , <code>eloqua-instance</code> , <code>eloqua-regions</code> (repeatable)
1002	GCMS	<code>gcms-form-id</code> (Fetch button), <code>gcms-regions</code> (repeatable with Fetch)
1003	LSG	Full spec TBD

1004	Non LSG	nonlsg-type (lead / case / quote / bulk quote)
1005	Marketo	marketo-form-id
1006	Email	email-template , email-subject , email-mailto (repeatable), regional config, email-key
1007	CORA	cora-template , cora-subject , cora-mailto (repeatable), regional CORA config, email-key
1009	PDX S3	s3-form-id
1010	PDX ELMS	elms-type
1011	FSBIO	fsbio-template , fsbio-subject , fsbio-mailto (repeatable), regional config, email-key
1013	Genesys DB	Full spec TBD

2.4 AEM Dialog Integrations

Three live AJAX calls are made from within the AEM dialog at authoring time:

Integration 1 — Division Dropdown

- Trigger: Dialog open
- Call: Sling Datasource → Content Fragments at `/content/dam/formcommons/cf/division`
- Response: Populates the Division dropdown with `{text, value}` pairs

Integration 2 — GCMS Form ID Fetch

- Trigger: Author clicks Fetch button in the GCMS action panel
- Call: `GET /apps/lifetech/generateFormId`
- Response: XML containing `<formId>` element; value is written into the `gcms-form-id` field
- Also used per-region for regional GCMS IDs in the repeatable multifield

Integration 3 — Email Attributes Persistence

- Trigger: Dialog save (blocking — `e.preventDefault()` holds submission)
- Call: `POST /bin/servlet/tf/form/postemailattributes.json`
- Payload: Email configuration (template, subject, mailto list, regional config)
- Response: Returns or confirms `emailResourceAllocatorKey`
- Behavior: The dialog does not close and the node is not saved until this call succeeds
- Applies to: Action types 1006 (Email), 1007 (CORA), 1011 (FSBIO)

Note on `emailResourceAllocatorKey` ownership: In the current AEM system, the `uniqueFormKey` was pre-stored in JCR and *sent* to the middleware — the key originated in AEM. The middleware stored the key-to-email-config mapping. For EDS, this ownership must be clarified. Two options are under consideration.

3. Target Architecture — DA/EDS Form Authoring

3.1 Architectural Principles

The EDS form authoring model replaces the AEM dialog system with three collaborating components:

1. **DA Library Panel Plugins** — Browser-based configuration UIs that help authors assemble and configure form blocks.
2. **App Builder Serverless Actions** — Adobe I/O Runtime functions that proxy the three external integrations previously handled by AEM servlets, making them accessible from the DA browser context.
3. **EDS Block JavaScript** — A single orchestrator block script (`tfs-form.js`) that reads all block table configurations at page load and builds the complete rendered form.

Rendering Architecture: Orchestrator Pattern

`tfs-form.js` uses the **Orchestrator pattern** — it is the single active script responsible for all form rendering.

Why the Orchestrator pattern?

- A single `<form>` element must wrap all fields — required for native HTML validation and submission serialization
- Multi-step grouping requires knowing all fields upfront before any HTML is rendered
- Field DOM order must be preserved exactly as authored in the DA document
- Fragment resolution (async fetch of remote DA documents) requires central coordination before any rendering can begin

Rendering flow in brief:

1. `tfs-form.js` reads its own block table config (action types, multistep, etc.)
2. Finds all sibling field block tables in the same section (`tfs-form-input` , `tfs-form-options` , `tfs-form-button` , `tfs-form-step` , `tfs-form-fragment` , etc.)
3. Reads each field block's key-value rows as configuration data
4. Resolves any fragment blocks (async fetch)
5. Builds the complete `<form>` HTML from scratch, including all inputs, selects, radios, buttons, and step panels
6. **Removes the original block tables** from the DOM — they are replaced by the rendered form
7. Outputs one unified `<form>` element

3.2 DA Document Model

A form in an EDS page is represented as a sequence of block tables within a single section of the DA document. The critical constraint is that **all form blocks must reside in the same section** — no section dividers between the `tfs-form` container and its field blocks. The `tfs-form.js` block script discovers field blocks by collecting all next-sibling blocks within the same section.

3.3 Block Naming Conventions

Each block has **two development deliverables**: the DA plugin panel (authoring) and the EDS block (delivery/runtime). Both must be built for the block to be fully functional end to end.

EDS Block structure and rendering model:

`tfs-form` (**Plugin 1 workstream**) — the ONLY active block script.

`blocks/tfs-form/tfs-form.js` contains all form rendering, field orchestration, multi-step logic, fragment resolution, and submission handling.

`blocks/tfs-form/tfs-form.css` contains all form styles.

Field blocks — `tfs-form-input`, `tfs-form-options`, etc. (**Plugin 2 workstream**) — Each field block folder (e.g., `blocks/tfs-form-input/`) must exist in the repository because EDS attempts to load a block JS file for every registered block name it encounters in the page.

The DA Plugin (authoring tool) inserts block tables into the DA document.

`tfs-form.js` reads those tables as configuration and builds the rendered form. The field block JS files exist solely to satisfy the EDS block registration requirement.

Block Name	Equivalent AEM Component	EDS Block Folder	Owned By
<code>tfs-form</code>	Form Container	<code>blocks/tfs-form/</code>	Plugin 1 workstream
<code>tfs-form-input</code>	Form Input, Form Textarea	<code>blocks/tfs-form-input/</code>	Plugin 2 workstream
<code>tfs-form-options</code>	Form Options	<code>blocks/tfs-form-options/</code>	Plugin 2 workstream
<code>tfs-form-button</code>	Form Button	<code>blocks/tfs-form-button/</code>	Plugin 2 workstream
<code>tfs-form-step</code>	Step boundary marker (no AEM equivalent)	<code>blocks/tfs-form-step/</code>	Plugin 2 workstream

tfs-form-fragment	Form XF Inclusion	blocks/tfs-form-fragment/	Plugin 2 workstream
tfs-form-hidden	Form Hidden	blocks/tfs-form-hidden/	Plugin 2 workstream
tfs-form-upload	Form Upload	blocks/tfs-form-upload/	Plugin 2 workstream
tfs-form-recaptcha	Form Recaptcha	blocks/tfs-form-recaptcha/	Plugin 2 workstream

3.4 Two-Plugin Model and Full Development Scope

The form authoring experience is delivered through two separate DA Library Panel plugins. However, the **full development scope** for forms spans plugins, App Builder integrations, and EDS block JavaScript files.

Plugin 1: TFS Form Config — 4 Deliverables

Deliverable	Type	Notes
TFS Form Config DA Plugin	DA Library Panel (UI code)	Authoring UI for the form container block
App Builder Action: GET /api/divisions	Adobe I/O Runtime serverless action	Populates Division dropdowns
App Builder Action: GET /api/gcms/generateFormId	Adobe I/O Runtime serverless action	GCMS Form ID fetch; blocked on TFS API spec
App Builder Action: POST /api/emailAttributes	Adobe I/O Runtime serverless action	Email config persistence; blocked on TFS middleware spec

Plugin 1 also owns the EDS delivery block for the form container — the only active block script in the entire form system:

Deliverable	Type
<code>tfs-form</code> block (<code>blocks/tfs-form/</code> — active JS + CSS)	EDS Block — the orchestrator; contains ALL form rendering logic

Plugin 2: TFS Forms — 9 Deliverables

Deliverable	Type	Notes
TFS Forms DA Plugin (UI)	DA Library Panel plugin	Authoring UI for all field block types
<code>tfs-form-input</code> block (<code>blocks/tfs-form-input/</code>)	EDS Block	Input / textarea; schema read by orchestrator
<code>tfs-form-options</code> block (<code>blocks/tfs-form-options/</code>)	EDS Block	Dropdown / radio / checkbox / multiselect
<code>tfs-form-button</code> block (<code>blocks/tfs-form-button/</code>)	EDS Block	Submit / reset / button
<code>tfs-form-step</code> block (<code>blocks/tfs-form-step/</code>)	EDS Block	Step boundary marker; used by orchestrator to partition panels
<code>tfs-form-fragment</code> block (<code>blocks/tfs-form-fragment/</code>)	EDS Block	Fragment path is fetched and inlined by the orchestrator
<code>tfs-form-hidden</code> block (<code>blocks/tfs-form-hidden/</code>)	EDS Block	Hidden field injection

<code>tfs-form-upload block (blocks/tfs- form-upload/)</code>	EDS Block	File upload field
<code>tfs-form- recaptcha block (blocks/tfs-form- recaptcha/)</code>	EDS Block	reCAPTCHA widget

This separation mirrors the logical separation between the form container (which owns action-type routing and system-level config) and the field blocks (which own field-level schema). Both plugins are loaded in the DA Library panel and accessible to authors during page editing. The App Builder actions and EDS block JS files are additional development work that must be scoped alongside the plugin UI work.

4. Current vs. Future State Comparison

Dimension	AEM (Current)	DA/EDS (Target)
Authoring surface	Touch-UI dialog, per-component	DA Library Panel plugin, block table
Form container config	Form Container dialog (complex, tabbed)	Plugin 1: TFS Form Config
Field configuration	Per-field touch-UI dialogs (Properties / Constraints / Accessibility tabs)	Plugin 2: TFS Forms (Properties / Constraints / About tabs)
Action type config	Conditional panels in Form Container dialog	Conditional panels in Plugin 1, values written to block table
Conditional field visibility	AEM Rule Editor (custom, per-field toolbar button)	Custom Rule Editor
Division dropdown source	Sling Datasource → JCR Content	App Builder → AEM GraphQL → CF data

	Fragments	
GCMS ID fetch	GET /apps/lifetech/generateFormId (AEM servlet)	App Builder GET /api/gcms/generateFormId
Email key persistence	POST /bin/servlet/tf/form/postemailattributes.json (blocking on dialog save)	App Builder POST /api/emailAttributes (blocking on "Add to Page")
Config storage	JCR node properties	Block table key-value rows in DA document
Multi-step authoring	No native concept; custom JS	tfs-form-step blocks as section markers, multistep: true on container
Fragment inclusion	Form XF Inclusion component	tfs-form-fragment block with path reference
Runtime rendering	Server-side HTL + Sling Models	Client-side tfs-form.js
Form submission	AEM action handler	TFS Backend endpoint (TFS Middleware)
Regional config encoding	Repeatable Granite multifield	region:subregion:value pipe-delimited string in block table
Integration execution context	AEM server-side (servlets, datasources)	Adobe I/O Runtime (App Builder serverless)

5. DA Plugin 1: TFS Form Config

5.1 Purpose and Placement

Plugin 1 (TFS Form Config) is a DA Library Panel plugin responsible for configuring the `tfs-form` container block. It provides a structured UI so authors do not need to manually author the block table for the container, which is the most complex block in the form system.

The plugin is accessed from the DA Library panel during page editing. Authors open it, configure the form container, and click "Add to Page" to insert the `tfs-form` block table at the current cursor position.

5.2 UI Structure

The plugin UI is organized into two tabs:

Tab 1: Properties

This tab contains all form-level configuration:

Field	Type	Required	Notes
Enable Multi-step Form	Checkbox	No	When checked, renders multi-step progress indicator at runtime; <code>multistep: true</code> written to block
Action Type	Multi-select dropdown	Yes	11 types; selecting a type reveals its conditional config section
MQO Form	Dropdown	Yes	
Division	Dropdown	Yes	Populated via App Builder → AEM GraphQL on plugin load
<i>[Per-action-type config sections]</i>	Conditional	Varies	See Section 5.3

5.3 Conditional Action-Type Configuration Panels

When the author selects one or more action types from the multi-select dropdown, the corresponding configuration section for each selected type is rendered below the main fields. Sections for unselected types are hidden.

Implementation approach: each action type has a corresponding HTML `<template>` element. On selection, the template is cloned and appended to the config area. On deselection, the cloned panel is removed and its values are discarded.

Only fields for currently selected action types are written to the block table on insertion or save.

Action type config panels:

Action Type	Config Fields in Panel
1001 — Eloqua	<code>eloqua-form-name</code> (text), <code>eloqua-instance</code> (text), <code>eloqua-regions</code> (repeatable rows: region:subregion:value)
1002 — GCMS	<code>gcms-form-id</code> (text + Fetch button), <code>gcms-regions</code> (repeatable rows, each with Fetch button)
1003 — LSG	TBD — full config spec not yet provided by TFS
1004 — Non LSG	<code>nonlsg-type</code> (dropdown: lead / case / quote / bulk quote)
1005 — Marketo	<code>marketo-form-id</code> (text)
1006 — Email	<code>email-template</code> (text), <code>email-subject</code> (text), <code>email-mailto</code> (repeatable email addresses), regional email config (repeatable), <code>email-key</code> (read-only, populated by middleware call)
1007 — CORA	<code>cora-template</code> (text), <code>cora-subject</code> (text), <code>cora-mailto</code> (repeatable), regional CORA config (repeatable), <code>email-key</code> (read-only)
1009 — PDX S3	<code>s3-form-id</code> (text)

1010 — PDX ELMS	<code>elms-type</code> (text)
1011 — FSBIO	<code>fsbio-template</code> (text), <code>fsbio-subject</code> (text), <code>fsbio-mailto</code> (repeatable), regional config (repeatable), <code>email-key</code> (read-only)
1013 — Genesys DB	TBD — full config spec not yet provided by TFS

5.4 Regional Config Encoding

Action types 1001, 1002, 1006, 1007, and 1011 support regional configuration overrides. In the AEM system, these were Granite multifield components storing individual JCR child nodes. In the block table, they are encoded as a single pipe-delimited string per row:

```
1 | region:subregion:value|region2:subregion2:value2
2 |
```

For example, Eloqua regions:

```
1 | north-america:us:test Region1|europe:uk:test Region2
2 |
```

The plugin renders these as repeatable UI rows (add/remove buttons), serializes them to the encoded format when writing to the block, and deserializes them when loading an existing block for editing.

5.5 GCMS Form ID Fetch

When the author clicks the Fetch button in the GCMS config panel:

1. Plugin calls App Builder `GET /api/gcms/generateFormId`
2. Response `{formId: "12345"}` is written into the `gcms-form-id` field
3. Each regional GCMS row also has its own Fetch button, which calls the same endpoint and populates that row's value

The field is editable after fetching, allowing manual override if needed.

5.6 Email Middleware — Blocking Insertion

For action types 1006, 1007, and 1011, the following behavior applies when the author clicks "Add to Page":

1. Plugin collects the email configuration (template, subject, mailto list, regional config)
2. Plugin makes a **blocking** `POST` call to App Builder `POST /api/emailAttributes`
3. While the call is in flight, the insert operation is blocked (spinner shown, button disabled)

4. On success: `email-key` is written into the block table.
5. On failure: an error message is displayed; the block is **not** inserted; the author must retry

This mirrors the AEM behavior where `e.preventDefault()` held the dialog save until the servlet call completed. The key difference is that in EDS this call blocks "Add to Page" rather than a dialog save event.

 **Quick Question-1: emailResourceAllocatorKey — Who generates it? (Decision Required)**

The `emailResourceAllocatorKey` (`email-key`) stored in the block table is the unique identifier linking the form to its email configuration in the middleware. **How this key originates is an open decision with two options:**

	Option A — Plugin generates key	Option B — Middleware generates key
AEM equivalent?	Yes — matches AEM pattern where <code>uniqueFormKey</code> was pre-stored in JCR and sent to middleware	No — AEM always sent an existing key, never received a new one
Plugin behavior	Plugin generates a UUID, includes it in the POST payload , and writes it to the block table	Plugin sends only email config (no key). Middleware returns the key in the response. Plugin writes the returned key to the block table
App Builder POST payload	<code>{actionType, emailResourceAllocatorKey: "uuid", emailConfig: {...}}</code>	<code>{actionType, emailConfig: {...}}</code>
App Builder response	Acknowledgement only (e.g., <code>{status: "ok"}</code>)	Returns new key: <code>{emailResourceAllocatorKey: "abc-123-xyz"}</code>

Plugin effort impact	Plugin must generate UUID before calling App Builder	Plugin is a pure pass-through — no key generation logic
-----------------------------	--	---

➔ **Decision owner: TFS + Adobe joint. Must be resolved before App Builder Action 3 can be built.**

5.7 Edit Mode

When an author selects an existing `tfs-form` block in the DA document and clicks "Edit Selected Form Config":

1. Plugin reads the selected block's table HTML
2. Extracts all key-value pairs from the table rows
3. Determines which action types are active from the `action-types` value
4. Renders and pre-populates all active action type panels with the stored values
5. Deserializes regional config strings back to repeatable UI rows
6. Author modifies and clicks "Update" — block table is replaced in-place

5.8 "Add to Page" Behavior Summary

1. Validate required fields (Action Type selected, MQO Form, Division)
2. If email action type selected: execute blocking middleware call (Section 5.6)
3. If GCMS selected and no Form ID fetched: warn author (soft warning, not blocking)
4. Serialize all active config to key-value pairs
5. Build block table HTML (`tfs-form` block)
6. Insert at cursor position in DA document
7. Plugin collapses / returns to entry state

6. DA Plugin 2: TFS Forms (Field Blocks)

6.1 Purpose and Placement

Plugin 2 (TFS Forms) is a DA Library Panel plugin that handles all form field block types. Authors use this plugin to add individual fields to the form — inputs, dropdowns, buttons, steps, and more. It is accessed from the same DA Library panel as Plugin 1.

6.2 Entry Screen — Field Type Grid

When the plugin is opened, it displays a grid of available field types. Authors click a type to enter its configuration panel:

Field Type	Block Produced
Input	<code>tfs-form-input</code>
Options	<code>tfs-form-options</code>
Button	<code>tfs-form-button</code>
Step	<code>tfs-form-step</code>
Fragment	<code>tfs-form-fragment</code>
Hidden	<code>tfs-form-hidden</code>
Upload	<code>tfs-form-upload</code>
Recaptcha	<code>tfs-form-recaptcha</code>

6.3 Form Input Field

Tab: Properties

Field	Type	Required	Notes
Label	Text	Yes	Displayed label for the field; rendered as <code><label></code> element
Name	Text	Yes	HTML <code>name</code> attribute; used in form submission payload
Type	Dropdown	Yes	text / email / telephone / number / date / url / textarea

Placeholder	Text	No	Placeholder text shown in empty input
Value (Default)	Text	No	Pre-populated default value
Query Param	Text	No	URL query parameter to pre-populate field on load
Hint Text	Text	No	Helper text shown below the field; referenced via <code>aria-describedby</code> . Use to explain format expectations (e.g., "Enter in format 123-456-7890")

Tab: Constraints

Field	Type	Notes
Required	Checkbox	Field must have a value on submit
Required Message	Text	Validation message shown when required and empty
Min Length	Number	Minimum character count
Max Length	Number	Maximum character count

Constraint Message	Text	Shown when min/max length violated
Read Only	Checkbox	Field rendered as non-editable

Tab: Accessibility

Field	Type	Notes
Aria Label	Text	Overrides the visible label for screen readers. Use when the visual label is insufficient or absent (e.g., icon-only fields, visually grouped fields). If left blank, the visible <code>Label</code> value is used automatically

6.4 Form Options Field

Tab: Properties

Field	Type	Required	Notes
Title	Text	Yes	Displayed label for the field group
Name	Text	Yes	HTML <code>name</code> attribute
Type	Dropdown	Yes	Drop-down / Radio / Checkbox / Multi-select

Source	Radio	Yes	Local (options authored in plugin) / External (options from data source — see OQ for external source spec)
Options	Textarea	Conditional (if Local)	Pipe-delimited, format <code>DISPLAY TEXT,VALUE DISPLAY TEXT,VALUE</code>
Hint Text	Text	No	Helper text shown below the field group; auto-wired to <code>aria-describedby</code> on the <code><fieldset></code> or <code><select></code>

Tab: Constraints

Field	Type	Notes
Required	Checkbox	At least one option must be selected on submit
Required Message	Text	Validation message shown when required and no selection made

Tab: Accessibility

Field	Type	Notes
-------	------	-------

Aria Label	Text	Overrides the visible Title for screen readers. Particularly important for radio/checkbox groups where the <code><fieldset></code> <code><legend></code> must be meaningful. If blank, the visible Title is used
Aria Describedby	Text	Manual ID reference to an external description element. Leave blank if using Hint Text

6.5 Form Step

Used only in multi-step forms. A step block acts as a section marker — the runtime groups all field blocks between consecutive step blocks into a single step panel.

Field	Type	Required
Step Title	Text	Yes
Step Number	Number	Yes

6.6 Form Button

Field	Type	Notes
Label	Text	Button display text; also used as <code>aria-label</code> if no override is set
Type	Dropdown	<code>submit</code> / <code>reset</code> / <code>button</code>
Aria Label	Text	Optional. Overrides the visible label for screen

		readers. Use when the button text alone is ambiguous (e.g., "Next" in a multi-step form — override to "Next: Step 2 — Contact Info")
--	--	--

6.7 Form Fragment

Field	Type	Required	Notes
Path	Text	Yes	DA document path to the fragment, e.g., /fragments/form-fragments/fragment1

The runtime fetches the fragment content and resolves its field blocks into the step grouping at render time.

6.8 Edit Mode

When an author selects an existing field block in the DA document and clicks "Edit Selected Block":

1. Plugin detects the block type from the table's header row
2. Loads the corresponding config panel
3. Parses the block table rows into field values
4. Pre-populates all form fields
5. Author modifies and clicks "Update" — block table is replaced in-place

6.9 "Add to Page" Behavior Summary

1. Validate required fields for the selected block type
2. Serialize all configured values to key-value pairs
3. Build block table HTML for the appropriate block type
4. Insert at cursor position in DA document

7. Multi-Step Form Authoring

7.1 Enabling Multi-Step

An author enables multi-step mode by checking "Enable Multi-step Form" in Plugin 1. This writes `multistep: true` to the `tfs-form` block. Without this flag, all step blocks are ignored by the runtime and fields render as a single-page form.

7.2 Authoring Model

In multi-step mode, the author interleaves `tfs-form-step` blocks among the field blocks. Each step block marks the beginning of a new step panel. All field blocks that follow a step marker (and precede the next step marker or the end of the section) belong to that step.

Example DA document structure for a 4-step form:

```
1 tfs-form          (multistep: true, action-types, ...)
2 tfs-form-step     (title: "Step 1: Let's get started", step: 1)
3 tfs-form-input    (First Name)
4 tfs-form-input    (Last Name)
5 tfs-form-step     (title: "Step 2: Contact Info", step: 2)
6 tfs-form-input    (Email)
7 tfs-form-options  (Country)
8 tfs-form-step     (title: "Step 3: Your Request", step: 3)
9 tfs-form-fragment (path: /fragments/form-fragments/request-fields)
10 tfs-form-step    (title: "Step 4: Review & Submit", step: 4)
11 tfs-form-input   (Subject)
12 tfs-form-button  (Submit)
13
```

7.3 Rendered Multi-Step Form

At runtime, `tfs-form.js` renders multi-step forms with the following behavior:

- **Step progress indicator:** Numbered circles connected by a line, styled in TFS red theme. The active step circle is filled; completed steps show a checkmark; future steps are greyed out.
- **Field panel visibility:** Only the fields belonging to the active step are visible. Other step panels are hidden.
- **Navigation buttons:**
 - Steps 1 through N-1: "Next" button (validates current step before advancing), "Previous" button (navigates back, no validation)
 - Last step: "Submit" + "Previous"
- **Step validation:** Clicking "Next" triggers validation for all required fields and constraint checks within the current step panel. Navigation is blocked until the step is valid.

- **Fragment resolution:** If a step contains a `tfs-form-fragment` block, the fragment content is fetched and its fields are resolved into that step's panel before rendering.

7.4 Authoring Constraints

- All form blocks (container + fields + steps) must be in the **same section** in the DA document.
- Step numbers must be sequential starting from 1; the `step` field on `tfs-form-step` is used to establish the correct order (the runtime does not rely solely on DOM order, though both should match).
- At least one `tfs-form-button` with `type: submit` must be present in the last step.

8. Block Schema — Complete Reference

This section documents the data schema for every form block. Each block is represented as a two-column table in the DA document: the first cell in each row is the key, the second is the value.

 **Note:** The dialog field names and configurations listed in this document are for illustration only. During implementation, refer to the existing AEM touch-UI dialogs (and their stored JCR properties) as the source of truth for the exact fields and mappings.

8.1 tfs-form (Container)

Key	Example Value	Notes
<code>action-types</code>	<code>1005,1004,1001</code>	Comma-separated list of active action type codes
<code>multistep</code>	<code>true</code>	Only present if multi-step enabled
<code>mgo-form</code>	<code>form-a</code>	MQO Form selection
<code>division</code>	<code>div-2</code>	Division selection
<code>marketo-form-id</code>	<code>111</code>	Present only if 1005 selected
<code>nonlsg-type</code>	<code>quote</code>	Present only if 1004 selected

		(lead/case/quote/bulk quote)
eloqua-form-name	eloqua Test	Present only if 1001 selected
eloqua-instance	instance-2	Present only if 1001 selected
eloqua-regions	north-america:us:testVal1	Present only if 1001 selected; pipe-delimited for multiple
gcms-form-id	12345	Present only if 1002 selected
gcms-regions	europa:uk:67890	Present only if 1002 selected; pipe-delimited for multiple
s3-form-id	s3-abc	Present only if 1009 selected
elms-type	elms-type-a	Present only if 1010 selected
email-template	template-a	Present only if 1006 selected
email-subject	Thank You	Present only if 1006 selected
email-mailto	a@b.com c@d.com	Present only if 1006 selected; pipe-delimited
cora-template	cora-template-1	Present only if 1007 selected
cora-subject	CORA Request	Present only if 1007 selected
cora-mailto	x@y.com	Present only if 1007 selected; pipe-

		delimited
fsbio-template	fsbio-t1	Present only if 1011 selected
fsbio-subject	FSBIO Subject	Present only if 1011 selected
fsbio-mailto	z@w.com	Present only if 1011 selected; pipe-delimited
email-key	abc-123-xyz	Present for 1006/1007/1011; populated by middleware call

8.2 tfs-form-input

Key	Example Value	Notes
label	Enter Email	Displayed label; rendered as <code><label></code> linked to input via <code>for / id</code>
type	email	text / email / telephone / number / date / url / textarea
name	testTxt	HTML name attribute; must be unique within the form
required	true	Omitted if false
required-message	This field is required	Optional; shown when required + empty
constraint-message	Input Constraint Message 1	Shown when min/max length violated
minlength	13	Optional

<code>maxlength</code>	<code>100</code>	Optional
<code>placeholder</code>	<code>Enter your email</code>	Optional
<code>value</code>	<code>default@example.com</code>	Optional default value
<code>query-param</code>	<code>param</code>	Optional URL query param to pre-populate
<code>readonly</code>	<code>true</code>	Optional; renders field as read-only

8.3 tfs-form-options

Key	Example Value	Notes
<code>label</code>	<code>Country</code>	Displayed label; rendered as <code><legend></code> for radio/checkbox groups, <code><label></code> for select
<code>type</code>	<code>select</code>	select / radio / checkbox / multiselect
<code>name</code>	<code>country</code>	HTML name attribute; must be unique within the form
<code>source</code>	<code>local</code>	local / external
<code>options</code>	<code>United States,US India,IN UK,GB</code>	Pipe-delimited; each option is <code>DISPLAY TEXT,VALUE</code>
<code>required</code>	<code>true</code>	Omitted if false
<code>required-message</code>	<code>Field is Required</code>	Optional
<code>aria-label</code>	<code>Country of residence</code>	Optional; overrides visible label for screen

		readers
--	--	---------

8.4 tfs-form-step

Key	Example Value	Notes
title	Step 1: Let's get started	Displayed in progress indicator
step	1	Step sequence number; must be sequential from 1

8.5 tfs-form-button

Key	Example Value	Notes
label	Submit	Button display text
type	submit	submit / reset / button
aria-label	Submit contact form	Optional; overrides visible label for screen readers. Use when label alone is ambiguous (e.g., "Next" in multi-step forms — override to "Next: Step 2 — Contact Info")

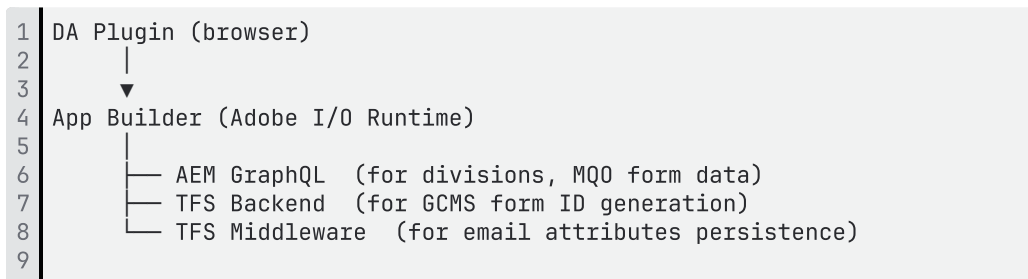
8.6 tfs-form-fragment

Key	Example Value	Notes
path	/fragments/form-fragments/fragment1	DA document path to the fragment

9. App Builder Integrations

Three AEM servlet-based integrations are replaced by Adobe I/O Runtime serverless actions. All three are authored through the DA plugins running in the browser; they cannot use AEM-side Sling infrastructure.

9.1 Architecture Overview



All App Builder actions must be accessible from the `da.live` domain. CORS configuration is required

9.2 Action 1 — GET `/api/divisions`

Attribute	Value
Method	GET
Trigger	Plugin 1 loads (automatic)
Caller	Plugin 1 (Division dropdown), Plugin 1 (MQO Form dropdown — same pattern)
App Builder →	AEM GraphQL → Content Fragments at <code>/content/dam/formcommons/cf/division</code>
Response	<code>[{text: "Life Sciences", value: "lsg"}, ...]</code>
Caching	1 hour (App Builder response cache)

Behavior: On plugin open, the division form dropdowns display a loading state while this call completes. On success, they are populated.

9.3 Action 2 — GET `/api/gcms/generateFormId`

Attribute	Value
Method	GET

Trigger	Author clicks "Fetch" button in GCMS config panel
App Builder →	TFS Backend DB (GCMS ID generation service)
Response	<code>{formId: "12345"}</code>
Caching	None (each call generates a new, unique ID)
Auth method	TBD — see Open Question 2

Behavior: Each click generates a new form ID and overwrites any previously fetched value in the input field. Also used for per-region GCMS ID rows — each row has its own Fetch button calling the same endpoint. The author may manually edit the value after fetching.

Open Question: Exact API spec, authentication method, and request/response format must be confirmed with TFS Backend team at the time of Implementation.

9.4 Action 3 — POST /api/emailAttributes

Attribute	Value
Method	POST
Trigger	Author clicks "Add to Page" with action type 1006, 1007, or 1011 selected
App Builder →	TFS Email Middleware (<code>/bin/servlet/tf/form/postemailattributes.json</code> equivalent)
Payload	
Response	
Blocking	Yes — block insertion is held until this call returns

Behavior:

1. Plugin collects email config (template, subject, mailto list, regional config)
2. Plugin POSTs to App Builder (payload structure depends on OQ-1 decision)
3. App Builder forwards to TFS middleware

4. On success: `email-key` is written into the `tfs-form` block table (value source depends on OQ-1)
5. On failure: error displayed to author; block is **not** inserted; author must retry

⚠ **Payload and response contract depend on (`emailResourceAllocatorKey` ownership)**

	Option A — Plugin generates key	Option B — Middleware generates key
POST payload	<code>{actionType, emailResourceAllocatorKey: "uuid", emailConfig: {...}}</code>	<code>{actionType, emailConfig: {...}}</code>
Response	<code>{status: "ok"}</code>	<code>{emailResourceAllocatorKey: "abc-123-xyz"}</code>
email-key source	UUID generated by plugin before the call	Value returned in the response

➔ **This action cannot be fully specced or built until OQ-1 is resolved.**

9.5 Network Accessibility

All three App Builder actions must be reachable from the `da.live` origin (plugin runs in browser against DA). TFS middleware must be reachable from Adobe I/O Runtime. Both require network allowlisting and CORS configuration to be confirmed .

10. Runtime — `tfs-form.js` (Orchestrator)

10.1 Entry Point and Rendering Model

`tfs-form.js` is the **sole active block script** in the entire form system. It is invoked by EDS when the browser decorates the `tfs-form` block at page load.

10.2 Core Responsibilities

`tfs-form.js` performs the following operations in sequence:

Step 1 — Parse container block Read all key-value rows from the `tfs-form` block table. Extract action types, multistep flag, division, MQO form, IDs, and all action-type-specific fields.

Step 2 — Collect field blocks Traverse all next-sibling DOM elements within the same section. Collect all elements that are form block tables (`tfs-form-input` , `tfs-form-options` , `tfs-form-button` , `tfs-form-step` , `tfs-form-fragment` , `tfs-form-hidden` , `tfs-form-upload` , `tfs-form-recaptcha`). Stop at a section divider.

Step 3 — Detect multi-step mode If `multistep: true` is set, partition the collected field blocks into step panels. Each `tfs-form-step` block starts a new panel. Field blocks before the first step block belong to a preamble (typically none in practice).

Step 4 — Resolve fragments For each `tfs-form-fragment` block encountered, fetch the referenced DA document path. Parse the returned HTML to extract the field blocks within the fragment. Inline those blocks into the current step panel at the position of the fragment block. Fragment fields participate in step grouping and constraint validation as first-class members.

Step 5 — Inject system hidden fields In addition to the authored field blocks, the runtime injects hidden fields that are required by the submission endpoint. These are not authored — they are generated programmatically:

Hidden Field	Source
<code>formSessionId</code>	Generated UUID at runtime
<code>actionType</code>	From <code>action-types</code> value in <code>tfs-form</code> block
<code>form:webUrl</code>	<code>window.location.href</code>
<code>form:region</code>	From page metadata / locale
<code>form:country</code>	From page metadata / locale
<code>form:language</code>	From page metadata / locale
<code>form:division</code>	From <code>division</code> value in <code>tfs-form</code> block
<code>elqSiteID</code>	From block (if Eloqua action type)
<code>elqFormName</code>	From <code>eloqua-form-name</code> in block
<code>gcmsFormId</code>	From <code>gcms-form-id</code> in block

emailResourceAllocat orKey	From email-key in block
marketo-form-id	From block (if Marketo action type)

Step 6 — Build and render the form Construct the `<form>` DOM element. For each collected field block (reading its key-value table as data), render the corresponding HTML element:

- **tfs-form-input:** `<label>` + `<input>` (or `<textarea>` if `type: textarea`) + optional hint `<span>`. Apply `name`, `type`, `placeholder`, `value`, `required`, `minlength`, `maxlength`, `readonly`. Wire accessibility attributes: `id` on input linked via `for` on label; if `hint-text` present, generate a unique ID for the hint span and set `aria-describedby` on the input; if `aria-label` present, set it on the input; if `aria-describedby` manually authored, use that value instead.
- **tfs-form-options (select):** `<label>` + `<select>` + `<option>` elements. Apply `name`, `required`. Wire `aria-describedby` to hint span if present.
- **tfs-form-options (radio/checkbox):** `<fieldset>` + `<legend>` + individual `<input type="radio|checkbox">` + `<label>` pairs. Wire `aria-describedby` on fieldset.
- **tfs-form-button:** `<button type="submit|reset|button">`. Apply `aria-label` if authored.
- **tfs-form-hidden:** `<input type="hidden">` with `name` and `value`.
- **tfs-form-upload:** `<label>` + `<input type="file">` with `accept`. Wire hint text via `aria-describedby`.
- **tfs-form-recaptcha:** reCAPTCHA widget container with configured `site-key` and `size`.

After rendering all field HTML, **remove the original block table elements** from the DOM. The rendered `<form>` replaces them. Attach client-side validation event listeners (blur/change for inline error display).

Step 7 — Multi-step UI (if applicable) Render the step progress indicator. Set step 1 as active. Hide all other step panels. Attach Next/Previous navigation handlers. Wire step-level validation to the Next button.

Step 8 — Submit handling On submit: validate all required fields and constraints across the entire form. On validation pass: serialize form data (including injected hidden fields) and POST to the configured submission endpoint.

10.3 Fragment Handling Detail

Fragments in TFS forms are DA documents that contain form field blocks without a `tfs-form` container. When the runtime encounters a `tfs-form-fragment` block:

1. Fetches the path as a DA plain HTML document
2. Parses the returned HTML for block tables matching `tfs-form-input`, `tfs-form-options`, etc.
3. Inserts those field blocks inline at the fragment's position in the step panel
4. The fragment fields are included in constraint validation and form submission as first-class members

Constraint: Fragment fields must follow the same block schema as directly authored fields.

10.4 Constraint Validation

Field-level constraint validation is performed client-side using the attributes authored in the block:

Block Attribute	Validation Rule
<code>required: true</code>	Field must not be empty on advance/submit
<code>minlength</code>	Value length must be $\geq$ minlength
<code>maxlength</code>	Value length must be $\leq$ maxlength
<code>type: email</code>	Value must match email pattern
<code>type: telephone</code>	Value must match telephone pattern
<code>type: number</code>	Value must be numeric
<code>constraint-message</code>	Shown for minlength/maxlength violations
<code>required-message</code>	Shown for required violations

11. Ownership and Dependencies

11.1 Component Ownership Matrix

Plugin 1 Workstream — Adobe Development Team

Deliverable	Type	Dependencies
-------------	------	--------------

TFS Form Config DA Plugin (UI)	DA Library Panel plugin	
App Builder Action: GET /api/divisions	Adobe I/O Runtime serverless action	
App Builder Action: GET /api/gcms/generateFormId	Adobe I/O Runtime serverless action	TFS GCMS API spec
App Builder Action: POST /api/emailAttributes	Adobe I/O Runtime serverless action	TFS middleware spec
tfs-form block (blocks/tfs-form/ — JS + CSS)	EDS Block — form container runtime	

Plugin 2 Workstream — Adobe Development Team

Deliverable	Type
TFS Forms DA Plugin (UI)	DA Library Panel plugin
tfs-form-input block (blocks/tfs-form-input/)	EDS Block
tfs-form-options block (blocks/tfs-form-options/)	EDS Block
tfs-form-button block (blocks/tfs-form-button/)	EDS Block
tfs-form-step block (blocks/tfs-form-step/)	EDS Block
tfs-form-fragment block (blocks/tfs-form-fragment/)	EDS Block
tfs-form-hidden block (blocks/tfs-form-hidden/)	EDS Block

tfs-form-upload block (blocks/tfs-form-upload/)	EDS Block
tfs-form-recaptcha block (blocks/tfs-form-recaptcha/)	EDS Block

External Dependencies — TFS / Other Teams

Component	Owner	Status
GCMS ID Generation Backend (API spec + endpoint)	TFS Backend Team	External dependency
Email Middleware (postemailattributes equivalent)	TFS Backend Team	External dependency

11.2 Integration Dependencies

1	EDS Development (Plugin 1 workstream) depends on TFS Backend for:
2	- Email middleware API contract (endpoint, auth, payload, response)
3	- GCMS Form ID generation API contract (endpoint, auth, request, response)
4	
5	EDS Development (Plugin 1 workstream) depends on AEM/Content Team for:
6	- AEM GraphQL endpoint stability (divisions, MQO form data)
7	- Content Fragment schema at /content/dam/formcommons/cf/division
8	
9	TFS Backend depends on EDS Development + TFS joint decision for:
10	- emailResourceAllocatorKey ownership – must be resolved before payload is agreed
11	- Final email config payload structure (follows from OQ-1 resolution)
12	

12. Complexity Areas

12.1 Dynamic Action-Type Rendering

The 11 action types, combined with multi-select capability, mean that the Plugin 1 config area can contain up to 11 simultaneous conditional panels, each with their own fields, repeatable rows, and Fetch buttons. The template-cloning approach (HTML `<template>` elements per action type) keeps the implementation manageable but requires careful state management to ensure that:

- Only active panels are serialized to the block
- Edit mode correctly reconstructs the exact set of active panels
- Adding and removing action types in a single editing session does not leave stale values

12.2 Regional Config Encoding

Five action types support regional overrides with a `region:subregion:value` pipe-delimited encoding. This encoding must be:

- Serialized correctly when writing to the block (plugin → block table)
- Deserialized correctly when loading for edit (block table → plugin UI)
- Handled correctly at runtime (block table → hidden fields injected into form)

The repeatable UI rows in the plugin must handle add, remove, and reorder operations without corrupting the encoded string.

12.3 Edit Mode Complexity

Edit mode requires full bidirectional mapping between block table HTML and plugin UI state. This is the most fragile aspect of the plugin implementation. Key risks:

- New fields added in future versions must not break parsing of old block tables
- Regional config deserialization must tolerate unexpected delimiters or malformed values
- Action types not yet fully specced (1003, 1013) must be handled gracefully when encountered in existing blocks

12.4 Fragment Fields in Steps

Fragments introduce indirection: the runtime must fetch remote content before it can fully render the step panels. This has implications for:

- Page load performance (additional HTTP requests before render)
- Step validation (fragment fields must participate in step-level constraint checks — fields inside fragments are treated identically to directly authored fields)
- Field naming uniqueness — fragment field `name` attributes must not collide with names of directly authored fields in the same form

The recommended approach is to resolve all fragments before rendering any step panels, using `Promise.all` to parallelize fragment fetches. A fetch timeout (suggested: 5 seconds) must be defined, with fallback to an empty panel and an error message on timeout.

13. Open Questions

Open Questions

Question	Owner	Impact
emailResourceAllocationKey ownership <i>(decision required before Plugin can be built):</i> Option A — Plugin generates UUID, includes it in POST payload, middleware stores it . Option B — Plugin sends only email config, middleware generates and returns the key. Option A: plugin has key-generation logic; App Builder payload includes the key. Option B: plugin is a pass-through; App Builder response contains the key.	TFS + Adobe	Changes App Builder Action 3 payload/response contract AND Plugin 1 logic
GCMS ID generation API: What is the exact endpoint URL, authentication method, and request/response format for the GCMS ID generation service?	TFS Backend Team	
Email middleware API contract: What is the exact endpoint URL, authentication method,	TFS Backend Team	

request payload schema, and response schema for the email attributes persistence service?		
Middleware network accessibility: Is the TFS email middleware and GCMS backend reachable from Adobe I/O Runtime? What allowlisting or VPN/private networking is required?	TFS + Adobe Infrastructure	Blocks App Builder actions from being deployable

Forms — Rule Editor, Validations

1. Overview

The Rule Editor provides authors the ability to define conditional visibility rules on form fields. A rule determines whether a form field is shown or hidden based on the value of another field at runtime.

The EDS Rule Editor retains the same authoring mental model and runtime behaviour as the AEM Rule Editor while eliminating server-side dependencies, encryption, and per-field rule storage. All rules are authored through a dedicated DA Plugin UI and stored in a single `tfs-form-rules` block in the DA document.

2. Current State — AEM Rule Editor

2.1 Authoring

The Rule Editor in AEM is registered as a toolbar action that appears on every form component. A global clientlib registers it dynamically for all components whose `resourceType` starts with `formcommons/components/form`, `tfsite/components/form`.

Rule types supported: Show / Hide only.

Conditions supported: equal, notequal, lessthan, lessthanequalto, greaterthan, greaterthanequalto, startwith, endwith, contains.

Logic operators: all (AND) / any (OR).

2.2 Field Discovery

When the Rule Editor dialog opens, it populates the trigger field dropdown via a server-side Sling datasource servlet (`GetFormFieldsServlet`) that reads the JCR directly.

Discovery scope:

- Only direct children of the immediate parent form container
- Fields inside nested panel containers are not visible from the parent level
- Fields inside Experience Fragment inclusions are not discoverable
- Multiple form containers on a page are scoped independently

2.3 Storage and Runtime

Rules are stored as `rules` child nodes on each individual field component node in JCR. At render time, `ContainerModelImpl` aggregates rules from all child fields into a single JSON array and encrypts it using AES-128-ECB. The encrypted string is placed on the `<form>` element as a `data-showhide` attribute. At runtime, `showhide.js` fetches a decryption key from a server endpoint, decrypts the rules, and wires jQuery change listeners.

3. Target Architecture — EDS Rule Editor

3.1 Design Decisions

Single rules block — not per-field. In AEM, rules live on individual fields because the server aggregates them at render time. In EDS there is no server-side rendering. All rules are defined in one place: a single `tfs-form-rules` block in the DA document. This gives authors a single location to view and manage all form rules.

No encryption. In EDS, `tfs-form.js` reads the rules block and removes it from the DOM entirely before the form is presented to the user. Rules are never present in the rendered HTML.

No server-side aggregation. `tfs-form.js` reads the `tfs-form-rules` block directly during decoration. No server call, no Sling Model required.

Plugin-driven authoring only. The `tfs-form-rules` block table is authored exclusively through the Plugin Rule Editor UI.

3.2 Architecture Flow

1. Author opens Rule Editor in the Plugin
2. Plugin reads the editor DOM to discover all `tfs-form-*` blocks and their field names
3. Author builds rules visually — selecting target fields, actions, conditions from dropdowns
4. Plugin writes the `tfs-form-rules` block to the DA document
5. At page render, `tfs-form.js` reads and parses the rules block before building the form
6. The rules block is removed from the DOM — never visible in rendered HTML
7. The rule engine evaluates initial state and attaches change listeners on form fields
8. At runtime, field changes trigger re-evaluation and fields are shown or hidden accordingly

3.3 Comparison: AEM vs EDS

Concern	AEM	EDS
---------	-----	-----

Rule entry point	Per-field toolbar button	Single Rule Editor in Plugin
Rule storage	Per-field JCR node	Single <code>tfs-form-rules</code> block
Aggregation	Server-side	Not needed
Runtime delivery	Encrypted DOM attribute	In-memory — never in DOM
Encryption	AES-128-ECB	None
Field discovery	Server-side JCR servlet	Plugin reads editor DOM
JS library	jQuery	Vanilla JS

4. `tfs-form-rules` Block

4.1 Block Structure

The `tfs-form-rules` block is a two-column DA table placed within the same form section. Rules are stored as numbered key-value rows using a `{ruleNumber}-{property}` naming convention.

Example for two rules:

Key	Value
<code>1-target</code>	<code>state</code>
<code>1-action</code>	<code>show</code>
<code>1-logic</code>	<code>any</code>
<code>1-cond-1</code>	<code>country~contains~IN</code>
<code>2-target</code>	<code>research</code>
<code>2-action</code>	<code>show</code>
<code>2-logic</code>	<code>any</code>
<code>2-cond-1</code>	<code>industry~is-equal-to~diagnosis</code>

4.2 Rule Properties

Each rule is identified by a numeric prefix (1- , 2- , etc.) and consists of:

Property	Values	Description
<code>{n}-target</code>	Field name	The field to show or hide
<code>{n}-action</code>	<code>show</code> / <code>hide</code>	Action to apply when conditions are met
<code>{n}-logic</code>	<code>any</code> / <code>all</code>	<code>any</code> = OR; <code>all</code> = AND
<code>{n}-cond-{m}</code>	<code>{field}~</code> <code>{operator}~</code> <code>{value}</code>	Condition string — tilde-delimited

A rule can have multiple conditions (`{n}-cond-1` , `{n}-cond-2` , etc.).

4.3 Supported Operators

<code>equal</code>	Field value exactly matches
<code>notequal</code>	Field value does not match
<code>lessthan</code>	Numeric field value is less than
<code>lessthanequalto</code>	Numeric field value is less than or equal to
<code>greaterthan</code>	Numeric field value is greater than
<code>greaterthanequalto</code>	Numeric field value is greater than or equal to
<code>startswith</code>	Field value starts with the given string
<code>endwith</code>	Field value ends with the given string
<code>contains</code>	Field value contains the given string

4.4 Block Placement

- The `tfs-form-rules` block must be placed within the same DA section as the `tfs-form` block

- At runtime, the block is read during form decoration and removed from the DOM before the form is rendered
 - The block is never visible in the rendered page
-

5. Plugin UI — Rule Editor

5.1 Entry Point

The Rule Editor is accessible from the Plugin as a dedicated button labelled **Rules** in the field type picker. It opens a dedicated rules screen.

5.2 Field Discovery

When the Rule Editor screen opens, the plugin discovers available fields by reading the editor's live DOM. It locates all `tfs-form-*` block tables and reads the `name` property from each. This ensures the dropdown always reflects the current document state — including unsaved changes — without requiring a page preview or save.

If the editor DOM is not accessible, the plugin falls back to fetching the document source from the DA admin API.

5.3 Rule Authoring

The Rule Editor presents an inline multi-rule interface:

- All rules are visible simultaneously as cards
- Each card contains: action dropdown (show/hide), target field dropdown, logic dropdown (any/all), and one or more condition rows
- Each condition row contains: source field dropdown, operator dropdown, and value text input
- Authors can add/remove rules and add/remove conditions within a rule
- All field dropdowns are populated dynamically from the document

5.4 Read-Back and Edit

When the Rule Editor opens, the plugin reads any existing `tfs-form-rules` block and pre-populates the UI with all existing rules. Authors can modify or delete existing rules. On save, the plugin writes the complete `tfs-form-rules` block to the DA document, replacing any previous version.

6. Rule Evaluation Engine

The rule evaluation engine is part of `tfs-form.js` and runs as part of the form decoration sequence.

6.1 Overview

1. **Parse** — Read the `tfs-form-rules` block from the DOM, parse all rules into an in-memory list, and remove the block from the DOM
2. **Initial state** — For rules with action `show`, hide the target field wrapper before the form is presented. Evaluate all rules once to set correct initial visibility
3. **Listen** — Attach change listeners on all form fields
4. **Evaluate** — On any field change, re-evaluate all rules. For each rule, evaluate every condition against current field values, apply the logic operator (`any` / `all`), and show or hide the target field wrapper accordingly

6.2 Show / Hide Behaviour

Action	Conditions met	Conditions not met
<code>show</code>	Show the field	Hide the field
<code>hide</code>	Hide the field	Show the field

Visibility is applied to the field's wrapper element, not the input directly.

7. Deliverables

Deliverable	Description	Owner
Rule Editor UI	Rule Editor screen in Plugin. Includes field discovery, rule builder, read-back of existing rules, and write to <code>tfs-form-rules</code> block	Adobe Implementation Team
<code>tfs-form-rules</code> block	Block folder with no-op <code>decorate()</code> — all	Adobe Implementation Team

	processing owned by <code>tfs-form.js</code>	
Rule Evaluation Engine	Evaluation logic within <code>tfs-form.js</code> — parse, initial state, change listeners, condition evaluation, and show/hide	Adobe Implementation Team

8. Scope Boundaries

Boundary	Detail
Rule types	Show and hide only — same as AEM
Field discovery	Plugin discovers <code>tfs-form-*</code> blocks in the current DA document. Fields inside Fragment inclusions are not discoverable — same behavior as AEM
JS library	Vanilla JavaScript only
Rule authoring	Plugin is the only supported way to author rules
DOM exposure	Rules are never present in rendered page HTML

[T Best Practices For Pipetting Ergonomic On-demand webinar | Thermo Fisher Scientific - US](#)

[T Page not available | Thermo Fisher Scientific - US](#)

Form Validations

The validation layer is designed to perform client-side validation using HTML5-native capabilities and lightweight JavaScript, with backend validation handled by middleware as the source of truth. The following defines validation behavior, coverage, and implementation approach.

Client-side validation first: All form validations will run in the browser before submission — including multi-step forms — preventing unnecessary calls to middleware when fields are invalid.

HTML5 + lightweight JS (no jQuery): Validation will use native HTML5 input types and attributes (e.g. `required`, `type="email"`, `pattern`, `minlength`, `maxlength`) supplemented by small custom JavaScript helpers. This replaces the jQuery Validate plugin used in AEM and eliminates that dependency entirely.

Standard validation types covered: The solution will support required fields, email format, min/max character length, numeric validation, pattern-based (regex) checks, and file upload constraints including allowed file types, maximum file size, and maximum file count.

Custom constraints supported: Additional business rules such as confirmation field matching (e.g. email and confirm email must match) and any other cross-field checks will be implemented via custom JavaScript in `tfs-form.js`.

Consistent behaviour across steps: For multi-step forms, validation will run per step on "Next" and again on final submit, ensuring users cannot proceed with invalid data at any step.

Backend as source of truth: Middleware remains the final validation authority. Where middleware returns error responses, the front end will surface them inline next to the corresponding fields.

No encrypted constraint delivery: Unlike AEM, which aggregated and encrypted all field constraints into a `data-constraints` attribute on the form element for consumption by jQuery Validate, EDS reads validation rules directly from the field block definitions at runtime. No encrypted payload, no server-side aggregation.

Form - Request a Quote

1. Overview

Request for Quote (RFQ) forms support a `catalogNumbers` URL query parameter that allows a specific product SKU to be passed into the form at page load. When this parameter is present, the form calls a backend catalog service to retrieve product details and writes them into the `digitalData` and other page-level scripts.

This section covers the current AEM implementation of this integration and the target EDS approach.

2. Current State — AEM

2.1 Trigger Condition

`container.js` checks for the presence of `data-ispdprfq="1"` or `data-ispdpraq="1"` on the form element at page load. When either flag is present, `preparePdpRfqForm()` is called automatically.

2.2 Runtime Flow

```
1 RFQ form page loads with ?catalogNumbers=T8005
2   ↓
3 container.js reads catalogNumbers from URL query parameter
4 container.js reads expType from window._lt.user.info.expType
5   ↓
6 GET /bin/servlet/tf/form/catalogdetails
7   Query params: catalogNumbers={value}, expType={value}
8   Headers:      countryCode={digitalData.page.country}
9                 languageCode={digitalData.page.language}
10  ↓
11 AEM servlet (GetCatalogDetailsServlet) calls ThermoFisher
12 Catalog Microservice via OAuth Bearer token
13 Only products where isVisible=true are returned
14  ↓
15 Response mapped and written to digitalData.productCatalog:
16   product_sku, product_interest, brand,
17   product_line, is_instrument, product_link
18  ↓
19 $(document).trigger('digitalDataUpdated')
```

2.3 AEM Backend

Component	Detail
Servlet	<code>GetCatalogDetailsServlet</code> — path <code>/bin/servlet/tf/form/catalogdetails</code>
Service	<code>CatalogDetailsServiceImpl</code>

Auth	OAuth Bearer token — client credentials flow, token cached 55 minutes
Backend	ThermoFisher Catalog Microservice — external REST API
Credentials	<code>clientId</code> , <code>clientSecret</code> , <code>clientHost</code> — OSGi configuration, server-side only
Filter	Only products where <code>isViewable=true</code> are returned to client

2.4 Response Fields

Field	Description
<code>catalogNumber</code>	Product SKU
<code>productTitle</code>	Product name
<code>leadTarget</code>	<code>SIEBEL</code> or <code>ELMS</code> — determines <code>product_line</code> field mapping
<code>productLineCode</code>	Used as <code>product_line</code> when <code>leadTarget=SIEBEL</code>
<code>connexId</code>	Used as <code>product_line</code> when <code>leadTarget=ELMS</code>
<code>productUrl</code>	Product detail page URL
<code>opportunityChannel</code>	Brand name
<code>isInstrument</code>	<code>[1]</code> = instrument, <code>[0]</code> = non-instrument

3. Target Architecture — EDS

3.1 Design Decisions

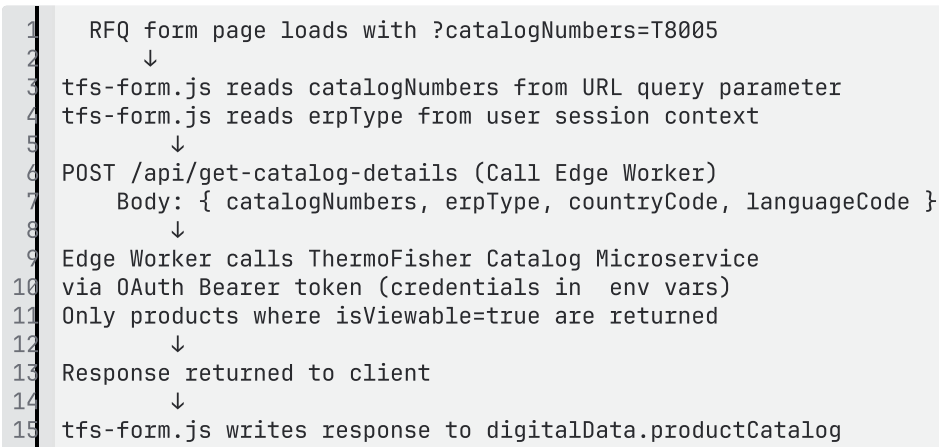
Edge Worker required

The Catalog Microservice call requires OAuth client credentials. These cannot be held client-side or stored in DA. An Edge Worker (`get-catalog-details`) acts as the secure proxy — credentials remain in environment variables, never exposed to the browser.

Behaviour is identical to AEM

`tfs-form.js` reads `catalogNumbers` from the URL on page load, calls the Edge Worker, and writes the response to `digitalData.productCatalog`.

3.2 Runtime Flow



3.3 Comparison: AEM vs EDS

Concern	AEM	EDS
Trigger	<code>data-ispdprfq</code> / <code>data-ispdpraq</code> on form element	<code>catalogNumbers</code> present in URL — detected by <code>tfs-form.js</code>
Catalog call	AEM servlet → Catalog Microservice	Edge Worker → Catalog Microservice
Credential storage	OSGi configuration (server-side)	Environment variables
Response target	<code>digitalData.productCatalog</code>	<code>digitalData.productCatalog</code> — unchanged

4. Edge Worker — `get-catalog-details`

Property	Detail
Action name	<code>get-catalog-details</code>
Trigger	Called by <code>tfs-form.js</code> on RFQ form page load when <code>catalogNumbers</code> URL param is

	present
Input	<code>catalogNumbers</code> , <code>erpType</code> , <code>countryCode</code> , <code>languageCode</code>
Output	<code>{ catalogNumber, productTitle,</code> <code>leadTarget, productLineCode,</code> <code>connexId, productUrl,</code> <code>opportunityChannel, isInstrument }</code>
Auth	OAuth client credentials — <code>clientId</code> , <code>clientSecret</code> stored as environment variables
Filter logic	Returns only products where <code>isViewable=true</code> — same rule as AEM
Owner	Adobe

Dynamic Dropdown, Cascading Options, Prefil

Dynamic Dropdowns and Cascading Options

1. Current State

TFS forms use an options component that supports two modes of populating dropdown values — static (author-defined items) and dynamic (data-driven from Content Fragments). In AEM, dynamic dropdowns are powered by a CF Root Path configured per component in the dialog. A custom Sling servlet reads the specified CF folder server-side, applies filters such as region and locale variation, and pre-renders all `<option>` tags in the page HTML at request time. No client-side call is made — options arrive fully rendered in the HTML.

For cascading options (Country → State), a separate servlet is called once on page load by `cascadeoptions.js`. It builds a complete parent-to-children map across all countries and their states, returns it in a single response, and the JS caches it in memory. Every subsequent user selection is a pure in-memory lookup — no additional network call is made per selection.

In both cases the AEM publish host, CF paths, and GraphQL schema are internal server concerns — never visible to the browser.

2. EDS Approach

In EDS there is no server-side rendering of form fields. `tfs-form.js` builds the entire form HTML in the browser. For dropdowns whose data comes from Content Fragments, `tfs-form.js` calls an App Builder action at runtime. The App Builder action calls AEM CF GraphQL API server-to-server, normalizes the response, and returns clean JSON to the browser. The AEM publish host and GraphQL schema remain hidden from the browser at all times.

CF Root Path is not exposed to authors in EDS. All known CF paths — Countries, States, Sales Regions — are hardcoded inside the App Builder action. Authors configure only the dropdown type and region via Plugin 2. No DAM path entry is required in the DA document or Plugin.

3. App Builder Action — `get-form-options`

One App Builder action handles all dynamic dropdown and cascading scenarios via an `op` parameter. The action selects the appropriate GraphQL query and CF path internally based on the operation requested.

Operation	When called	Input params	Returns
-----------	-------------	--------------	---------

<code>countries</code>	Form has dynamic country dropdown only — no state cascade	<code>region</code> , <code>language</code> , <code>format</code>	Flat array [{ <code>label</code> , <code>value</code> }] filtered by region
<code>cascade-map</code>	Form has state cascade only — country dropdown is static	<code>region</code> , <code>language</code> , <code>format</code>	Nested map { <code>"US": { "US-CA":</code> <code>"California"</code> <code>, ... }, ...</code> }
<code>country-cascade</code>	Form has both dynamic country dropdown AND state cascade	<code>region</code> , <code>language</code> , <code>format</code>	Combined { <code>countries:</code> [{ <code>label</code> , <code>value</code> }], <code>cascadeMap:</code> { ... } }

Language is read from the page locale (URL path) by `tfs-form.js` and passed to the action. If no CF variation exists for the requested language the action falls back to the master CF variation.

4. `tfs-form.js` — Operation Selection Logic

When `tfs-form.js` builds the form it scans the blocks to determine which operation to request:

```

1 | Scan all tfs-form-options blocks in the form
2 |   ↓
3 | Has dynamic country dropdown (source=dynamic, type=country)? → Yes /
   | No
4 | Has state dropdown with cascade-to set? → Yes /
   | No
5 |   ↓
6 | Both present → call country-cascade (1 call, combined response)
7 | Country only → call countries (1 call, flat list)
8 | Cascade only → call cascade-map (1 call, full nested map)

```

5. Dynamic Dropdown — Runtime Flow

```

1 | tfs-form.js builds form
2 |   ↓
3 | Detects tfs-form-options block with source = dynamic, type = country

```

```

4 Reads region + language from block
5   ↓
6 Calls App Builder: get-form-options?op=countries&region=EMEA&lang=en
7   ↓
8 App Builder queries AEM CF GraphQL internally
9 Filters countries by region, applies locale variation
10 Returns [{ label: "Germany", value: "DE" }, { label: "France", value:
    "FR" }, ...]
11   ↓
12 tfs-form.js renders <option> tags from response

```

6. Cascading Options — Runtime Flow

```

1   tfs-form.js builds form
2   ↓
3 Detects country dropdown (dynamic) + state dropdown (cascade-to =
  state)
4 Reads region + language from blocks
5   ↓
6 Calls App Builder: get-form-options?op=country-
  cascade&region=EMEA&lang=en
7   ↓
8 App Builder queries CF GraphQL
9 Returns:
10 {
11   "countries": [{ label: "Germany", value: "DE" }, ...],
12   "cascadeMap": {
13     "DE": { "DE-BW": "Baden-Württemberg", "DE-BY": "Bavaria", ... },
14     "FR": { "FR-IDF": "Île-de-France", ... }
15   }
16 }
17   ↓
18 tfs-form.js uses countries array → renders country dropdown
19 tfs-form.js caches cascadeMap in memory
20 Attaches change listener on country dropdown
21   ↓
22 User selects a country
23   ↓
24 tfs-form.js looks up selected value in cascadeMap (pure JS – zero
  network call)
25 Repopulates state dropdown with matching states
26   ↓
27 If state also cascades to a third field → recursively repopulates
  grandchild

```

7. No-Match Handling

When a selected parent value has no corresponding child options in the cascade map, the child dropdown is disabled and a configurable no-match message is displayed. Author sets this message in Plugin 2.

8. Plugin 2 — Options Block Authoring

The Plugin 2 options tab mirrors the equivalent AEM dialog fields. Fields shown or hidden based on Options Type selection — same behaviour as the AEM dialog.

Plugin Field	Values	Shown when	Notes
Options Type	Default / Country / State	Always	Controls App Builder operation and cascade behaviour
Source	Static / Dynamic	Always	Static = author items in block table. Dynamic = App Builder fetch
Region	Global / NA / EMEA / IPAC / JP / LATAM	Source = Dynamic, Type = Country	Hardcoded list — no server call needed
Format	ISO / Siebel	Source = Dynamic, Type = Country	Controls value format returned by App Builder
Cascade	Toggle on/off	Type = Default or State	Disabled for Country type — same restriction as AEM
Cascade to	Dropdown of sibling options blocks	Cascade = on	Populated by Plugin scanning DA document — same mechanism as Rule Editor field discovery
No-match message	Free text	Cascade = on	Shown in child dropdown when parent selection has no matching children

Note: Cascade is not available for Country type. When Country is selected in Options Type, the Cascade toggle is disabled and a warning is shown — identical to the AEM dialog behaviour.

9. DA Document — tfs-form-options Block Schema

Key	Value	Notes
name	country	Field name — used by tfs-form.js and Rule Engine
label	Country	Display label
type	drop-down	Rendered as <code><select></code>
source	dynamic	Triggers App Builder fetch
options-type	country	Determines App Builder operation
region	EMEA	Passed to App Builder as region filter
format	iso	ISO or Siebel value format
cascade-to	state	Name of child field — empty if no cascade
no-match-msg	No states available	Shown when cascade has no match
required	true	Validation
required-message	Please select a country	Validation message

OQ — Additional CF sources: Apart from Countries, States and Sales Regions, are there any other Content Fragment folders used to populate dynamic dropdowns across TFS forms? All CF paths must be confirmed before the App Builder action can be fully built, as no CF path picker will exist in the EDS authoring experience.

Prefill Logic

Current State

Overview

TFS forms support prefill from data sources. These are handled by separate client-side JS files and one server-side servlet. `container.js` orchestrates the primary prefill sequence.

Data Source 1 — User Profile (userdetails servlet)

`container.js` polls `window._lt.user.info` every 300ms up to 50 attempts (15-second maximum). `window._lt` is the ThermoFisher identity layer loaded on the page. Once available, it reads `userId` and `erpType` from `window._lt.user.info` and calls:

```
1 GET /bin/servlet/tf/form/userdetails.html?userId={userId}&erpType={erpType}
```

The AEM servlet (`UserDetailsServlet`) calls an external **Catalog Detail Service** API using OAuth Bearer token authentication (client credentials, 55-minute token cache). The API returns a `consumers[]` array. The servlet:

1. Extracts the `user` object and flattens `userRoles`
2. Locates the account where `defaultShipTo = true`
3. Enriches the account with human-readable country and state names by looking up country and state Content Fragments in AEM DAM
4. Returns a flat JSON object to the browser

`prefillFormFields()` in `utility.js` then maps the response fields to form fields. Field matching uses normalised lowercase keys and a hardcoded alias map to handle naming mismatches between the API response and form field names (e.g. `firstname` → `firstName`, `C_FirstName`, `first_name`).

Fallback: If request params are absent, the servlet attempts to decrypt the `store_jwt_persistence` browser cookie using AES-256-CBC (32-char key) and extracts

`uId` , `unm` , `erp` claims from the JWT payload.

Data Source 2 — URL Query Parameters (visible fields)

`input.js` reads a `data-qparam` attribute on each input field. If present, it reads the matching query parameter from the current page URL and sets the field value on load.

Data Source 3 — Cookies, digitalData, URL Parameters (hidden fields)

`readdataobject.js` handles prefill for hidden fields. It reads from three sources based on attributes authored on the field:

Attribute	Source
<code>data-cookiename</code>	Browser cookie
<code>data-digitaldata</code>	<code>window.digitalData</code> object
<code>data-qparam</code>	URL query parameter

Data Source 4 — JS Window Objects (hidden fields)

`hiddenFormFields.js` reads a `data-jsparamname` attribute. It resolves the value from the specified `window` object property using dot-notation path traversal.

Current State Summary

Source	Trigger	Handler	Fields
User profile (Catalog Detail Service)	<code>window._lt</code> available	<code>container.js</code> + <code>userdetails</code> servlet	Visible profile fields
URL query params	Page load	<code>input.js</code> (<code>data-qparam</code>)	Visible fields
Cookies	Page load	<code>readdataobje</code> <code>ct.js</code> (<code>data-</code> <code>cookiename</code>)	Hidden fields

<code>window.digitalData</code>	Page load	<code>readdataobject.js (data-digitaldata)</code>	Hidden fields
URL query params (hidden)	Page load	<code>readdataobject.js (data-qparam)</code>	Hidden fields
JS window objects	Page load	<code>hiddenFormFields.js (data-jsparamname)</code>	Hidden fields
Telephone country default	Server render	<code>InputModelImpl.java (CK_ISO_CODE cookie)</code>	Telephone field

Target Architecture — EDS Recommended Approach

`tfs-form.js` owns all prefill orchestration

All prefill logic is consolidated in `tfs-form.js` as part of the orchestration sequence. The individual JS file handlers (`readdataobject.js` , `hiddenFormFields.js` , `input.js`) are not ported — their logic is absorbed into `tfs-form.js` .

`userdetails` servlet replaced by App Builder action

The Catalog Detail Service is an external API — AEM is acting only as a proxy. In EDS, the App Builder `get-user-profile` action calls the same Catalog Detail Service directly with the same OAuth client credentials.

Country/state CF enrichment handled in App Builder

The AEM servlet enriches ISO country and state codes with human-readable names by querying AEM Content Fragments. The `get-user-profile` App Builder action handles this enrichment internally using the same country/state data already served by the `get-form-options` action.

JWT cookie fallback ported to App Builder

The `store_jwt_persistence` cookie is a browser cookie present in all requests. The AES-256-CBC decryption and JWT claim extraction logic (`uId` , `unm` , `erp`) is ported to the App Builder action. The action accepts the raw cookie value as a request parameter and resolves the user identifier if explicit params are not provided.

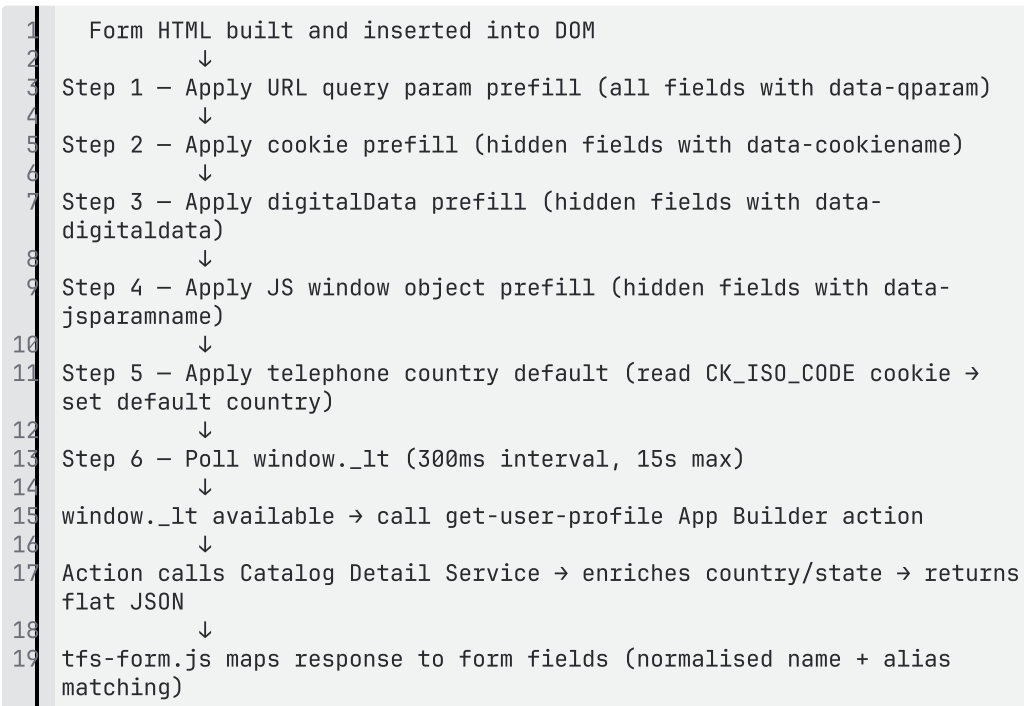
Telephone country default moved to client-side

The `CK_ISO_CODE` server-side cookie read is the only AEM server-side prefill. In EDS, `tfs-form.js` reads the `CK_ISO_CODE` cookie client-side directly and sets the telephone field's default country during form build. No server render dependency.

`window._lt` polling is unchanged

`window._lt` is a ThermoFisher identity layer — it is not AEM-specific and will be present on EDS pages. The polling behaviour (300ms interval, 15-second timeout) is retained in `tfs-form.js`.

Prefill Orchestration Sequence in `tfs-form.js`



Steps 1–5 are synchronous and complete before the form is presented. Step 6 is asynchronous — user profile prefill populates fields after the form is visible, consistent with current AEM behaviour.

`get-user-profile` App Builder Action

Concern	Detail
---------	--------

Endpoint	App Builder action — called by <code>tfs-form.js</code>
Input	<code>userId + erpType</code> , or <code>emailAddress + erpType</code> , or raw <code>store_jwt_persistence</code> cookie value as fallback
Authentication	OAuth client credentials — same <code>clientId</code> / <code>clientSecret</code> as current AEM OSGi config
Upstream API	Catalog Detail Service (same <code>consumerApiUrl</code>)
Response transformation	Extracts <code>consumers[0].user</code> , flattens <code>userRoles</code> , enriches <code>defaultShipTo</code> account with country and state names
Country/state enrichment	Handled internally — no AEM CF lookup; uses same data served by <code>get-form-options</code>
Token caching	Bearer token cached within the action runtime; refreshed on expiry

EDS Data Source Summary

Source	Trigger	Handler	Fields
User profile (Catalog Detail Service)	<code>window._lt</code> available	<code>tfs-form.js</code> → <code>get-user-profile</code> App Builder action	Visible profile fields
URL query params	Form build	<code>tfs-form.js</code> (<code>data-qparam</code>)	Visible and hidden fields
Cookies	Form build	<code>tfs-form.js</code> (<code>data-cookiename</code>)	Hidden fields
<code>window.digitalData</code>	Form build	<code>tfs-form.js</code> (<code>data-digitaldata</code>)	Hidden fields
JS window objects	Form build	<code>tfs-form.js</code> (<code>data-</code>)	Hidden fields

		jsparmname)	
Telephone country default	Form build	tfs-form.js (reads CK_ISO_CODE cookie)	Telephone field

Dependencies

All Catalog Detail Service API details, OAuth credentials, and JWT configuration are to be provided by the TFS Backend Team prior to implementation. Adobe will implement the `get-user-profile` App Builder action and `tfs-form.js` prefill orchestration against the confirmed API contract.

Forms - Recaptcha

1. Overview

TFS forms use Google reCAPTCHA v2 Invisible for bot detection. The user sees no challenge — Google's JS silently analyses browser behaviour and generates a token when the user is determined to be legitimate. The token is validated server-side before form submission proceeds. This document covers the current AEM implementation and the target EDS design.

2. Current State — AEM

2.1 Component Rendering

The reCAPTCHA component (`formcommons/components/form/recaptcha/v1`) is placed inside the form container by the author. At server render time, `RecaptchaModelImpl` reads OSGi configuration (`RecaptchaConfiguration`) and emits three HTML elements:

Script tag — loads Google's reCAPTCHA JS API:

```
1 <script src="{domain}/recaptcha/api.js?hl={pageLanguage}" async defer>
```

Invisible container div:

```
1 <div class="g-recaptcha cmp-form-recaptcha"
```

```
data-sitekey="{siteKey}"
```

```
data-size="invisible"
```

```
data-callback="onSubmitV2">
```

Google T&C privacy text div

If the page path matches a configured disabled country path (EU, DE, RU) — the entire reCAPTCHA div is suppressed at render time. Nothing is emitted to HTML.

For China and Hong Kong page paths, `www.recaptcha.net` is used as the base domain instead of `www.google.com` — these regions block Google's domain.

2.2 OSGi Configuration

All reCAPTCHA configuration is centralised in OSGi (`RecaptchaConfiguration`):

Property	Purpose
<code>defaultDomain</code>	<code>https://www.google.com</code>

<code>fallbackDomain</code>	<code>https://www.recaptcha.net</code> — used for China/HK
<code>fallbackContentPaths</code>	Page paths that use fallback domain
<code>jsResourceEndpoint</code>	<code>/recaptcha/api.js</code>
<code>validateTokenEndpoint</code>	<code>/recaptcha/api/siteverify</code>
<code>v2InvisibleSiteKey</code>	Public site key — rendered into HTML
<code>v2InvisibleSecretKey</code>	Secret key — server-side only, never in HTML
<code>RecaptchaDisabledContentPaths</code>	Page paths where reCAPTCHA is suppressed

2.3 Form Submission Flow

1 — Submit click

`container.js` intercepts submit. Runs field validation. Calls `triggerCaptcha()`.

2 — `triggerCaptcha()`

Checks `isCaptchaConfigured()` — `.g-recaptcha` div present, `grecaptcha` library loaded, user not excluded. If configured and no token exists yet → calls `grecaptcha.execute()`. Returns immediately — does not wait for Google's response.

3 — `ajaxComplete` listener registered

`container.js` registers `$(document).ajaxComplete()` watching for the captcha callback URL. Form submission is paused here.

4 — Google invisible challenge

`grecaptcha.execute()` triggers Google's JS to silently analyse the user. Entirely between browser and Google's servers — no AEM call. Google injects the token into the hidden `g-recaptcha-response` field and calls `onSubmitV2`.

5 — `onSubmitV2` callback

Sets `data-validated = true` on the `.g-recaptcha` div. Makes AJAX POST to AEM:

```
1 | POST /bin/servlet/tf/form/getRecaptchaResponseServlet.json
```

```
token={token}
```

```
formstart={pagePath}
```

6 — AEM servlet calls Google siteverify

`GetRecaptchaResponseServlet` receives the token. `RecaptchaResponseServiceImpl` calls:

```
1 GET {domain}/recaptcha/api/siteverify?secret={secretKey}&response={token}
```

Returns Google's `{ "success": true/false }` response to the browser.

7 — Form submission gate

`ajaxComplete` fires `fireSubmission()`. Checks `getCaptchaFState()` — reads `data-validated` on the reCAPTCHA div. If `true` → proceeds to `getFormSecurityToken()` then `saveFormData()`. The `g-recaptcha-response` field is explicitly disabled before form serialisation — excluded from the submission payload.

2.4 All API Calls in reCAPTCHA Flow

Call	Direction	Purpose
<code>grecaptcha.execute()</code>	Browser → Google (JS library)	Triggers invisible challenge, token generation
POST <code>/bin/servlet/tf/form/getRecaptchaResponseServlet.js</code>	Browser → AEM	Sends token for server-side validation
GET <code>{domain}/recaptcha/api/siteverify?secret=xxx&response={token}</code>	AEM → Google	Actual token verification

3. Target Architecture — EDS

3.1 Design Decisions

reCAPTCHA v2 Invisible retained

Same Google reCAPTCHA v2 Invisible approach.

AEM servlet replaced by App Builder / Edge Worker

The only AEM-specific call in the entire flow is `POST`

`/bin/servlet/tf/form/getRecaptchaResponseServlet.json`. This is replaced by an App Builder action. The secret key moves to App Builder environment variables — never in client-side code.

Rendering config in DA Sheet

AEM used OSGi configuration read server-side at render time. In EDS there is no server render — `tfs-form.js` builds the form dynamically. All non-sensitive rendering configuration is stored in a DA Sheet (`tfs-config`) and delivered as a CDN-cached JSON endpoint. Only the secret key remains in App Builder environment variables.

`tfs-form.js` owns all reCAPTCHA orchestration

Component detection, config fetch, script injection, div construction, `onSubmitV2` definition, submission gating — all consolidated in `tfs-form.js`. No separate clientlib.

3.2 reCAPTCHA Configuration — DA Sheet

A DA Sheet named `tfs-config` is maintained by the TFS team. It is delivered by EDS as a CDN-cached JSON endpoint:

```
1 | /{path}/tfs-config.json
```

Key	Value	Purpose
<code>siteKey</code>	<code>6LeK8o0...</code>	Public site key — placed in <code>data-sitekey</code> on div
<code>defaultDomain</code>	<code>https://www.google.com</code>	Base domain for JS load and siteverify
<code>fallbackDomain</code>	<code>https://www.recaptcha.net</code>	Used for China/HK locales
<code>fallbackLocales</code>	<code>cn,hk</code>	Locales that use fallback domain
<code>disabledLocales</code>	<code>eu,de,ru</code>	Locales where reCAPTCHA is suppressed

Any config change is a sheet edit — no code deploy required, no DA document changes needed across forms.

3.3 Secret Key — App Builder

`v2InvisibleSecretKey` is stored as an App Builder / Edge worker environment variable. It is never returned to the client, never in the DA Sheet. Only the `validate-recaptcha` App Builder action reads it.

3.4 `tfs-form-recaptcha` Block in DA Document

Authors place a `tfs-form-recaptcha` block inside the form section in the DA document. The block has no editable configuration — its presence alone signals to `tfs-form.js` that reCAPTCHA should be wired up for this form. All configuration comes from the DA Sheet.

3.5 EDS Form Rendering Flow

```
1 tfs-form.js initialises
2   ↓
3 tfs-form-recaptcha block present in DA document?
4   └ No → skip all reCAPTCHA logic
5   └ Yes → continue
6   ↓
7 fetch /tfs-config.json
8   ↓
9 Check page locale against disabledLocales
10  └ Locale disabled → do not render reCAPTCHA, form submits without
    it
11  ↓
12 Check page locale against fallbackLocales
13  └ Fallback locale (CN/HK) → use fallbackDomain
14  └ Otherwise → use defaultDomain
15  ↓
16 Inject <script src="{domain}/recaptcha/api.js?hl={locale}"> into page
17   ↓
18 Build <div class="g-recaptcha" data-sitekey="{siteKey}"
19     data-size="invisible" data-callback="onSubmitV2">
20   ↓
21 Define window.onSubmitV2 globally
22   ↓
23 Form rendered to user
```

3.6 EDS Form Submission Flow

```
1 User clicks Submit
2   ↓
3 tfs-form.js runs field validation
4   ↓
5 isCaptchaConfigured() → .g-recaptcha div present + grecaptcha loaded
6   ↓
7 grecaptcha.execute()
8   ↓
9 Google invisible challenge (browser ↔ Google – unchanged from AEM)
10  ↓
11 onSubmitV2(token) fires
12  ↓
13 POST App Builder validate-recaptcha action
14   token={token}
15  ↓
16 App Builder reads secretKey from env var
```

```

17 GET {domain}/recaptcha/api/siteverify?secret={secretKey}&response=
   {token}
18     ↓
19 { "success": true/false } returned to tfs-form.js
20     ↓
21 success = true → form submitted (g-recaptcha-response excluded from
   payload)
22 success = false → show error, abort

```

3.7 API Calls in EDS reCAPTCHA Flow

Call	Direction	Purpose
<code>fetch /tfs-config.json</code>	Browser → EDS CDN	Get rendering config from DA Sheet
<code>grecaptcha.execute()</code>	Browser → Google (JS library)	Triggers invisible challenge — unchanged
<code>POST validate-recaptcha</code> App Builder / Edge worker	Browser → App Builder/Edge Worker	Send token for server-side validation
<code>GET {domain}/recaptcha/api/siteverify</code>	App Builder → Google	Actual token verification

3.8 AEM vs EDS Comparison

Concern	AEM	EDS
reCAPTCHA variant	v2 Invisible	v2 Invisible
Rendering config storage	OSGi (<code>RecaptchaConfiguration</code>)	DA Sheet (<code>recaptcha-config.json</code>)
Config change impact	OSGi config update — all forms	Sheet edit — all forms
Site key delivery	Server-rendered into HTML	<code>tfs-form.js</code> reads from DA Sheet JSON
Secret key storage	OSGi (server-side)	App Builder env var

Token validation	AEM servlet → Google siteverify	App Builder action → Google siteverify
reCAPTCHA div construction	Server-side HTL render	<code>tfs-form.js</code> at runtime
<code>onSubmitV2</code> definition	AEM clientlib	<code>tfs-form.js</code>
Disabled locale check	Server-side path matching at render	<code>tfs-form.js</code> locale check against DA Sheet list
Fallback domain (CN/HK)	Server-side path matching	<code>tfs-form.js</code> locale check against DA Sheet list
<code>g-recaptcha-response</code> exclusion from payload	<code>container.js</code> disables field	<code>tfs-form.js</code> disables field
JS library	jQuery (<code>container.js</code>)	Vanilla JS (<code>tfs-form.js</code>)

4. Out of Scope

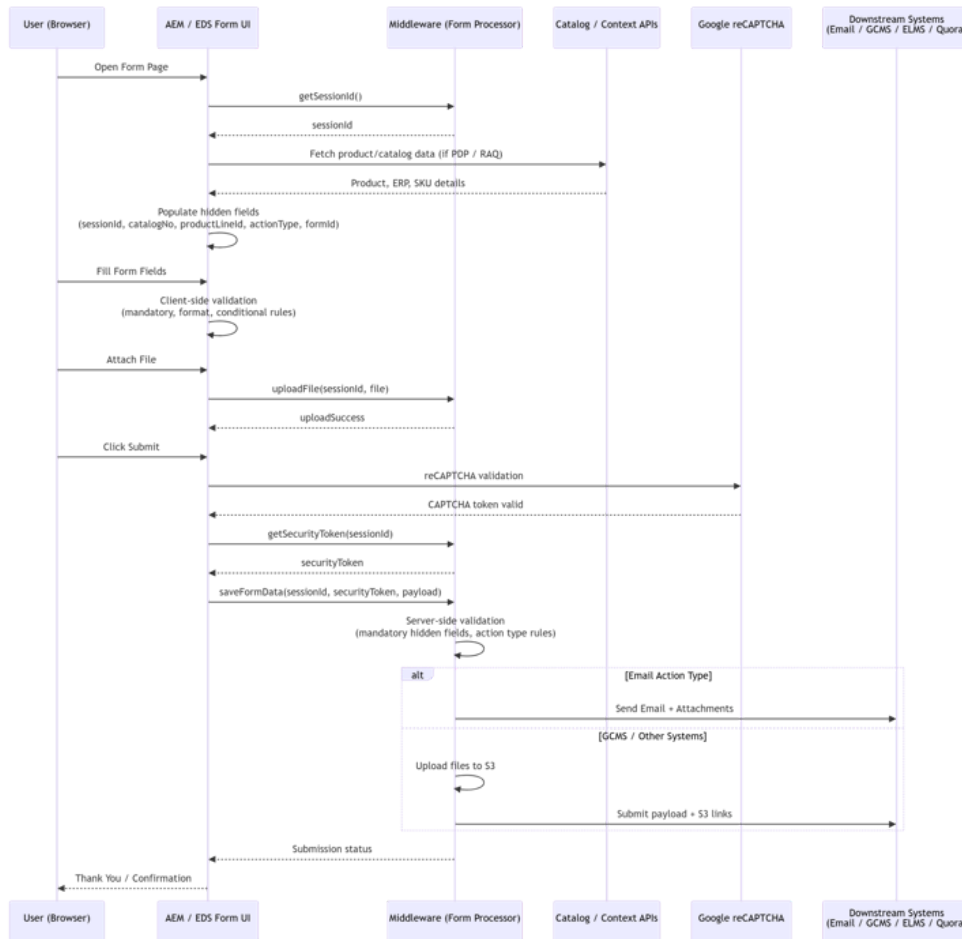
Item	Reason
User exclusion for automated testing	TFS-owned internal service . TFS implements and owns the exclusion service and its integration.

5. Runtime Dependencies and Ownership

Dependency	Owner	Required For
App Builder /Edge Worker <code>validate-recaptcha</code>	Adobe	Token validation — replaces AEM servlet

tfs-form.js reCAPTCHA orchestration	Adobe	Config fetch, div construction, submission gate
User exclusion service (automated testing)	TFS	TFS to implement independently if required

Form – Render, Submission, and Document Upload



1. Overview

Forms follow a **middleware-driven architecture** where:

- **EDS/DA acts as UI + client-side orchestrator**
- **All business logic, storage, validation, and integrations remain in middleware**

Key principles:

- No server-side form processing in EDS
- No storage of form data or uploaded files in EDS
- Browser directly interacts with middleware APIs

2. Document Upload

Behavior

- Multi-file upload supported

- File upload flow:
 - Browser uploads files directly to middleware
 - Middleware:
 - Stores files temporarily (DB or storage)
 - Associates files with a `sessionId`
- On final form submission:
 - Middleware:
 - Attaches files to email **or**
 - Uploads to S3 and passes references to downstream systems
- AEM/EDS:
 - Does **not store or process files**

EDS Approach

- Same pattern retained:
 - Browser → Middleware (direct upload)
- EDS block JS:
 - Collects selected files
 - Calls upload API
- Middleware:
 - Stores files and maps to session

Key Points

- No file storage in EDS
- Upload handling remains fully middleware-driven
- EDS requires:
 - Upload endpoint configuration
 - Error handling (size/type validation)

3. Form Rendering (Load)

Behavior

On page load, form initialization includes:

Session Initialization

- Browser calls middleware to generate `formsessionId`

- `formsessionId` :
 - Stored in hidden field and/or JS state
 - Used for upload and submission

User Prefill

- User profile is fetched via backend API
- Fields are prefilled based on mapping logic

Forms — Rule Editor, Validations

Product / Metadata Prefill (PDP Forms)

- For product-based forms:
 - Browser calls Catalog API
 - Populates hidden fields:
 - Catalog number
 - Product line ID
 - ERP / metadata fields

EDS Approach

- All logic executed via **form block JavaScript**

On page load:

- Call middleware session API:
- Store sessionId in hidden field / JS state
- Call:
 - User details API (for prefill)
 - Catalog API (for PDP forms, if applicable)

Catalog API Handling

- If browser-safe:
 - Direct call from EDS
- If restricted:
 - Call via App Builder / Edge Worker

Key Points

- No server-side rendering of user/product data
- All data populated at runtime via APIs
- EDS only renders hidden fields expected by middleware

4. Form Submission

Behavior

Submission follows a **two-step middleware protocol**:

Step 1 – Get Security Token

- Input: `sessionId`
- Output: `securityToken`

Step 2 – Submit Form

```
1 | POST /saveFormData
```

Payload includes:

- User-entered fields
- Hidden fields (sessionId, formId, product info, ERP data)
- `securityToken`

Middleware Responsibilities

- Validate input
- Process and store data
- Attach uploaded files (via sessionId)
- Route to downstream systems:
 - Email
 - S3 / GCMS / CRM / marketing platforms

EDS Approach

- Same protocol retained

Flow:

1. Run client-side validation
2. Call `getSecurityToken`
3. Call `saveFormData` with full payload

Response Handling

- Success:
 - Show confirmation / redirect
- Failure:
 - Display backend error messages

Key Point

Form submission is **browser** → **middleware**;

EDS only constructs and sends payload, it does not process it.

5. Summary

EDS (UI + Client JS)

- Render form and hidden fields
 - Retrieve `sessionId`
 - Call profile / catalog APIs
 - Prefill fields
 - Handle file uploads (via middleware APIs)
 - Perform client-side validation
 - Orchestrate:
 - Token generation
 - Form submission
 - Display success/error UI
-

Middleware

- Generate `formsessionId` and security token
- Handle file uploads and storage
- Validate and process submissions
- Execute business logic
- Integrate with downstream systems
- Ensure security, logging, auditing

User Groups and Permissions

Workflows

Data Layer & Analytics

Integrations

<code>.pfpsnippet</code>	Sling selector	Triggers the syndication servlet
<code>.html</code>	Extension	Standard AEM HTML extension
<code>/sku/CHROMELEON7/3</code>	Suffix	type=sku, value=CHROMELEON7, pod=3

2.2 Component Architecture in AEM

Content Snippet Template (`lifetech/templates/content_snippet`)

- Specialized AEM page template for fragment pages
- Page component: `lifetech/components/pages/content-snippet`
- Renders via `content-snippet.jsp`
- On publish: outputs raw HTML with no page shell
- Uses a custom paragraph system: `SnippetParsys`

SnippetParsys (`lifetech/components/snippetparsys`)

- Custom AEM paragraph system (container)
- Supports column layouts (START/BREAK/END)
- Strips decoration tags on publish for clean HTML output
- Allowed components: text, textimage, heading, Brightcove video, raw HTML, interactive containers

Tag Taxonomy

- Namespace: `pfp:` (Product Family Page)
- Tag format: `pfp:sku/{SKU-VALUE}/pod_{POD-ID}`
- Example: `pfp:sku/CHROMELEON7/pod_3`
- Authors tag each Content Snippet page with the appropriate tag to register it for a SKU + pod combination

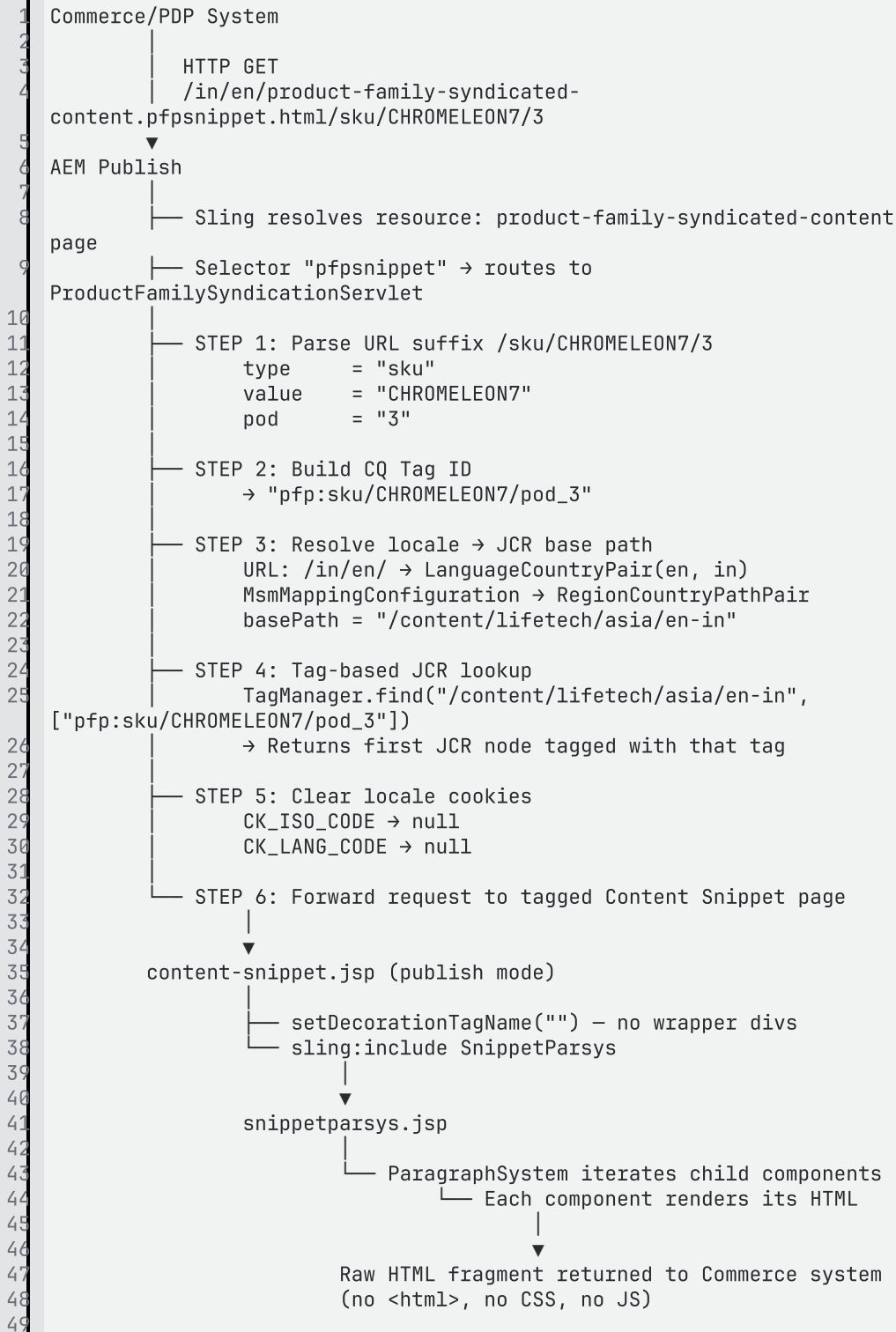
2.3 Servlet — ProductFamilySyndicationServlet

File: `lifetech-web-core/src/main/java/com/lifetechnologies/web/servlets/ProductFamilySyndicationServlet.java`

Registration:

```
1 sling.servlet.resourceTypes = sling/servlet/default (any resource)
2 sling.servlet.selectors     = pfpsnippet
3 sling.servlet.extensions    = html
4 sling.servlet.methods       = GET
5
```

2.4 End-to-End Request Flow



3. Proposed EDS Architecture

3.1 Core Principles

The EDS migration replaces the entire AEM-side syndication stack with three components:

1. **DA (da.live)** — authors create and manage fragment content (replaces AEM Sites + Content Snippet Template)
2. **EDS blocks** — define the rendering and styling of content components (replaces JSP components + SnippetParsys)
3. `aem-embed.js` — a Web Component that fetches and renders fragments in the commerce page (replaces the Java servlet + `content-snippet.jsp` pipeline)

3.2 The `aem-embed` Web Component

Adobe provides an open-source Web Component specifically designed for this pattern:

Repository: `https://github.com/adobe/aem-embed`

Documentation: `https://www.aem.live/docs/aem-embed`

`aem-embed.js` is a standards-based **Custom Element** (`<aem-embed>`) that:

1. Reads the `url` attribute
2. Automatically appends `.plain.html` to fetch the EDS fragment
3. Creates an isolated **Shadow DOM** — prevents CSS/JS conflicts with the commerce page
4. Loads `styles/styles.css` from the EDS repo into the Shadow DOM
5. Runs EDS's `decorateMain()` pipeline to convert DA table markup into block structure
6. For every block in the fragment, loads its dedicated `blocks/{name}/{name}.css` and `blocks/{name}/{name}.js` into the Shadow DOM
7. Renders fully styled, fully functional content — isolated from the commerce page

3.3 How Commerce System Uses It

One-time setup — add script to commerce /PDP page `<head>` :

```
1 <script src="https://www.thermofisher.com/scripts/aem-embed.js"
2   type="module"></script>
```

Per-product — drop tag where each pod should appear:

```
1 <aem-embed
   url="https://www.thermofisher.com/in/en/snippets/sku/CHROMELEON7/pod-
   1"></aem-embed>
2 <aem-embed
   url="https://www.thermofisher.com/in/en/snippets/sku/CHROMELEON7/pod-
   2"></aem-embed>
```

```

3 <aem-embed
  url="https://www.thermofisher.com/in/en/snippets/sku/CHROMELEON7/pod-
  3"></aem-embed>
4

```

The commerce system constructs the URL from data it already owns: locale + SKU + pod number.

3.4 EDS Repo Structure

```

1 TFS EDS Repository
2 |— scripts/
3 |   |— aem-embed.js      ← copied from adobe/aem-embed (Adobe-
  maintained)
4 |   |— scripts.js        ← standard EDS scripts, exports
  decorateMain()
5 |   |— styles/
6 |       |— styles.css    ← global EDS styles (loaded into Shadow
  DOM)
7 |       |— fonts.css     ← fonts (loaded into Shadow DOM)
8 |— blocks/
9 |   |— text/
10 |       |— text.js
11 |       |— text.css
12 |   |— textimage/
13 |       |— textimage.js
14 |       |— textimage.css
15 |   |— video/
16 |       |— video.js
17 |       |— video.css
18 |   ...
19 |— (DA content synced by EDS pipeline)
20 |   /in/en/snippets/sku/CHROMELEON7/pod-1
21 |   /in/en/snippets/sku/CHROMELEON7/pod-2
22 |   /us/en/snippets/sku/CHROMELEON7/pod-1
23

```

3.5 DA Authoring

Authors work in **da.live** . Each fragment is a document saved at the URL path convention:

```

1 /in/en/snippets/sku/{SKU}/pod-{N}
2

```

Inside the document, authors use standard EDS block syntax:

```

1 +-----+
2 | text                                     |
3 +-----+
4 | ## Chromeleon Software                 |
5 | Increase productivity...               |
6 | [Learn more](url)                     |
7 +-----+
8
9 +-----+
10 | video                                  |
11 +-----+
12 | https://brightcove.../xyz             |
13 +-----+
14

```

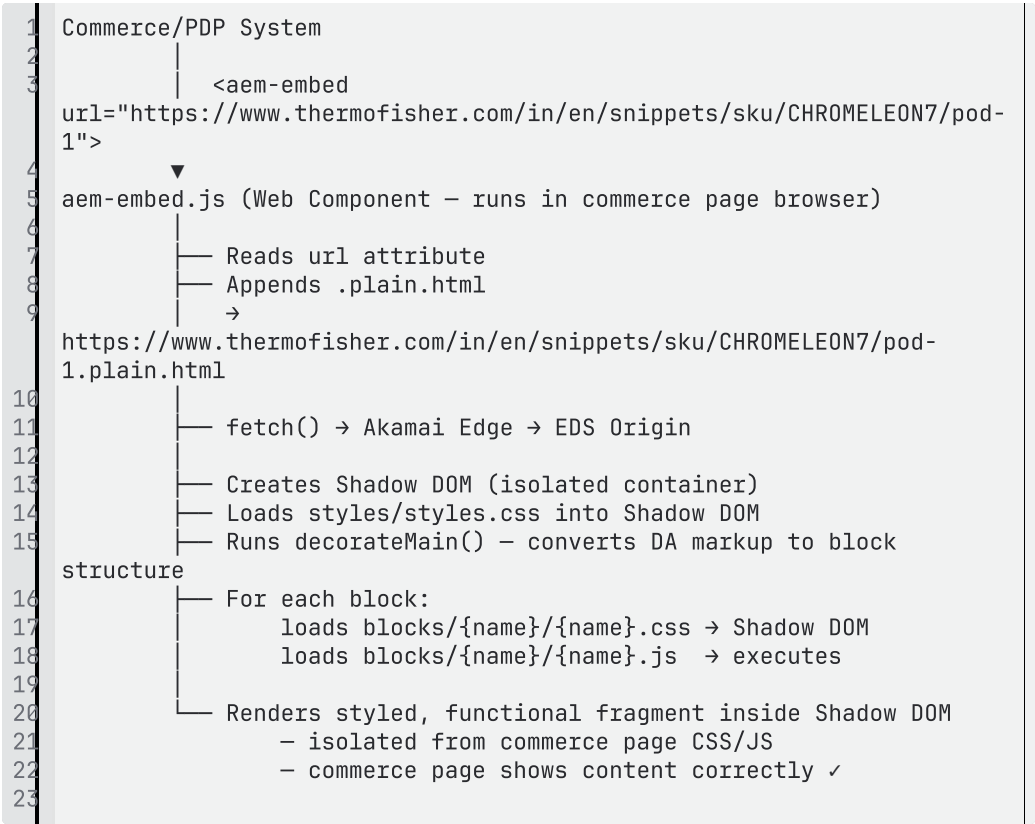

No tags. No taxonomy. No template selection. Just a document at the right path.

3.6 CORS Configuration

Because the commerce system and the EDS content are served from the same origin (`thermofisher.com`), no CORS configuration is required. If the commerce system is hosted on a different domain/origin, then configure CORS in the TFS EDS project headers config:

```
1 | access-control-allow-origin: *
2 |
```

3.7 End-to-End Request Flow (EDS)



4. Impact Analysis

4.1 Commerce / PDP System — Changes Required

Change	Details
Add <code>aem-embed.js</code> script tag	One <code><script></code> tag in page <code><head></code>
Replace servlet HTTP calls with <code><aem-embed></code> tags	Replace existing GET calls to <code>.pfpsnippet.html/sku/X/N</code>

	with <code><aem-embed url="..."></code> tags
URL construction logic	Build EDS URL from existing data: <code>/ {locale} / snippets / sku / {SKU} / pod - {N}</code>
CSS updates	Commerce page CSS may need updates to work with Shadow DOM boundaries. Global styles that targeted injected AEM HTML structure will need review

4.2 TFS AEM — What Is Decommissioned

AEM Artifact	Status
<code>ProductFamilySyndicationServlet.java</code>	Decommissioned
<code>MsmMappingConfiguration.java</code>	Decommissioned
<code>content-snippet.jsp</code>	Decommissioned
<code>snippetparsys.jsp</code> + <code>snippetparsys</code> component	Decommissioned
<code>content_snippet</code> AEM template	Decommissioned
<code>pfp:</code> tag namespace and taxonomy	Decommissioned
AEM MSM live copies for snippet pages	Decommissioned (replaced by DA MSM)
All Content Snippet pages in AEM Sites	Migrated to DA

5. Runtime Ownership

5.1 Adobe Development Team

Responsibility	Task
EDS repo setup	Initialise TFS EDS project with standard <code>scripts/scripts.js</code> , <code>styles/styles.css</code>
Copy <code>aem-embed.js</code>	Copy from <code>github.com/adobe/aem-embed</code> into <code>/scripts/aem-embed.js</code>
Build EDS blocks	Develop <code>text</code> , <code>textimage</code> , <code>video</code> , <code>heading</code> blocks with JS + CSS
CORS configuration	Add <code>access-control-allow-origin: *</code> to EDS headers config if domain differs
DA MSM setup	Configure base site + satellite site relationships in DA org config
URL convention definition	Define and document the <code>/snippets/sku/{SKU}/pod-{N}</code> path convention

5.2 TFS Content Team (Authors)

Responsibility	Task
Path convention adherence	Save documents at <code>/in/en/snippets/sku/{SKU}/pod-{N}</code> — path IS the identifier
No tagging required	Tags are fully eliminated — authors do not need to manage any tag taxonomy

5.3 Commerce / PDP Team

Responsibility	Task
----------------	------

Script integration	Add <code>aem-embed.js</code> script tag to commerce page template
Tag integration	Replace existing servlet HTTP calls with <code><aem-embed url="..."></code> tags
URL construction	Build EDS URL from locale + SKU + pod data available in commerce system
CSS review	Review and update any commerce-side CSS that targets AEM-specific HTML structure

6. Parity Check

6.1 Functional Parity

Capability	AEM (Current)	EDS + DA (Target)	Parity
Rich content authoring (text, images, video)	AEM Sites — drag & drop components	DA — block authoring	✅ Equivalent
Raw HTML fragment delivery	<code>content-snippet.jsp</code> strips page shell	<code>.plain.html</code> returns <code><main></code> content only	✅ Equivalent
Styled content delivery	Commerce page CSS styles AEM HTML	<code>aem-embed.js</code> loads block CSS into Shadow DOM	✅ Improved — TFS now owns styling
Locale-specific content	AEM MSM live copies per locale	DA MSM satellite sites per locale	✅ Equivalent
Multiple pods per SKU	Multiple tagged pages per SKU	Multiple DA documents per path convention	✅ Equivalent

SKU-based content lookup	Tag taxonomy <code>pfp:sku/{SKU}/pod_{N}</code>	URL path <code>/snippets/sku/{SKU}/pod-{N}</code>	✓ Simplified
Commerce system independence	Commerce calls AEM servlet URL	Commerce uses <code><aem-embed></code> tag	✓ Improved — cleaner contract
Content updates without commerce deploy	Yes — AEM publish	Yes — DA publish	✓ Equivalent
Block JS initialisation (video players etc.)	AEM clientlibs per component	<code>aem-embed.js</code> loads block JS per block	✓ Equivalent
CSS isolation from commerce page	No isolation — commerce CSS must not conflict	Full Shadow DOM isolation	✓ Improved

6.2 Technical Parity

Technical Concern	AEM (Current)	EDS + DA (Target)
Servlet / entry point	<code>ProductFamilySyndicationServlet.java</code> (Java OSGi)	<code>aem-embed.js</code> (Web Component, browser-side)
Content lookup mechanism	<code>TagManager.find()</code> — JCR tag search	URL path convention — no lookup needed
Locale resolution	<code>MsmMappingConfiguration.java</code> — server-side Java	DA MSM — platform-level, at delivery time
Rendering pipeline	<code>content-snippet.jsp</code> +	EDS <code>decorateMain()</code> + block decoration

	snippetparsys.js p	
Maintenance	Java OSGi bundle — requires AEM developer	Standard JS/CSS — any front-end developer

7. Tag Removal and URL Simplification

7.1 URL Convention Replaces Tags Entirely

In EDS + DA, the **path where the author saves the document IS the identifier**. No separate tagging step. No taxonomy to manage.

The rule is simple:

```
1 /{locale}/snippets/sku/{SKU-VALUE}/pod-{POD-NUMBER}
2
```

Examples:

Content	DA Document Path
CHROMELEON7, Pod 1, India English	/in/en/snippets/sku/CHROMELEON7/pod-1
CHROMELEON7, Pod 2, India English	/in/en/snippets/sku/CHROMELEON7/pod-2
CHROMELEON7, Pod 1, US English (base)	/us/en/snippets/sku/CHROMELEON7/pod-1
CHROMELEON7, Pod 1, German	/de/de/snippets/sku/CHROMELEON7/pod-1
NEWPRODUCT123, Pod 1, US English	/us/en/snippets/sku/NEWPRODUCT123/pod-1

7.2 URL Format Comparison

	AEM (Current)	EDS + DA (Target)
--	---------------	-------------------

Commerce calls	<code>/ {locale} /product -family- syndicated- content.pfpsnippe t.html/sku/{SKU}/ {POD}</code>	<code><aem-embed url="/ {locale} /sn ippets/sku/{SKU}/ pod-{N}"></code>
Example	<code>/in/en/product- family- syndicated- content.pfpsnippe t.html/sku/CHROME LEON7/3</code>	<code>url="/in/en/snipp ets/sku/CHROMELEO N7/pod-3"</code>
Content identifier	Tag: <code>pfpsku/CHROMELEO N7/pod_3</code>	Path: <code>/in/en/snippets/s ku/CHROMELEON7/po d-3</code>
Lookup mechanism	JCR TagManager search	Direct URL resolution
Human readable	No	Yes

Brightcove

1. Objective

Integrate Brightcove video playback into the EDS site in a way that is:

- Easy for authors to use (no technical IDs exposed)
- Scalable across multiple accounts and player configurations
- Lightweight for the frontend
- Aligned with Edge Delivery principles
- SEO-optimized with automated  [Schema.org](https://schema.org) - [Schema.org](https://schema.org) metadata

The solution provides a simple authoring experience via a DA plugin, avoids exposing Brightcove technical IDs to authors, supports both video and playlist use cases, and keeps the integration frontend-driven.

2. Architecture

2.1 Integration Layers

Layer	Responsibility
Brightcove Plugin (DA)	Guided authoring — account selection, video/playlist ID entry, display mode. Writes block table to document.
Configuration Sheet	Single source of truth for account-to-ID and player-to-ID mappings. Published as JSON.
Video / Video Playlist Block (EDS)	Reads authored config from block table, fetches config sheet, resolves friendly names to Brightcove IDs, injects Brightcove player dynamically.
Edge Worker	Detects video/playlist blocks, fetches metadata from Brightcove Playback API, injects  Schema.org - Schema.org JSON-LD into page response.

Brightcove CDN	Hosts player JS. Player loads and talks directly to Brightcove backend — no EDS backend integration required.
-----------------------	---

2.2 Design Principles

- **No Brightcove IDs exposed to authors** — authors see friendly names (CBD, PGH, FEI), config sheet resolves to IDs
- **Frontend-driven rendering** — Brightcove player JS loaded from Brightcove CDN, no EDS server-side processing for playback
- **Lazy loading** — player script loaded when block enters viewport (protects Lighthouse scores)
- **Centralized configuration** — one sheet governs all account/player mappings site-wide

3. Configuration Sheet

3.1 Location

The Brightcove configuration sheet is stored at:

```
1 /config/brightcove.xlsx
2
```

Published as JSON at: `/config/brightcove.json`

This location is chosen because:

- `/config/` is the standard EDS location for site-wide configuration
- The sheet is shared across all Video and Video Playlist blocks site-wide
- It's separate from content pages and data sheets
- Authors do not need to access this — it's maintained by the implementation/admin team

3.2 Sheet Structure

Account Name	Account ID	Player Name	Player ID	Player Type	Playlist Align
CBD	3663210762001	Default Player	abc123def	default	
CBD	3663210762001	Playlist with right rail	xyz456ghi	playlist	right-rail

CBD	36632107 62001	Playlist with bottom rail	jkl789mno	playlist	bottom- rail
PGH	66500159 10001	Default Player	pqr012stu	default	
PGH	66500159 10001	Playlist with right rail	vwx345yz a	playlist	right-rail
PGH	66500159 10001	Playlist with bottom rail	9wemoeX 81	playlist	bottom- rail
FEI	88765432 10001	Default Player	bcd678efg	default	
FEI	88765432 10001	Playlist with right rail	hij901klm	playlist	right-rail
FEI	88765432 10001	Playlist with bottom rail	nop234qr s	playlist	bottom- rail

3.3 How It's Used

- **Brightcove Plugin** fetches this sheet to populate the Account dropdown in the authoring dialog
- **Block JS** fetches this sheet at runtime to resolve friendly names to Brightcove account/player IDs

3.4 Maintenance

The config sheet is maintained by the implementation/admin team. Changes include:

- Adding new accounts
- Adding new player configurations
- Updating player IDs when Brightcove players are reconfigured

Changes are a spreadsheet edit — no code deployment needed.

4. Runtime Flow

4.1 Video Block Runtime

```
1 1. Page loads → Block JS detects Video block
2 2. Block reads authored values (account name, video ID, display mode)
3 3. Block fetches /config/brightcove.json
4 4. Resolves: account name → account ID, default player → player ID
5 5. Constructs Brightcove player markup:
6   <video data-video-id="..." data-account="..." data-player="..."
7   ...>
8 6. Injects Brightcove player script from CDN:
9   https://players.brightcove.net/ACCOUNT_ID/PLAYER_ID_default/index.min.
10  js
11 7. Brightcove player initializes and handles playback
```

4.2 Video Playlist Block Runtime

```
1 1. Page loads → Block JS detects Video Playlist block
2 2. Block reads authored values (account name, playlist ID, player type)
3 3. Block fetches /config/brightcove.json
4 4. Resolves: account name → account ID, player type → player ID
5 5. Constructs Brightcove playlist player markup
6 6. Injects appropriate player script
7 7. Brightcove playlist player initializes with selected layout (right-
8   rail or bottom-rail)
```

4.3 Brightcove Player Behavior

Once loaded, the Brightcove player JS communicates directly with Brightcove's backend using the configured account, player, and video/playlist IDs. This means:

- No extra EDS backend integration is required for playback
- No EDS server-side processing is needed
- The Brightcove embed remains fully managed by the frontend block

5. [Schema.org - Schema.org](#) (JSON-LD) via Edge Worker

5.1 Approach

 [Schema.org - Schema.org](#) metadata (VideoObject) is automatically injected into pages containing Brightcove video or playlist blocks via an **Edge Worker**. No manual authoring of  [Schema.org - Schema.org](#) metadata is required.

5.2 Flow

1. Author publishes page with Video/Playlist block
2. HTML is served via Edge Delivery CDN
3. Edge Worker intercepts the response:

- Parses HTML to detect Brightcove blocks
- Extracts account name and video/playlist ID from block markup
- Fetches `/config/brightcove.json` to resolve account ID
- Calls Brightcove Playback API to retrieve video metadata (title, description, thumbnail, duration, upload date)
- Constructs VideoObject JSON-LD
- Injects `<script type="application/ld+json">` into the HTML `<head>`

4. Final response is delivered to the browser with  [Schema.org](https://schema.org) - [Schema.org](https://schema.org) metadata

5.3 Sample JSON-LD Output

```

1 {
2   "@context": "https://schema.org",
3   "@type": "VideoObject",
4   "name": "Video Title",
5   "description": "Video description",
6   "thumbnailUrl": ["https://.../image.jpg"],
7   "uploadDate": "2022-01-25T22:55:01.489Z",
8   "duration": "PT1M9S",
9   "contentUrl": "https://.../video.mp4",
10  "embedUrl":
11    "https://players.brightcove.net/{accountId}/{playerId}_default/index.html?videoId={videoId}"
12 }
```

5.4 Benefits

- Better search engine visibility
- Improved content discoverability
- Compatibility with LLM-based consumption
- Zero authoring effort — fully automated

6. Brightcove Plugin

6.1 Overview

The Brightcove plugin is a DA library panel tool that provides guided authoring for both Video and Video Playlist blocks. It replaces manual block table creation with a dialog-driven experience.

6.2 Plugin Capabilities

Capability	Description
Account selection	Dropdown populated from <code>/config/brightcove.json</code> —

	shows friendly names
Video/Playlist toggle	Author selects whether they're adding a video or playlist
Display mode selection	Dropdown: Inline, Button, Link, Thumbnail, Teaser
Conditional fields	Plugin shows only fields relevant to the selected display mode
Player selection (playlist only)	Dropdown: Default, Right rail, Bottom rail
Block table output	Plugin writes the complete block table with correct variant and rows
Edit existing block	Plugin reads existing block, pre-populates fields, allows update

6.3 What the Plugin Outputs

For Video:

The plugin writes a `Video` or `Video (variant)` block table.

For Playlist:

The plugin writes a `Video Playlist` or `Video Playlist (variant)` block table.

The plugin determines the variant from the Display Mode selection and writes it into the block header automatically.

7. Ownership and Boundaries

Component	Owner	Responsibility
Brightcove Plugin (DA)	Adobe / Implementation Team	Plugin UI, config sheet fetch, block table generation, edit support
Video Block (EDS)	Adobe / Implementation Team	Block JS/CSS, config resolution, Brightcove

		player injection, lazy loading, display mode rendering
Video Playlist Block (EDS)	Adobe / Implementation Team	Block JS/CSS, config resolution, playlist player injection, rail layout rendering
Edge Worker ( Schema.org - Schema.org)	Adobe / Implementation Team	Block detection, Playback API call, JSON-LD construction and injection
Configuration Sheet	Implementation/Admin Team	Account/player mappings, maintained via spreadsheet edits
Brightcove Playback API	TFS / Brightcove	Video metadata for  Schema.org - Schema.org , video/playlist content delivery
Brightcove Player CDN	Brightcove	Player JS hosting, player rendering

EDS Migration Strategy

Content Syndication Pods — Migration Strategy

ACS Commons Generic Lists – Migration Strategy

migration of default offers XF before page scraping

friendly urls/ vanity urls migration

Migration Strategy -- Forms

Forms — Rules Migration Strategy

In AEM, form conditional visibility rules are stored per field in JCR, aggregated server-side, and delivered to the browser as an encrypted payload on the rendered form. In EDS, rules are stored in a single `tfs-form-rules` block in the DA document and evaluated client-side by `tfs-form.js`.

AEM rules reference form fields using auto-generated numeric IDs (e.g. `form-input-field-1958684667`). These IDs are AEM-specific and have no equivalent in EDS. EDS rules reference fields by their authored field name (e.g. `C_TypeofSamples`).

Migration must resolve every numeric ID to its corresponding field name before rules can be written to the EDS `tfs-form-rules` block.

Migration Approach

- AEM live form pages render all conditional visibility rules as an encrypted value (`data-showhide`) in the page HTML at runtime
- A migration script will extract this encrypted value from each live form page and decrypt it to retrieve the complete set of rule definitions for that form — including trigger fields, conditions, operators, and target fields
- The decrypted rules reference form fields using AEM auto-generated numeric IDs. Since EDS does not use auto-generated IDs, the script will resolve each numeric ID to its corresponding authored field name using the same live page, where both the numeric ID and the field name are present together in the rendered HTML
- Once all field references are resolved, the rules are transformed to the EDS `tfs-form-rules` format — numeric IDs replaced with field names, AEM-specific properties dropped, and all rules consolidated into a single block structure
- The transformed rules are written to the corresponding DA document as a `tfs-form-rules` block via the DA API, ready to be consumed by `tfs-form.js` at runtime

constraints	how we will move all constraint sin EDS and is it needed all values ?	

	check if constraints in field values or now to move ?	

- Observed that if we scrape live url of form than if some options are there for country/state which have long fields we will make it local source but ideally they would be dynamic and that path info is not in live it is in jcr.
- When xf are included live page do not have xf path so as TFS has created XF for forms and want to continue using it so when we would be migrating form we need which fragment path to add and that will not be there in live but in jcr
-

Non-functional Requirements

- [DA-SEO](#)
- [EDS Site Performance](#)
- [EDS Accessibility](#)
- [Backup and Recovery](#)
- [Best practices EDS](#)
- [Testing Details](#)

This section captures the platform capabilities and project-specific expectations for **SEO**, **site performance**, and **accessibility** when using **AEM Sites as a Cloud Service with Edge Delivery Services (EDS) and Universal Editor (UE)**.

Supported Browsers

- Internet Explorer is not supported.
- Latest version of modern edge browser Google Chrome, Microsoft Edge, Apple Safari, Mozilla Firefox are supported.

Devices

Supported devices: Mobile, Tablet and Desktop

View Ports

```
/* Mobile-first (phones) */

@media (width < 768px) {

  /* Mobile styles */

}

@media (width >= 768px) and (width < 1200px) {

  /* Tablet & small laptop styles */

}

@media (width >= 1200px) {

  /* Desktop styles */

}
```

DA-SEO

To be updated

Performance as a Ranking Factor

The project relies on Edge Delivery Services' performance-first architecture to support Core Web Vitals that influence search ranking:

- Content is served from the edge with aggressive caching of HTML, assets, and APIs
- Phased rendering and lightweight client-side includes for shared elements (e.g., header/footer) ensure above-the-fold content is delivered quickly
- EDS best-practice guidance ("keeping it 100") is followed for block implementations, images, and fonts

Crawlability & Indexation

- **Sitemaps:** EDS auto-generates XML sitemaps based on published content. Custom sitemap configurations can be managed via the sitemap sheet.
- **Robots.txt:** Managed as a file in the EDS project root. Configurable per environment (preview vs. live).
- **Canonical tags:** Authored per page via metadata. EDS supports canonical URL metadata out of the box.
- **Redirects:** Managed via a redirects spreadsheet (bulk redirect mapping). Supports 301/302 redirects. Critical for migration to preserve link equity from existing URLs.
- **URL structure:** EDS uses clean, folder-based URL paths derived from the content hierarchy. URL structure is defined by the content folder organization in the authoring environment.

On-Page SEO Controls

- **Metadata management:** Page-level metadata (title, description, og:image, canonical, robots directives) is authored per page via the metadata section in Universal Editor. Bulk metadata can also be applied via a metadata spreadsheet for pattern-based defaults.
- **Heading structure:** Authors control heading hierarchy (H1–H6) directly in the content. Block implementations enforce semantic heading usage.
- **Authoring guardrails:** UE component models support restricting available options to enforce SEO best practices (e.g., requiring alt text on images, enforcing single H1 per page). Specific guardrails can be configured based on governance requirements.

Structured Data

Structured data (JSON-LD) can be implemented at the page level via metadata or programmatically via block/script decoration. Common schema types are generated at build/render time based on page content and metadata.

Rendering & Crawlable Content

EDS serves fully rendered, semantic HTML from the CDN — no client-side JavaScript is required for content rendering. All page content, text, links, and images are present in the initial HTML response. This ensures full crawlability by all search engines without reliance on JavaScript execution.

Media Optimization

- **Alt text:** Authored per image in Universal Editor. Required field enforcement can be configured in the block model.
- **Image optimization:** EDS automatically serves images in modern formats (WebP/AVIF) with responsive srcset attributes and appropriate sizing.
- **File naming:** Image file names are preserved from the authored asset name, supporting descriptive, keyword-relevant naming.
- **Lazy loading:** Images below the fold are lazy-loaded by default. Above-the-fold images (e.g., hero, LCP) are eagerly loaded.

Internal Linking

Internal links are authored directly in content via Universal Editor. Navigation structures (header, footer, sub-nav, breadcrumb) provide consistent internal linking across the site. Content authors are responsible for maintaining contextual internal links within page content.

Migration & Launch SEO Strategy

- **Redirect mapping:** Redirects are managed via a spreadsheet in EDS, supporting bulk 301/302 redirects. Any retired, consolidated, or restructured URLs will be mapped to their new destinations to preserve link equity.
- **SEO validation (pre-launch):** Crawl comparison between reference site and EDS site to verify page coverage, metadata accuracy, redirect completeness, and structured data presence.
- **SEO validation (post-launch):** Monitor Google Search Console for indexation issues, crawl errors, and ranking changes. Compare organic traffic and keyword positions against pre-migration baseline.
- **Staged rollout:** DNS-level cutover can be done per path or folder, allowing SEO impact to be monitored incrementally rather than in a single big-bang migration.

SEO Measurement & Tooling

- **Google Search Console:** Primary tool for monitoring indexation, crawl health, and search performance post-migration.
- **SiteImprove:** Already integrated — used by 200+ authors for broken link detection, SEO checks, and readability scoring. Will run against published EDS URLs.
- **EDS RUM:** Built-in Real User Monitoring tracks Core Web Vitals (LCP, CLS, INP) automatically.
- **KPIs:** Recommended baseline KPIs include organic traffic, indexed page count, crawl error rate, Core Web Vitals pass rate, and keyword ranking stability.

Ongoing Governance

SEO governance is an operational concern shared between the content team and the technical team:

- **Content team:** Responsible for metadata quality, alt text, heading structure, internal linking, and redirect requests.
- **Technical team:** Responsible for sitemap configuration, robots.txt, structured data implementation, and performance maintenance.
- **SiteImprove:** Serves as the ongoing automated QA layer for SEO compliance across published content.

EDS Site Performance

- HTML, media assets, and static resources will be delivered via **Adobe-managed Edge Delivery Services CDN** (or an approved BYO CDN pattern), providing:
 - Global edge caching with automatic invalidation / revalidation on publish.
 - Optimized delivery of images and static assets (compression, modern formats where supported).
 - Support for customer CDN integration patterns where required.
- The implementation will follow **EDS performance best practices** for blocks, client-side code, fonts, and third-party scripts.
- Cloud Manager **Experience Audit** (Lighthouse) and/or EDS pre-flight checks will be used as a gate in the CI/CD pipeline.

We would also Suggest having current KPIs baselined, so that after Implementation we can compare these with parameters.

Key Performance Indicators

What are KPIs?

Key Performance Indicators (KPIs) are **quantifiable measures** that show how effectively we are achieving our business and digital experience objectives.

In our context, KPIs help us measure the impact of improvements to Thermo Fisher’s **digital experience**.

Why do we need a KPI baseline?

A KPI baseline is the “**before**” **picture** – the current performance level *before* we introduce new capabilities, content, or optimizations. It will allow us to compare **before vs. after**.

Our KPI Framework Focus

1. Business KPI

Drives measurable growth through conversions, leads, and customer acquisition.

2. Digital Visibility KPI

Strengthens search presence and content discoverability.

3. Customer Experience KPI

Enhances user engagement and journey quality across all touchpoints

4. Operational Excellence KPI

Ensures a fast, stable, and scalable digital platform

Business KPIs				
Pillar	KPI	Description	Ownership Level	Values
Conversion	Overall Conversion Rate	% of sessions resulting in a conversion	Tertiary	

Conversion	Total Conversions	Total conversions tracked monthly/quarterly	Tertiary	
Lead Generation	Form Submissions	Completed forms submitted by users	Secondary	
Lead Generation	Form Starts	Initiated form interactions	Secondary	
Friction	Form Error Rate	% of form submissions with errors	Secondary	
Friction	Drop-off Rate	% of users abandoning funnel steps	Secondary	
Efficiency	Time to Publish	Time from creation to live	Primary	
Efficiency	Pages per Author	Avg pages published per author	Primary	
Efficiency	Content Volume	Total content produced- Pages and Assets	Primary	
Reusability	Component Reusability	% of reused components	Primary	
Workflow	Approval Cycle Time	Time taken for content approval	Primary	

Content velocity	Average Time to Create a Page	Time to create a standard page	Primary	
Content velocity	Average Time to Update a Page	Time to update an existing page	Primary	
Content velocity	Pages Updated per Year	Average no of pages updated per year	Secondary	

Customer Experience KPIs				
Pillar	KPI	Description	Ownership Level	Values
Behavior	Bounce Rate	% of users leaving after one page	Secondary	
Behavior	Engagement Rate	GA4 metric for meaningful interactions	Secondary	
Behavior	Pages per Session	Average number of pages viewed per visit	Secondary	
Behavior	Avg Session Duration	Time spent per session	Secondary	

Behavior	Avg Time per User	Time spent per unique user	Secondary	
Interaction	CTA Click Rate	% of users clicking CTAs	Secondary	
Device	Mobile vs Desktop Split	Traffic breakdown by device type	Secondary	
Device	Engagement by Device	Engagement metrics segmented by device	Secondary	
Consumption	Page Views per User	Avg pages viewed per user	Secondary	

Digital Visibility KPIs				
Pillar	KPI	Description	Ownership Level	Values
Traffic	Organic Traffic	Visits from search engines	Secondary	
Traffic	Organic Traffic per Page	Avg organic traffic per page	Tertiary	
Visibility	Keyword Rankings	Position of target keywords	Tertiary	

Visibility	Indexed Pages	Pages indexed by search engines	Primary	
Migration Health	Redirect Success Rate	% of successful redirects post-migration	Primary	
Migration Health	Broken Links / Crawl Errors	404 errors and crawl issues	Primary	
Performance	Core Web Vitals	LCP, CLS, INP scores for UX quality	Primary	
Distribution	Top Pages Traffic Share	% of traffic to top pages	Secondary	
Quality	High Bounce Pages	% of pages with high bounce rates	Secondary	
Conversion	Content Conversion Rate	Conversion rate from content pages	Secondary	
Journey	Homepage Path Depth	Avg depth of paths from homepage	Secondary	
Journey	Content → CTA Click Rate	% of CTA clicks from content pages	Secondary	

Contribution	Content → Form Contribution	% of forms initiated from content	Secondary	
--------------	-----------------------------------	---	-----------	--

Operational Excellence KPIs				
Pillar	KPI	Description	Ownership Level	Values
Speed	Page Load Time	Avg time to load a page	Primary	
Speed	Time to First Byte	Server response time	Primary	
Stability	Platform Uptime	% of time site is operational	Primary	
Stability	System Stability	Frequency of crashes/issu es	Primary	
Errors	404 / 403 / 503 Errors	Error types encountered by users	Primary	

We ran a report on the home page using Page Insight and obtained the following results. These data are indicative, averaged from three runs. Results may vary under different conditions. The tool's actual data is the most reliable baseline. Hence, it would be great to get that data from tool.

KPI	Desktop	Mobile	Unit
Largest Contentful Paint (LCP)	2.7	2.7	s

Interaction to Next Paint (INP)	62	95	ms
Cumulative Layout Shift (CLS)	0	0.04	
First Contentful Paint (FCP)	1	1	s
Time to First Byte (TTFB)	0.7	0.6	s
Performance score	84	68	0-100
Accessibility score	79	88	0-100
Best Practices score	96	96	0-100
SEO score	85	85	0-100
First Contentful Paint (FCP)	0.4	2.1	s
Largest Contentful Paint (LCP)	2.5	9.4	s
Total Blocking Time (TBT)	110	80	ms
Cumulative Layout Shift (CLS)	0	0	
Speed Index	1.3	5.7	s

EDS Accessibility

Edge Delivery Services provides Semantic and Accessible markup by OOTB. We will to follow the OOTB approach as well as ensure Accessibility is maintained at the same level as the current .

Details on how the semantic markup is created in Edge Delivery Services by default is documented here: [Markup, Sections, Blocks, and Auto Blocking](#)

Backup and Recovery

Self-Service Restore: Cloud Manager provides a self-service restore process that copies data from Adobe system backups and restores it to its original environment

Types of Backups:

Point-In-Time (PIT): Restores from continuous system backups from the last 24 hours.

Last Week: Restores from system backups in the last seven days, excluding the previous 24 hours

Selective Content Restoration: Options include reinstalling packages, using the Restore Tree function, re-uploading original files, and restoring previous versions of assets.

References:

1.  [Restore Content in AEM as a Cloud Service | Adobe Experience Manager](#)

Best practices EDS

During implementation Adobe will make sure that below best practices are followed, these have to be adhered during authoring as well.

1. **Lean, Semantic Markup**

- Use Standard, Meaningful HTML
- Prefer semantic elements such as header, main, section, article, nav, and footer.
- Avoid unnecessary wrapper div elements.
- Use ARIA only when native HTML semantics are insufficient.
- Keep DOM structure shallow to improve performance and maintainability.

2. **Avoid Deep Fragment Nesting**

- Do not embed full fragments inside other fragments unless strictly required.
- Keep fragments modular, focused, and purpose-driven.

3. **Accessibility by Default**

- Ensure proper heading hierarchy (h1 → h2 → h3).
- Always provide alt text for images.
- Ensure interactive elements are keyboard accessible with visible focus states.

4. **Minimal and Focused JavaScript**

- Prefer Vanilla JavaScript

- Use native browser APIs wherever possible.
- Avoid introducing heavy frameworks for simple interactions.
- Reduce client-side payload to improve performance and stability.

5. Load Strategy

- Use defer or async for non-critical scripts.
- Avoid blocking rendering with synchronous scripts.
- Serve critical assets from the same host when possible.

6. Code Organization

- Keep scripts modular and component scoped.
- Avoid global variables.
- Ensure progressive enhancement where core functionality works without JavaScript.

7. Optimized Asset Delivery

- Images
 - o Use lazy loading for non-critical images.
 - o Use responsive image techniques.
 - o Compress and optimize all assets before deployment.
- Icons
 - o Prefer lightweight SVG usage.
 - o Avoid large icon libraries when only a few icons are needed.
- Performance Monitoring
 - o Maintain performance budgets.

- o Validate performance in pull requests.
- o Monitor Core Web Vitals metrics.

8. **Consistency and Modularity**

- Block-Based Authoring
- Each block must be self-contained.
- Avoid hidden cross-dependencies between blocks.
- Keep styling scoped to the block.

9. **Naming Conventions**

- Use consistent naming methodology such as BEM.

10. **Reusability**

- Prefer composable components over duplicated code.
- Maintain shared utilities carefully to avoid tight coupling.

11. **Code Quality Enforcement**

- ESLint (JavaScript / TypeScript)
- Detect bugs early and maintain consistent coding standards.
- Use a shared base ESLint configuration.
- Integrate ESLint into local development, pre-commit hooks, and CI pipelines.
- Fail CI builds on lint errors.
- Restrict console usage in production builds.

12. **Stylelint (CSS / SCSS)**

- Enforce consistent styling rules.

- Prevent specificity conflicts.
- Use a shared Stylelint configuration.
- Integrate Stylelint into pre-commit hooks, CI pipelines, and editor extensions.
- Limit nesting depth.
- Disallow use of !important.
- Enforce consistent naming patterns.
- Prevent duplicate properties.

13. **Design Tokens for CSS and Cross-Platform Consistency**

- What Are Design Tokens?

Centralized, reusable variables representing design decisions such as colors, typography, spacing, radius, shadows, opacity, and motion.

- Token Principles
 - o Use semantic, intent-based naming.
 - o Separate base tokens from semantic tokens.
 - o Version and document tokens with changelog and migration guidance.
- Implementation Guidelines
 - o Generate CSS variables from token definitions.
 - o Avoid hard-coded values in components.
 - o Maintain tokens in a central repository.
 - o Require design and engineering review for token changes.

14. **CI/CD Quality Gates**

- All pull requests must pass ESLint.
- All pull requests must pass Stylelint.
- Performance benchmarks must be validated.

- Accessibility checks must pass.
- Avoid unnecessary bundle size increases.

15. **Accessibility and Performance Standards**

- Accessibility
 - o Ensure keyboard navigation.
 - o Provide visible focus states.
 - o Maintain proper color contrast.
 - o Use semantic HTML before ARIA.
- Performance
 - o Avoid large JavaScript bundles.
 - o Reduce unused CSS.
 - o Minimize render-blocking resources.
 - o Monitor Core Web Vitals continuously.

16. **Generic Engineering Best Practices**

- Follow progressive enhancement.
- Keep CSS specificity low.
- Avoid over-engineering solutions.
- Prefer clarity over cleverness.
- Document each component's purpose and usage.
- Maintain a clear folder structure.
- Track and manage technical debt.

17. **Quick Implementation Checklist**

- Semantic HTML structure implemented.
- Minimal vanilla JavaScript used.
- Lazy-loaded assets enabled.
- Modular block-based architecture followed.
- ESLint integrated locally and in CI.
- Stylelint integrated locally and in CI.
- Centralized design tokens implemented.
- Accessibility validated.
- CI quality gates enabled.

Testing Details

Testing Strategy

A detailed testing strategy will be prepared during the implementation phase by the Adobe QA team and will cover tooling, test data management, performance benchmarking, environments, and sprint-level execution processes. The sections below define the governing framework for that strategy.

1.1 Scope of Testing

- Validation of all agreed functional requirements against approved acceptance criteria.
- End-to-end validation of integration flows across EDS front-end components and third-party services.
- Sprint-level gate checks so that all deliverables are tested and approved in lower environments before promotion.
- Verification of visual rendering across key page components, layouts, and responsive breakpoints.
- Validation of authoring, publishing, and related operational workflows.
- Post-migration verification to confirm content integrity after transfer.

1.2 Test Types and Approach

Test Type	Approach
Functional Testing	Validation of UI behaviour, authoring workflows, and business rules against defined acceptance criteria.
Integration Testing	End-to-end validation of data and functional flows across EDS front-end components, Adobe ecosystem dependencies, and third-party integrations.
Regression Testing	Verification that existing functionality is not adversely affected by new code changes,

	configuration changes, or defect fixes.
Mobile and Browser Testing	Cross-browser and cross-device validation across current versions of Chrome, Firefox, Safari, and Edge on desktop, Android, and iOS.

2. QA Team Involvement

The Adobe QA team will be engaged throughout the delivery lifecycle rather than only at the point of release.

Phase	Adobe QA Team Activities
Implementation and Sprint Testing	QA participates in sprint ceremonies. Test cases are prepared in parallel with development once stories are groomed and prioritised. Test execution begins after deployment to the QA environment, and defects are logged and tracked through the agreed delivery process.
UAT and Release	Following sprint completion, the QA team conducts regression and sanity testing in the relevant pre-production environment before handing over for business-led UAT and final release readiness checks.

3. Validation Approach

3.1 QA Execution Process

- Test cases are prepared in parallel with development throughout each sprint.
- Test cases are reviewed and approved before execution.

- Code is promoted to the QA environment only after successful CI pipeline checks.
- Each test case is executed with supporting evidence recorded for pass or fail outcome.
- Defects are logged, triaged, and tracked to closure or formal deferral.
- A sprint cycle is considered complete only when planned test execution is complete and all defects are either resolved or formally accepted into the backlog.

3.2 Defect Gating

Promotion between environments will be governed by the following defect-gating criteria:

Promotion Gate	Gating Criteria
Dev → QA	All CI pipeline checks pass, including successful build and unit test execution.
QA → UAT	No open Critical or High defects. Medium and Low defects are either resolved or formally accepted into the backlog.
UAT → Production	No open Critical or High defects. Medium defects are triaged jointly with the client and resolved before go-live unless formally accepted. Low defects may be moved to the backlog.

4. Parity Validation Framework

Parity validation confirms that the migrated solution is correct, complete, stable, and functionally equivalent to the legacy platform for all agreed business-critical scenarios. The objective is to confirm equivalence against the agreed baseline, not to redesign scope during migration.

Validation will be assessed across the following layers:

Validation Layer	What Is Tested
Content Transfer Parity	Presence and accuracy of migrated content. Automated and manual

	methods will be used to compare source and target content at scale.
Functional Parity	Business-critical user journeys including authoring, preview, publishing, search, forms, navigation, and third-party integrations.
Experience Parity	Page rendering, SEO metadata, analytics instrumentation, and accessibility behaviour compared with the agreed legacy baseline.
Operational Parity	Permissions, user and group configurations, and workflow behaviour aligned to the target operating model.

4.1 Acceptance Criteria for Parity Sign-off

- No open Critical defects.
- Content migration validation evidence is on record for the release wave.
- UAT is complete for all priority user journeys in scope for the release wave.
- Formal sign-off is received from the designated Thermo Fisher stakeholder for the release wave.

5. Coexistence Strategy

The migration will operate in a controlled coexistence model so that **T Thermo Fisher Scientific - US** remains stable while migrated sections are progressively released on EDS.

- **T Thermo Fisher Scientific - US** remains the single public brand domain.
- Legacy AEM 6.4 remains the live source for unmigrated paths during transition.
- EDS becomes the live source only for migrated and signed-off scope.
- Authors continue to manage approved migrated content in AEM as a Cloud Service using Universal Editor.

- Shared integrations must operate across both the legacy and new delivery layers for the duration of coexistence.

5.1 Content and Authoring During Coexistence

Scope	Authoring / Source of Truth	Public Delivery
Unmigrated pages	Legacy AEM 6.4	Legacy AEM 6.4
Migrated wave in validation	AEMaaCS + EDS	EDS validation URL only, for internal validation
Migrated wave after go-live	AEMaaCS + EDS	EDS on the public domain

Next-Level Detail During Implementation

The detailed artefacts that will be produced during implementation include the full Testing Strategy, parity validation matrix, Go-Live Checklist, Cutover Runbook, wave-level dependency register, and launch-day smoke test scripts. These documents will convert the governing framework in this response into execution-level procedures.